



Workshop
Proceedings

Human Computation Workshop 2010

July 25, 2010 | Washington DC, USA

ACM SIGKDD

Human Computation Workshop 2010

Workshop Proceedings

Washington DC, USA

July 25, 2010

Preface

Research in data mining and knowledge discovery relies heavily on the availability of datasets. However, compared to the amount of work in the field on techniques for pattern discovery and knowledge extraction, there has been relatively little effort directed at the study of effective methods for collecting and evaluating the quality of data.

Human computation is a relatively new research area that studies the process of channeling the vast internet population to perform tasks or provide data towards solving difficult problems that no known efficient computer algorithms can yet solve. There has been a lot of work on games with a purpose (e.g., the ESP Game) that specifically target online gamers who, in the process of playing an enjoyable game, generate useful data (e.g., image tags). There has been considerable interest and research on crowdsourcing marketplaces (e.g. Amazon Mechanical Turk), which are essentially human computation applications that coordinate workers to perform tasks in exchange for monetary rewards.

While there have been significant research challenges, increasing business interest and active work in human computation, till last year there was no dedicated forum to discuss these ideas. The first Human Computation Workshop (HComp2009) was held on June 28th, 2009, in Paris, France, collocated with KDD 2009. With HComp2010, we hope to continue to serve the needs of researchers and practitioners interested in this area, integrating work from a number of fields including KDD, information retrieval, gaming, machine learning, game theory, and Human Computer Interaction. Learning from HComp 2009, we expanded the topics of relevance to the workshop. We have also updated the organizing and program committees, bringing new people in, while keeping enough of the last year's team for continuity.

This year, our call for papers resulted in 35 submissions (18 long papers, 11 short papers and 6 demos) from a range of perspectives. All submissions were thoroughly reviewed, typically with 3 reviews each, by the organizing and program committees and external reviewers. Even though we had a number of submissions of high quality, given that we had only a half-day workshop, we could accept only 4 long papers, 4 short papers, 5 posters and 5 demos; these are the submissions that appear in the proceedings.

The overall themes that emerged from last year's workshop were very clear: on the one hand, there is the experimental side of human computation, with research on new incentives for users to participate, new types of actions, and new modes of interaction. On the more theoretic side, we have research modeling these actions and incentives to examine what theory predicts about these designs. Finally, last year's program focused on how to best handle noise, identify labeler expertise, and use the generated data for data mining purposes. This year's submissions demonstrated the continuation and evolution of these themes with the accepted papers divided into three sessions: "Market Design", "Human Computation in Practice", and "Task and Process Design". In the first session, the authors discuss the economic context of human computation and the practical lessons learned from building a human computation resource. In the second session we see many practical examples of human computation in fields as far-flung as virtual world construction to language translation and beyond. In the final session, the authors focus on how to get the best quality results out of the crowdsourced work. Given the strong submissions for demos both last year and this, we see -- and the community seems to recognize -- the poster and demo session as an integral part of the workshop, where participants can showcase their human computation applications.

An undertaking like this workshop takes a large effort from many of talented people. We would like to thank the KDD 2010 workshop committee and chairs, the authors, the program committee members, the external reviewers, and contributors for their support of this workshop.

**Raman Chandrasekar, Ed Chi, Max Chickering, Panagiotis G Ipeirotis,
Winter Mason, Foster Provost, Jenn Tam, Luis von Ahn**

Organizing Committee

Table of Contents

Preface ... iii

HCOMP2010 Organization ... viii

Invited Talk

Using Games to Improve Productivity in Software Engineering

Ross Smith (Microsoft Corporation)

Session 1: Market Design

Task Search in a Human Computation Market ... 1

Lydia Chilton (University of Washington)

John Horton (Harvard University)

Robert Miller (MIT CSAIL)

Shiri Azenkot (University of Washington)

The Anatomy of a Large-Scale Human Computation Engine ... 10

Shailesh Kochhar (Metaweb Technologies)

Stefano Mazzocchi (Metaweb Technologies)

Praveen Paritosh (Metaweb Technologies)

Sellers' problems in human computation markets ... 18

M. Six Silberman (University of California, Irvine)

Joel Ross (University of California, Irvine)

Lilly Irani (University of California, Irvine)

Bill Tomlinson (University of California, Irvine)

Session 2: Human Computation in Practice

Frontiers of a Paradigm -- Exploring Human Computation with Digital Games ... 22

Markus Krause (University of Bremen)

Aneta Takhtamysheva (University of Bremen)

Marion Wittstock (University of Bremen)

Rainer Malaka (University of Bremen)

GiveALink Tagging Game: An Incentive for Social Annotation ... 26

Li Weng (Indiana University)

Filippo Menczer (Indiana University)

**Crowdsourcing Participation Inequality:
A SCOUT Model for the Enterprise Domain ... 30**

Osamuyimen Stewart (IBM Research)
David Lubensky (IBM Research)
Juan M. Huerta (IBM Research)
Julie Marcotte (IBM GBS)
Cheng Wu (IBM Research)
Andrzej Sakrajda (IBM Research)

Mutually Reinforcing Systems ... 34

John Ferguson (University of Glasgow)
Marek Bell (University of Glasgow)
Matthew Chalmers (University of Glasgow)

A Note on Human Computation Limits ... 38

Paul Rohwer (Indiana University)

**Reconstructing the World in 3D:
Bringing Games with a Purpose Outdoors ... 41**

Kathleen Tuite (University of Washington)
Noah Snaveley (Cornell University)
Dun-Yu Hsiao (University of Washington)
Adam Smith (UC Santa Cruz)
Zoran Popovic (University of Washington)

Improving Music Emotion Labeling Using Human Computation ... 45

Brandon G. Morton (Drexel University)
Jacquelin A. Speck (Drexel University)
Erik M. Schmidt (Drexel University)
Youngmoo E. Kim (Drexel University)

Webpardy: Harvesting QA by HC ... 49

Hidir Aras (University of Bremen)
Markus Krause (University of Bremen)
Andreas Haller (University of Bremen)
Rainer Malaka (University of Bremen)

Measuring Utility of Human-Computer Interaction ... 53

Michael Toomim (University of Washington)
James A. Landay (University of Washington)

Translation by Iterative Collaboration between Monolingual Users ... 54

Chang Hu (University of Maryland)
Benjamin B. Bederson (University of Maryland)
Philip Resnik (University of Maryland)

Session 3: Task and Process Design

Sentence Recall Game: A Novel Tool for Collecting Data to Discover Language Usage Patterns ... 56

Jun Wang (Syracuse University)

Bei Yu (Syracuse University)

Word Sense Disambiguation via Human Computation ... 60

Nitin Seemakurty (Carnegie Mellon University)

Jonathan Chu (Carnegie Mellon University)

Luis von Ahn (Carnegie Mellon University)

Anthony Tomasic (Carnegie Mellon University)

Quality Management on Amazon Mechanical Turk ... 64

Panagiotis G. Ipeirotis (New York University)

Foster Provost (New York University)

Jing Wang (New York University)

Exploring Iterative and Parallel Human Computation Processes ... 68

Greg Little (MIT)

Lydia B. Chilton (University of Washington)

Max Goldman (MIT CSAIL)

Robert C. Miller (MIT CSAIL)

Toward Automatic Task Design: A Progress Report ... 77

Eric Huang (Harvard University)

Haoqi Zhang (Harvard University)

David C. Parkes (Harvard University)

Krzysztof Z. Gajos (Harvard University)

Yiling Chen (Harvard University)

Author Index ... 86

HCOMP2010 Organization

Organizing Committee

Raman Chandrasekar, Microsoft Research, USA
Ed Chi, Xerox PARC, USA
Max Chickering, Microsoft, USA
Panagiotis G Ipeirotis, New York University, USA
Winter Mason, Yahoo! Research, USA
Foster Provost, New York University, USA
Jenn Tam, Carnegie Mellon University, USA
Luis von Ahn, Carnegie Mellon University, USA

Program Committee

Serge Belongie, University of California at San Diego, USA
Paul Bennett, Microsoft Research, USA
Sheng-Wei (Kuan-Ta) Chen, Academia Sinica, Taiwan
Ling-Jyh Chen, Academia Sinica, Taiwan
Laura Dabbish, Carnegie Mellon University, USA
Ralf Herbrich, Microsoft Research, Cambridge, UK
Jane Hsu, National Taiwan University, Taiwan
Markus Krause, University of Bremen, Germany
Edith Law, Carnegie Mellon University, USA
Hao Ma, Chinese University of Hong Kong, Hong Kong
Gabriele Paolacci, University of Venice, Italy
David Parkes, Harvard University, USA
Zoran Popovic, University of Washington, USA
Paul Resnick, University of Michigan, USA
Victor Sheng, University of Central Arkansas, USA
Alexander Sorokin, University of Illinois at Urbana-Champaign, USA

Additional Reviewers

John Horton, Harvard University, USA
Shaili Jain, Harvard University, USA
Haoqi Zhang, Harvard University, USA

Task Search in a Human Computation Market

Lydia B. Chilton
University of Washington
hmslydia@cs.washington.edu

Robert C. Miller
MIT CSAIL
rcm@mit.edu

John J. Horton
Harvard University
horton@fas.harvard.edu

Shiri Azenkot
University of Washington
shiri@cs.washington.edu

ABSTRACT

In order to understand how a labor market for human computation functions, it is important to know how workers search for tasks. This paper uses two complementary methods to gain insight into how workers search for tasks on Mechanical Turk. First, we perform a high frequency scrape of 36 pages of search results and analyze it by looking at the rate of disappearance of tasks across key ways Mechanical Turk allows workers to sort tasks. Second, we present the results of a survey in which we paid workers for self-reported information about how they search for tasks. Our main findings are that on a large scale, workers sort by which tasks are most recently posted and which have the largest number of tasks available. Furthermore, we find that workers look mostly at the first page of the most recently posted tasks and the first two pages of the tasks with the most available instances but in both categories the position on the result page is unimportant to workers. We observe that at least some employers try to manipulate the position of their task in the search results to exploit the tendency to search for recently posted tasks. On an individual level, we observed workers searching by almost all the possible categories and looking more than 10 pages deep. For a task we posted to Mechanical Turk, we confirmed that a favorable position in the search results do matter: our task with favorable positioning was completed 30 times faster and for less money than when its position was unfavorable.

Categories and Subject Descriptors

H.3.3 [Information Storage and Retrieval]: Information Search and Retrieval; J.4 [Social and Behavioral Sciences]: Economics

General Terms

Economics, Experimentation, Human Factors, Measurement

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

KDD-HCOMP'10, July 25th, 2010, Washington, DC, USA.
Copyright 2010 ACM 978-1-4503-0222-7 ...\$10.00.

Keywords

Search, Amazon Mechanical Turk, Human Computation, Crowdsourcing, Experimentation

1. INTRODUCTION

In every labor market, information plays a critical role in determining efficiency: buyers and sellers cannot make good choices unless they know their options. According to a rational economic model of labor supply, workers use information about the availability and the nature of tasks to make decisions about which tasks to accept. If workers lack full information about the tasks, they are likely to make sub-optimal decisions, such as accepting inferior offers or exiting the market when they would have stayed, had they known about some other task. In large markets with many buyers and sellers, lack of knowledge about all available options is a key source of friction. This lack of knowledge stems from the fact that searching is neither perfect nor costless.

The “search problem” is particularly challenging in labor markets because both jobs and workers are unique, which means that there is no single prevailing price for a unit of labor, nevermind a commodified unit of labor. In many labor markets, organizations and structures exist to help improve the quality and quantity of information and reduce these “search frictions.” We have already seen conventional labor markets augmented with informational technology, such as job listing web sites like Monster.com [2], and yet a great deal of information is privately held or distributed through social networks and word of mouth.

In online labor markets, there is relatively little informal sharing of information as in traditional markets. Given that online labor markets face no inherent geographic constraints, these markets can be very large, further exacerbating search frictions. These frictions can and are eased by making the data searchable through a variety of search features. If agents can sort and filter tasks based on characteristics that determine the desirability of a task, they are able to make better choices, thereby improving market efficiency. Of course, this hypothetical gain in efficiency depends upon the search technology provided and the responses of buyers to that technology.

Amazon’s Mechanical Turk (MTurk) is an online labor market for human computation. It offers several search features that allow workers to find Human Intelligence Tasks (HITs). HITs can be sorted by fields such as most available, highest reward, and title. HITs are also searchable by keyword, minimum reward, whether they require qualifica-

tions (and combinations thereof). In this paper, we seek to provide insight into how workers search for tasks.

We present the results of two methods for investigating search behavior on MTurk. First, we scrape pages of available HITs at a very high rate and determine the rate at which a type of HIT is being taken by workers. We use a statistical model to compare the rate of HIT disappearance across key categories of HITs that MTurk allows workers to search by. Our premise for observing search behavior is that search methods which return HITs with higher rates of disappearance are the search methods which workers use more. This method relies only on publicly available data—the list of HITs available on www.mturk.com. Second, we issued a survey on MTurk asking over 200 workers how they searched for tasks. We posted the survey with carefully chosen parameters so as to position it so that it can be found more easily by some search methods and not others. This way, we can target search behaviors that are not picked up by the analysis of the scraped data.

In this paper, we first motivate studying search behavior in an online human computation labor market by reviewing related work. We next describe MTurk’s search features and our method of data collection based on these search features. We then present a statistical analysis of our high-frequency scraped data followed by the results of a survey of MTurk workers.

2. RELATED WORK

In the field of human computation, it is important to ask “Why are people contributing to this effort?” What are the motivations behind labeling an image [17], uploading videos to YouTube [11], or identifying the genre of a song [13]? On MTurk, the main motivation seems to be money [12, 10]. However, financial motivations do not imply that workers are only motivated to find HITs offering the highest rewards: workers may choose from among thousands of HITs that differ in required qualifications, level of difficulty, amount of reward, and amount of risk. With all of these factors at play, workers face significant decision problems. Our task was to model this decision problem.

Because MTurk is a labor market, labor economics—which models workers as rational agents trying to maximize net benefits over time—may provide a suitable framework. Research on the labor economics of MTurk has shown that workers do respond positively to prices but that price is not indicative of work quality [14]. Previous work [9] shows that not all workers follow a traditional rational model of labor supply when choosing their level of output. Instead of maximizing their wage rate, many workers create target earnings—they focus on achieving a target such as \$.25 or \$1.00 and ignore the wage rate. Although price is important to task choice, it is clearly not the only factor; research on search behavior is necessary in order to more fully understand how workers interact with the labor market.

In other domains where users must cope with a large information space, progress and innovation in search have had enormous positive impacts. Developments such as Page-Rank [4] have made it possible to search the long tail of large, linked document collections by keyword. Innovations such as faceted browsing [8] and collaborative filtering [7] have made it faster and easier to search within more structured domains such as shopping catalogues. Web queries are sensitive to changes in content over time, and searchers

have an increasing supply of tools to help them find updates on webpages they revisit [1]. In labor and human computation markets, search tools are still evolving and it remains to be determined what features will prove most useful in overcoming search frictions.

Web search analysis has shown that the first result on the first page of search results has by far the highest click-through rate [16]. In a market model where parties have full information, there would be no need for search technologies—no purely “positional effects”—all parties would instantly and costlessly find their best options. In reality, search technologies are likely to have strong effects on the buyer-seller matches that are made. This point is evident by the fact that companies are willing to spend billions of dollars to have advertisements placed in prime areas of the screen [5].

While there are similarities between many kinds of online search activities, we must be careful in generalizing. Search behavior in one domain does not necessarily carry over to other domains. In this paper we focus solely on the domain of workers searching for HITs on MTurk, using web search behavior as a guide to what features may be important.

3. MTURK SEARCH FEATURES

At any given time, MTurk will have on the order of 100,000 HITs available. HITs are arranged in HIT groups. All HITs posted by the same person with the same features (title, reward, description, etc) are listed together in a HIT group to avoid repetition in the list of available HITs. HIT groups are presented much like traditional web search engine results, with 10 HIT groups listed on each page of search results. By default, the list is sorted by “HITs Available (most first).” The default view gives seven pieces of information:

1. Title (e.g., “Choose the best category for this product”)
2. Requester (e.g., “Dolores Labs”)
3. HIT Expiration Date (e.g., “Jan 23, 2011 (38 weeks)”)
4. Time Allotted (e.g., “60 minutes”)
5. Reward (e.g., “\$0.02”)
6. HITs Available (e.g., 17110)
7. Required qualifications (if any)

By clicking on a HIT title, the display expands to show three additional fields:

1. Description (e.g., “Assign a product category to this product”)
2. Keywords (e.g., categorize, product, kids)
3. Qualifications. (e.g., “Location is US”)

The MTurk interface also offers several search features prominently positioned at the top of the results page, including a keyword search and a minimum reward search. Additionally, HITs can be sorted in either ascending or descending order by six categories:

1. HIT Creation Date (*newest* or *oldest*)
2. HITs Available (*most* or *fewest*) (i.e., how many sub-tasks may be performed)

3. Reward Amount (*highest* or *lowest*)
4. Expiration Date (*soonest* or *latest*)
5. Title (*a-z* or *z-a*)
6. Time Allotted (*shortest* or *longest*)

Some sort results change more quickly than others. Sorting by most recent creation date will produce a very dynamic set of HIT groups. However, sorting by most HITs Available will produce a fairly static list because it takes a long time for a HIT group that contains, say, 17,000 HITs to fall off the list.

On each worker’s account summary page, MTurk suggests 10 particular HITs, usually with high rewards but no other apparent common characteristic. Other than this, we are unaware of any services to intelligently recommend HITs to workers. We presume that nearly all searching is done using the MTurk search interface. A Firefox extension called Turkopticon [15] helps workers avoid tasks posted by requesters that were reported by other users as being unfair in their view. This could affect workers’ choices of which tasks to accept, but probably has little effect on the category workers choose to sort by and does not reorder or remove HITs from the results pages.

Requesters post tasks on MTurk and pay workers for acceptable work. There are two basic types of HITs that encompass most postings: (1) tasks such as image labeling, where workers are invited to perform as many various tasks as are available to be completed, and (2) tasks such as surveys, which require many workers but allow each worker to do the HIT only once. Type (2) HIT groups appear to workers as single HIT because there is only one HIT available to each worker. If a worker does not accept the HIT, it will remain on his list of available HITs until the total number of performances desired by the requester are completed. It often takes a substantial amount of time to achieve the throughput of workers necessary to complete type (2) HITs where the task only appears as a single available HIT to each worker. Because it is a small wonder they those HITs get done at all, we call them “1-HIT Wonders.”

For HIT groups that allow individual workers to perform a single task multiple times (type (1)), the number of “HITs Available” is constantly updated. The number can increase in only two cases: first, when a HIT is returned by a worker unperformed and, second, in the rare event that more tasks are added to the HIT group before it is finished. Fortunately, this second case is easy to detect.

In this paper, we use HIT disappearance rates to perform a statistical analysis of search behavior for type (1) HIT groups containing multiple tasks available to all workers. We complement our analysis with the results of a survey where workers were asked about how they search for tasks such as 1-HIT Wonders that the scraping does not address.

4. METHOD A: INFERENCES FROM OBSERVED DATA

We want to determine whether the sort category, the page on which a HIT appears (page placement), and the position it occupies on that page (page position) affect the disappearance rate of a HIT. Our premise is that if any of these factors affect the disappearance rate of a HIT, then workers

are using that search feature to find HITs. This approach makes use of what economists call a “natural experiment”.

To understand it, consider the following thought experiment: Imagine that we could randomly move HITs to different pages and page positions within the various search categories without changing the attributes of the HITs. With this ability, we could determine the causal effects of HIT page placement and page position by observing how quickly a HIT is completed. Of course, we do not actually possess the ability to manipulate search results in this way for HITs we do not post, but we are able to observe the functioning of the market and potentially make a similar inference based on the scraped data.¹

The problem with this approach, however, is that HIT characteristics that are likely to affect the popularity of a HIT should also affect its page placement and page position within the search results of the various sorting categories. In fact, the relationship between attributes and search results page placement and page position is the whole reason MTurk offers search features—if search results page placement and page position were unrelated to HIT characteristics that workers cared about, then any search category based on those characteristics would offer no value.

To deal with the problem of correlation between attributes and page placement and page position, we needed to make several assumptions and tailor our empirical approach to the nature of the domain. Before discussing our actual model, it will be helpful to introduce a conceptual model. We assume that the disappearance of a particular HIT during some interval of time is an unknown function of that HIT’s attributes and its page placement and page position (together, its position in the list of search results):

$$\text{Disappearance of HIT } i = F(\text{position of } i, X_i)$$

where X_i are all the HIT attributes, market level variables, and time-of-day effects that might also affect disappearance rate. Our key research goal was to observe how manipulations of the position of i —while keeping X_i constant—affect our outcome measure, the disappearance of HITs (i.e., work getting done). We were tempted to think that we could include a sufficient number of HIT co-variables (e.g., reward, keywords, etc.) and control for the effects of X_i , but this is problematic because we cannot control for all factors.

Our goal of untangling causality was complicated by the fact that we cannot exogenously manipulate search results and the fact that there are HIT attributes for which we cannot control. Our approach was to acknowledge the existence of unobserved, HIT-specific idiosyncratic effects but to assume that those effects are constant over time for a particular HIT group. With this assumption, we relied upon movement in HIT position within a sort category as a “natural experiment.” When we observed a HIT moving from one position to another, and then observed that the HIT disappeared more quickly in the new position, we attributed the difference in apparent popularity to the new position, and not to some underlying change in the nature of the HIT.

4.1 Data Collection

There are 12 ways in which MTurk allows workers to sort HITs—six categories, each of which can be sorted in ascend-

¹In Method B, we will actually manipulate our HITs position in the search results (albeit by modifying the HIT attributes).

ing or descending order. We scraped the first three pages of search results for each of the 12 sorting methods. Each page of results lists 10 hits, which meant that we scraped 360 HITs in each iteration. We scraped each of the 36 pages approximately every 30 seconds—just under the throttling rate. Although we ran the scraper for four days, due to our own data processing constraints, in this analysis we used data collected in a 32-hour window beginning on Thursday, April 29, 2010, 20:37:05 GMT and ending on Saturday, May 1, 2010, 04:45:45 GMT. The data contains 997,322 observations. Because of the high frequency of our scraping, each posted HIT is observed many times in the data: although the data contains nearly 1,000,000 observations, we only observed 2,040 unique HITs, with each HIT observed, on average, a little more than 480 times.

For each HIT, we recorded the 10 pieces of information that the MTurk interface offers (seven default and three additional features—see Section 3), as well as the sort category used to find the HIT, the page placement of the HIT (1, 2 or 3), and the page position of the HIT (first through tenth).

4.1.1 Measuring HIT disappearance

Formally, let s index the sort category, let g index the groups of observations of the same HIT (which occur because each HIT is observed many times), and let i index time-ordered observations within a HIT group. Let HITs be ordered from oldest to most recently scraped, such that $i + 1$ was scraped after i . Let the number of HITs available for observation i be y_{igs} . The change is simply $\Delta y_{igs} = y_{(i+1)gs} - y_{igs}$.

Unfortunately, there are several challenges in treating this outcome measure as a direct measure of uptake. First, requesters can add and delete tasks in a HIT group over time. If a requester deletes a large number of tasks, then a regression might incorrectly lead us to believe that there was something very attractive to workers about the page placement and/or page position of the HIT. Second, for 1-HIT Wonders, the HITs-available measure might not change even though the HIT is very popular. Finally, the absolute change is “censored” in that the number of HITs available cannot go below zero, which means that HIT groups with many tasks available are able to show greater changes than HIT groups with few tasks available. For these reasons, we make our outcome variable an indicator for a drop in HITs available, regardless of the magnitude of the change:

$$Y_{igs} = 1 \cdot \{\Delta y_{igs} < 0\}$$

4.2 Econometric set-up

To implement our conceptual model, we assumed a linear regression model in which the expected outcome is a linear function of a series of variables. Obviously, many factors determine whether or not a HIT disappears from search results. Page placement, page position, and sort category certainly matter, but so do variables that are hard to observe and which we cannot easily include in a regression. For example, a comparatively high-paying, short task that looks fun will probably be accepted more quickly than one that is low-paying, long, and tedious. In creating our model, we sought to separate the desirability of a task from the page placement and page position it occupies in the search results of a particular sorting category.

4.2.1 Choosing a model

Rather than simply introduce a number of controls that will invariably miss factors that are unmeasured, we included a HIT group-specific control in the regressions called a “group random effect.” Each HIT group was modeled as having its own individual level of attractiveness that stays constant over time. In this way, all factors that have a static effect on HIT attractiveness were controlled for, regardless of whether or not we are able to observe the factors that determine a HIT groups attractiveness.

With a group-specific random effect, we eliminate one of the key problems that would arise from simply using a pooled model. For example, a fun HIT that always stays on the third page of results sorted by highest reward might cause us to erroneously believe that the third page is very important, when the case is simply that one HIT group is disappearing quickly.

One necessary word on the nomenclature of these two models: when dealing with data that has a natural group structure, a model with a group specific-effect is called a “random effects” model or “multilevel” model; a model that ignores the group structure and does not include any group-specific effects is called a “pooled” model. Both types are linear regression models. While our group-specific effect model “controls” for group factors, we do not actually include a dummy variable for each group (which is called the “fixed effects” model). Rather, we start with a prior assumption that each effect is a random draw from a normal distribution (the random effects model) [6]. As the number of observations increases, this fixed effects/random effects distinction becomes less important.

4.2.2 Model

The group-specific random effects model is as follows: for an observation i of group g , we estimate

$$Y_{igs} = \sum_{r=1}^{30} \beta_s^r x_{igs}^r + \gamma_g + \tau_{H(i,g)} + \epsilon \quad (1)$$

where $x_{ig}^r = 1$ if the HIT i is at position r (and $x_{ig}^r = 0$ otherwise) and where $\gamma_g \sim N(0, \sigma_\gamma^2)$ is the group-specific random effect and $\tau_{H(i)} \sim N(0, \sigma_\tau^2)$ is a time-of-day random effect. The pooled model ignores this grouped structure and imposes the constraint that there is no group effect and $\gamma_g = \gamma = 0$.

4.3 Results

We applied our models to four of MTurk’s 12 sorting options: *newest* HITs, *most available* HITs, *highest reward*, and *title a-z*. The others—including *shortest* time allotted and *latest expiration*—tend to produce 1-HIT Wonders and don’t seem like natural ways to produce HITs.

In Figure 1, the collection of coefficients, β_s^r , from Equation 1 are plotted for both the group-specific random effects model (panel (a)) and the pooled model (panel (b)), with error bands two standard errors wide. The four sorting options are listed at the top of the figure: *newest*, *most*, *highest reward*, *a-z*. The points are the coefficients (with standard error bars) for each page and position on that page, ranging from one to 30 (positions are displayed as negative numbers to preserve the spatial ordering). The coefficients $\hat{\beta}_s^r$ are interpretable as the probability that a HIT occupying position r decremented by one or more HITs during the scraping

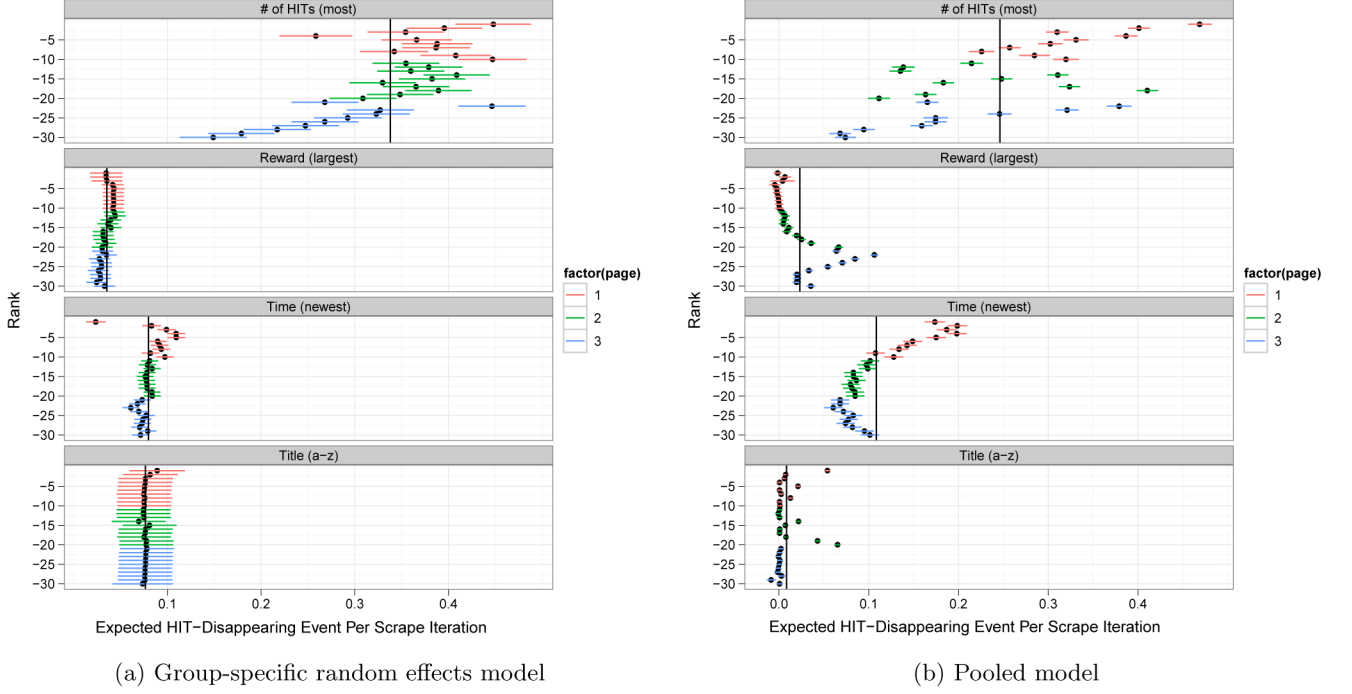


Figure 1: Effects of position in sort category on HIT disappearance. Pages are illustrated with color. Horizontal lines represent one standard error on either side.

interval. Our premise was that sort categories, page placements, and page positions with higher probability of the HIT decrement are those that workers search by.

4.4 Group random effects model

Under the assumptions of the group-specific random effects model, we interpret the position coefficients as the probability that a HIT occupying a particular page position will have a disappearance event during the scraping interval. The only sort category that shows strong positional effects is *most*, with similar rates for the first and second results pages, and then a strong drop-off on the third page.

From this, we conclude that workers actively sort by the *most* available HITs and look at results on the first two pages of search results. On the third page of *most* available HITs, the average coefficient is high, but position effects seem to matter. We conjecture this to mean that the further down the third page the HIT is located, the more likely workers are to abandon this search strategy. However, more investigation is needed in order to confirm this interpretation.

The page placements and page positions generated by the *highest reward* sorting category have no apparent effect on search behavior. The overall levels of disappearance for HITs among the search results are low, which is to be expected given the relative unpopularity of high-reward HITs. Also unsurprisingly, the page placements and page positions generated by the *title a-z* sorting category have no apparent effect on search behavior.

4.5 Pooled model

The group-specific random effects results shown in panel (a) of Figure 1 are more credible than those of the pooled

model in panel (b). Nevertheless, panel (b) coefficients are far more precise. This is because panel (a) coefficients are determined solely by within-group movements in search result position. For sorting categories that do not show much movement—such as *title*—the panel (a) estimates are comparatively imprecise. However, the panel (b) estimates only capture the innate attractiveness of whatever HIT was occupying a particular position. For example, at $r = 20$ in panel (b), *title*, we see a strong statistically significant effect, but this is presumably only a feature of whatever HIT was occupying that position.

This “stationary HIT” bias issue also arises in panel (b), *reward*: on the first page of search results for *highest*, there is almost zero probability of the HIT disappearing. It may seem surprising that high-reward tasks aren’t being taken, but this fact can be readily explained. Most MTurk HITs have a reward between \$0.02 and \$1.00. However, there are some HITs that get posted for larger sums, on the order of \$10.00. There are very few such HITs and they tend to be unpopular, so they stay on the *highest reward* page for a long time without disappearing. We do see that on the second and third results pages, the probability of non-zero disappearance rate goes up.

4.5.1 Searching by newest HITs?

According to panel (a), the search results position within *newest* HITs is generally either irrelevant, or, in the case of position 1, page 1, is actually very harmful! This *is* surprising. Sorting by the *newest* HITs seems to be a very good way to find fresh and interesting tasks. Further, as our survey results (described below) show, most workers report searching for HITs based *newest*.

Interestingly, *newest* is a sort category where we would expect the inclusion of group effects to be irrelevant, since position in the *newest* sort category should be essentially mechanical—a HIT group is posted, starts off at the first position, and then moves down the list as newer HIT groups are posted. Yet, in comparing *newest* across the two panels, the effects of position are radically different: in panel (b), we see that position has a strong effect on uptake, with the predicted pattern, while in panel (a), there appears to be little of no effect. To summarize, the pooled model suggests strong effects, while the random effects model suggests no effects.

4.5.2 Discrepancy

We hypothesized that there were no positional effects in the group-specific random effects model because certain requesters actively game the system by automatically re-posting their HIT groups in order to keep them near the top of *newest*. This hypothesis reconciles two surprising findings.

First, gaming explains the absence of *newest* effects in panel (a): the benefits to being in a certain position are captured as part of the group effect. To see why, consider a requester that can always keep his HIT at position 1. No matter how valuable position 1 is in attracting workers, all of this effect will be (wrongly) attributed to that particular HIT’s group-specific effect.

Second, gaming explains why position 1 appears to offer far worse performance in panel (a) (and why there is a generally increasing trend in coefficient size until position 4). A very large share of the HITs observed at position 1 in the data came from gaming requesters. A still large but relatively smaller share of the HITs observed at position 2 came from gaming requesters who were trying to get their HIT groups back into position 1, and so on. Because all the purely positional benefits get loaded onto the group effects of gaming requesters, positions more likely to be occupied by gaming requesters will appear to offer smaller positional benefits.

Gaming not only reconciles the data—it is also directly observable. We identified the HITs that had 75 or more observations in position 1 of *newest*. In Figure 2, the position (1–30) is plotted over time with the HIT group ID and title posted above. It was immediately clear that some HITs are updated automatically so that they occupy a position near the top of the search results: until day 0.5, the HIT groups in the second and third positions were “competing” for the top spot; around day 0.9, they faced competition from a third HIT group (first panel) that managed to push them down the list.

Under these gaming circumstances, the group-specific random effects model is inappropriate, as the positions are not randomly assigned. For this reason, the pooled model probably offers a clearer picture of the positional effects, though the coefficient estimates are certainly biased because they incorporate the nature of HITs occupying position one, which we know is from gaming.

We conclude that the pooled regression results are the correct analysis for the *newest* HITs category. The pooled regression results show that workers are actively searching by *newest* HITs and focus on the first page of results.

5. METHOD B: WORKER SURVEY

Another way to study how workers search for HITs is to



Figure 2: Position of three HITs over time sorted by *newest*. (Time measured in days)

ask the workers directly, by conducting a survey on MTurk. Although this approach is more expensive, more obtrusive, and smaller-scale than scraping the MTurk web site, it has two advantages. First, surveying complements the scraping technique. Scraping is better suited to tracking postings of large numbers of type (1) HITs whose tasks can be performed multiple times by any worker, so an anonymous scraper can observe their acceptance. Surveys, by contrast, are necessarily 1-HIT Wonders, offered at most once to every worker, and MTurk offers no way for a scraper to observe how many workers have accepted the offer. Since a substantial use of MTurk involves 1-HIT Wonders for online experimentation and surveys of other kinds, we need a method for exploring search behavior on this side of the market. Second, surveys can provide more detailed and subjective explanations for worker behavior than aggregate statistics.

This section presents the results of a survey of roughly 250 workers about how they sort and filter HITs. Since MTurk was used to conduct the survey, the characteristics of the HIT used to solicit survey respondents have an effect on the results. We explored this effect by posting the survey in several different ways in order to sample workers who exhibited different search behaviors. In the process, we observed how certain choices of HIT characteristics can make a HIT very hard to find, substantially reducing the response rate. Finally, the survey also collected free-form comments from workers about how they find good and bad HITs.

5.1 Procedure

The survey was designed to probe respondents’ immediate search behavior—specifically, how respondents used MTurk’s search interface to discover the survey HIT itself. The survey asked several questions: (1) which of the 12 sort categories they were presently using; (2) whether they were filtering by a keyword search or minimum price; and (3) what page number of the search results they found the survey HIT on. A final question asked the worker for free-form comments about how easy or hard it is to find HITs.

The survey was posted on MTurk in four ways, using different HIT characteristics for each posting to give it high and low positions in the various sorted search results. Aside

from these manipulated parameters, the four postings were identical (e.g. in title, description, preview, and actual task).

Best-case posting. In one posting, the parameters of the HIT were chosen to place it among the top search results under each of the six primary sort categories. Thus the HIT would appear on the first page, or as close as possible, when HITs were sorted by *any* attribute, though only in one direction (ascending or descending). The goal of this posting was to capture as many workers as possible from each of the six chosen sort orders that the posting optimized.

The HIT was automatically one of the *newest*, at least at first, and had the *fewest* possible HITs available because it was only offered once to each worker. For reward, we chose the *least* amount: \$0.01. For the *soonest* expiration date and the *shortest* time allotted, we chose 5 hours and 5 minutes, respectively. Finally, we optimized for alphabetical title order, *a-z*, by starting the title with a space character.

The best-case posting was also labeled with the keyword “survey”, which we knew from pilot studies was frequently used by turkers to find desirable survey HITs. The other three postings had the same title and description as the best-case posting, but no “survey” keyword.

Worst-case posting. To test the impact of especially poor position on a HIT’s reachability, we also posted the survey in a way that was *hard to find* as possible for workers using any sorting category. This was done by choosing parameter values near the median value of existing HITs on MTurk, so that the survey HIT would appear in the middle of the pack, as far as possible from the first page in both ascending and descending order by that attribute.

For example, at the time the survey was posted, the median number of HITs available was two. Postings with two HITs covered pages 55–65 (out of 100 pages of HITs requiring no qualifications) when the list was sorted by *fewest* HITs available, and pages 35–45 when sorted by *most* HITs. As a result, a worker sorting in either direction would have to click through at least 35 pages to reach any 2-HIT posting, which is highly unlikely. We therefore posted the survey with two independent HITs.

For the remaining attributes, we chose a reward amount of \$0.05 and an expiration period of one week and allotted 60 minutes in order to position the HIT at least 20 pages deep among search results sorted by reward amount, creation date, and time allotted, in both ascending and descending order. The median title of all current HITs started with “Q,” so we started the survey title with the word “Question” to make it hard to find whether sorting by *a-z* or *z-a*.

Creation date, however, cannot be chosen directly. In particular, a HIT cannot be posted with a creation date in the past. Freshly posted HITs generally appear on the first page of the *newest* sort. In order to artificially age our survey HIT before making it available to workers, we initially posted a nonfunctional HIT, which presented a blank page when workers previewed it or tried to accept it. After six hours, when the HIT had fallen more than 30 pages deep in the *newest* sort order, the blank page was replaced by the actual survey, and workers who discovered it from that point on were able to complete it.

Newest-favored and a-z-favored posting. We performed two additional postings of the survey, which were intended to favor users of one sort order while discouraging other sort orders. The *newest-favored* posting was like the worst-case posting in all parameters except creation date.

It appeared functional immediately, so users of the *newest* sort order would be most likely to find it. Similarly, the *a-z-favored* posting used worst-case choices for all its parameters except its title, which started with a space character so that it would appear first in *a-z* order.

All four postings were started on a weekday in May 2010, with their creation times staggered by 2 hours to reduce conflict between them. The best-case posting expired in 5 hours, while the other three postings continued to recruit workers for 7 days.

5.2 Results

Altogether, the four postings recruited 257 unique workers to the survey: 70 by the best-case posting (in only 5 hours), 58 by the worst-case posting, 59 by newest-favored, and 70 by a-z-favored (all over 7 days). Roughly 50 workers answered more than one survey posting, detected by comparing MTurk worker IDs. Only the first answer was kept for each worker.

Figure 3 shows the rate at which each posting recruited workers to the survey over the initial 24-hour period. The best-case posting shows the highest response rate, as expected, and the worst-case posting has the lowest. The newest-favored posting shows a pronounced knee, slowing down after roughly 1 hour, which we believe is due to being pushed off the early pages of the *newest* sort order by other HITs. Because the posting’s parameters make it very hard to find by other sort orders, it subsequently grows similarly to worst-case. The a-z-favored posting, by contrast, recruits workers steadily, because it remains on the first page of the a-z sort order throughout its lifetime. These results show that HIT parameters that affect sorting and filtering have a strong impact on the response rate to a HIT. All four postings were identical tasks with identical descriptions posted very close in time, and the best-case posting actually offered the lowest reward (\$0.01 compared to \$0.05 for the others), but its favorable sort position recruited workers roughly 30 times faster than the worst-case posting.

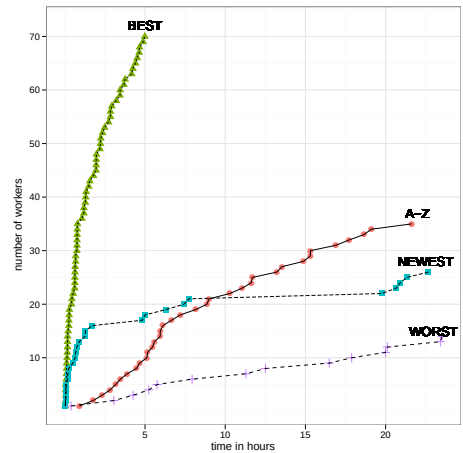


Figure 3: Number of workers responding to each survey posting within the first 24 hours of posting (time measured in days)

Figure 4 shows the responses to the survey question about sorting category, grouped by the posting that elicited the re-

sponse. The best-case posting recruited most of its workers from the *newest* sort order, reinforcing the importance of this sort order for 1-HIT wonders like a survey. The *newest* sorting category also dominated the responses to the newest-favored posting, as expected, and *a-z* appeared strongest in the *a-z*-favored posting. Strong appearance of *most* HITs was a surprise, however, since each posting had only 1 or 2 HITs.

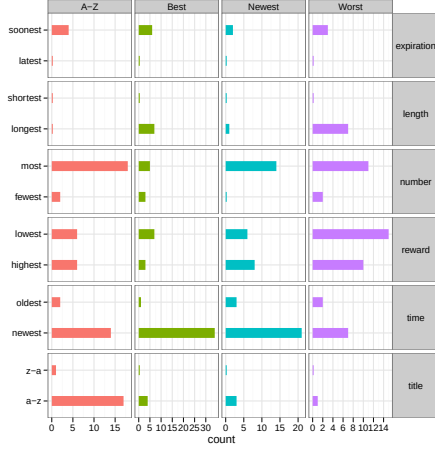


Figure 4: Number of workers who reported using each sort order to find each of the four postings.

For the survey question about reward amount filtering, roughly 65% of the survey respondents reported that they had not set any minimum amount when they found the survey HIT. Surprisingly, however, 9% reported that they had used a reward filter that was *greater* than the reward offered by the survey HIT. For example, even though the best-case posting offered only \$0.01, ten respondents reported that they found the survey while searching for HITs that paid \$0.05 or more. We followed up by sending email (through MTurk) to several of these workers, and found that the problem can be traced to a usability bug in MTurk: “Once we submit a hit MTurk takes us back to all hits available, 1st page with no \$ criteria.” In other words, a filter is discarded as soon as the worker performs a single task. Several other workers mentioned similar problems in their free-form comments.

Similarly, roughly 20% of the workers reported using a keyword filter when they found the survey, but half of these reported using keywords that were not actually included in the survey HIT, suggesting that the filter they thought they were using was no longer in effect. 21 workers reported using the “survey” keyword, of whom 16 were responding to the best-case posting, which actually did include that keyword.

Figure 5 shows the distribution of result page numbers on which workers reported finding the survey HIT. Roughly half found it on the first or second page of results, as might be expected. Yet a substantial fraction (25%) reported finding the survey beyond page 10. Because of the way the survey was posted, particularly the worst-case posting, workers would not have been likely to find it anywhere else. Yet it is still surprising that some workers are willing to drill dozens of pages deep in order to find tasks.

The free-form comment question generated a variety of

feedback and ideas. Many respondents reported that they would like to be able to search for or filter out postings from particular requesters. Other suggestions included the ability to group HITs by type, and to offer worker-provided ratings of HITs or requesters. Several workers specifically mentioned the burden of clicking through multiple pages to find good HITs:

“Scrolling through all 8 pages to find HITs can be a little tiring.”

“I find it easier to find hits on mechanical turk if I search for the newest tasks created first. If I don’t find anything up until page 10 then I refresh the page and start over otherwise it becomes too hard to find tasks.”

Several workers pointed out a usability bug in MTurk that makes this problem even worse:

“It is difficult to move through the pages to find a good HIT because as soon as you finish a HIT, it automatically sends you back to page 1. If you are on page 25, it takes so much time to get back to page 25 (and past there).”

“Please keep an option to jump to a page directly without opening previous pages.”

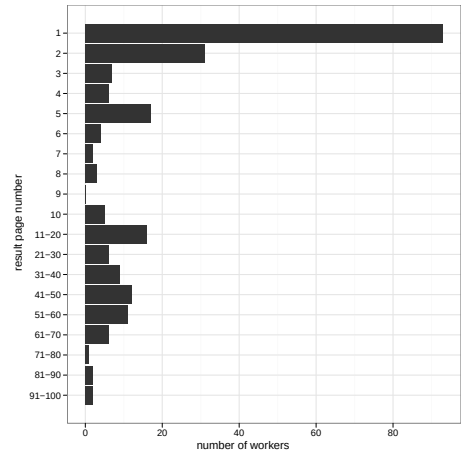


Figure 5: Histogram of result page numbers on which workers found the survey HIT. Most found it on page 1, but many were willing to drill beyond page 10.

6. CONCLUSION

In this paper, we studied the search behavior of MTurk workers using two different methods. In our first method, we scraped pages of available tasks at a very high rate and used the HIT disappearance rates to see if certain sorting methods resulted in greater task acceptance.

Because this method does not accurately measure all types of HITs, we also posted a survey on MTurk asking workers how they search for our survey task. We posted the task with specially chosen posting parameters complementary to those of the HITs we gathered information on using the scraper.

Both methods show that workers tend to sort by *newest* HITs. The scraping revealed that workers also sort by *most* HITs and focus mainly on the first two pages of search results but ignore the positions of the HITs on the page. The survey data confirms that even though *newest* and *most* are the most popular search strategies, the other sort categories are also used—even relatively obscure ones such as *title a-z*

and *oldest* HITs, but to a much lesser extent. The survey confirms that the first two pages of the search results are most important, but also shows evidence that some workers are willing to wade fairly deep into search results pages and to periodically return to the *newest* HITs results.

7. FUTURE WORK

Method A does not account for keyword searching or sorting by required qualifications. The Method A empirical results provide insight into how workers search, but it would be useful to develop a structural model of search behavior that incorporated the quantity of work being done could make predictions.

8. ACKNOWLEDGMENTS

John thanks the NSF-IGERT Multidisciplinary Program in Inequality & Social Policy for generous financial support (Grant No. 0333403). Thanks to Marc Grimson for help running the survey and to Greg Little and Jaime Teevan for feedback. Thanks to Robin Y. Horton, Carolyn Yerkes and three anonymous reviewers for very helpful comments. All plots were made using `ggplot2` [18] and all multilevel models were fit using `lme4`[3].

9. REFERENCES

- [1] E. Adar, M. Dontcheva, J. Fogarty, and D. Weld. Zoetrope: interacting with the ephemeral web. In *Proceedings of the 21st annual ACM symposium on User interface software and technology*, pages 239–248. ACM, 2008.
- [2] D. Autor. Wiring the labor market. *Journal of Economic Perspectives*, 15(1):25–40, 2001.
- [3] D. Bates and D. Sarkar. lme4: Linear mixed-effects models using S4 classes. URL <http://CRAN.R-project.org/package=lme4>, R package version 0.999375-28, 2008.
- [4] S. Brin, L. Page, R. Motwami, and T. Winograd. The PageRank citation ranking: bringing order to the web. In *Proceedings of ASIS'98*, pages 161–172, 1998.
- [5] B. Edelman, M. Ostrovsky, and M. Schwarz. Internet advertising and the generalized second-price auction: Selling billions of dollars worth of keywords. *The American Economic Review*, pages 242–259, 2007.
- [6] A. Gelman and J. Hill. *Data analysis using regression and multilevel/hierarchical models*. Cambridge University Press Cambridge, 2007.
- [7] D. Goldberg, D. Nichols, B. M. Oki, and D. Terry. Using collaborative filtering to weave an information tapestry. *Commun. ACM*, 35(12):61–70, 1992.
- [8] M. Hearst, A. Elliott, J. English, R. Sinha, K. Swearingen, and K. Yee. Finding the flow in web site search. *Communications of the ACM*, 45(9):49, 2002.
- [9] J. Horton and L. Chilton. The labor economics of paid crowdsourcing. *Proceedings of the 11th ACM Conference on Electronic Commerce 2010 (forthcoming)*, 2010.
- [10] J. J. Horton, D. Rand, and R. J. Zeckhauser. The Online Laboratory: Conducting Experiments in a Real Labor Market. *NBER Working Paper w15961*, 2010.
- [11] B. A. Huberman, D. Romero, and F. Wu. Crowdsourcing, attention and productivity. *Journal of Information Science (in press)*, 2009.
- [12] P. Ipeirotis. Demographics of mechanical turk. *New York University Working Paper*, 2010.
- [13] E. Law, L. Von Ahn, R. Dannenberg, and M. Crawford. Tagatune: A game for music and sound annotation. In *International Conference on Music Information Retrieval (ISMIR'07)*, pages 361–364. Citeseer, 2003.
- [14] W. Mason and D. J. Watts. Financial incentives and the ‘performance of crowds’. In *Proc. ACM SIGKDD Workshop on Human Computation (HCOMP)*, 2009.
- [15] M. Silberman, J. Ross, L. Irani, and B. Tomlinson. Sellers’ problems in human computation markets. 2010.
- [16] A. Spink and J. Xu. Selected results from a large study of Web searching: the Excite study. *Inform. Resear.—Int. Electron. J*, 6(1), 2000.
- [17] L. Von Ahn and L. Dabbish. Labeling images with a computer game. In *Proceedings of the SIGCHI conference on Human factors in computing systems*, pages 319–326. ACM, 2004.
- [18] H. Wickham. ggplot2: An implementation of the grammar of graphics. *R package version 0.7*, URL: <http://CRAN.R-project.org/package=ggplot2>, 2008.

The Anatomy of a Large-Scale Human Computation Engine

Shailesh Kochhar
Metaweb Technologies
kochhar@metaweb.com

Stefano Mazzocchi
Metaweb Technologies
stefano@metaweb.com

Praveen Paritosh
Metaweb Technologies
paritosh@metaweb.com

ABSTRACT

In this paper we describe RABJ¹, an engine designed to simplify collecting human input. We have used RABJ to collect over 2.3 million human judgments to augment data mining, data entry, and curation tasks at Freebase over the course of a year. We illustrate several successful applications that have used RABJ to collect human judgment. We describe how the architecture and design decisions of RABJ are affected by the constraints of *content agnosticity*, *data freshness*, *latency* and *visibility*. We present work aimed at increasing the yield and reliability of human computation efforts. Finally, we discuss empirical observations and lessons learned in the course of a year of operating the service.

1. INTRODUCTION

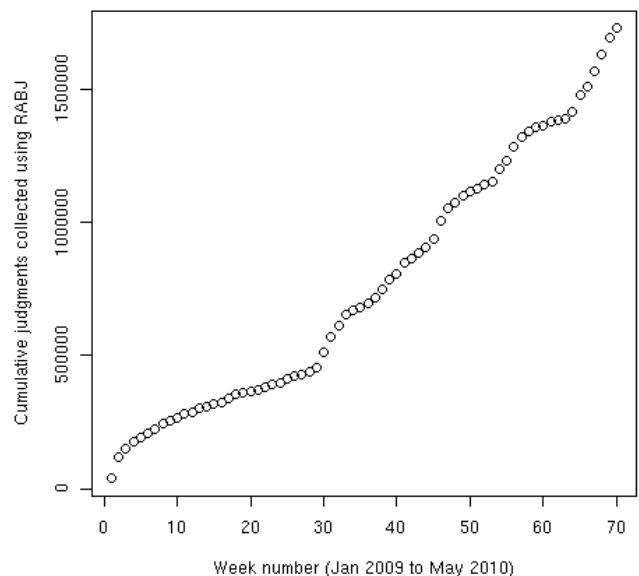
At Freebase we are building the world's largest structured database of entities and the relationships between them. The data in Freebase is collaboratively created, structured, and maintained [1]. Freebase currently contains more than 300 million triples across 12 million topics which span more than 17,000 types and 46,000 properties. A significant fraction of this data comes from learning and data mining processes which operate on semi-structured data sources such as Wikipedia.

We are in the business of finding, sifting through and organizing structured data. Automated processes form the backbone of our efforts and require human vetting to ensure that we meet our 99% accuracy standards. This quality assurance step verifies that data load processes have not compromised the integrity of a data set and will not reduce the quality of our database. From our experience, timely turn-around for these tasks is critical, empowering our data engineers to iterate rapidly and course-correct. We turn here to human computation or crowdsourcing.

RABJ is a question answering service designed to integrate into our data processing pipelines and simplify human oversight of our machine driven processes. Over the course of

one year we have collected 2.3 million judgments on 1.7 million unique questions which span over 300 different tasks (see Fig. 1). These tasks range in nature from contribution to curation, validation and reconciliation. 555 volunteers from the Freebase user community have made 1.05 million judgments while 84 paid contractors have made 1.25 million judgments. In its year of operation, 6 person-years of work has been accomplished through RABJ at approximately 18 seconds per judgment.

Figure 1: Cumulative Judgments in RABJ



RABJ is built around the paradigm of real-time queues which are populated with questions. Data flows through the various systems using a HTTP based REST-ful API. In subsequent sections we describe some of the applications developed using RABJ, the components of our architecture and the entities they manipulate, the constraints which RABJ attempts to satisfy and the life cycle of a question. We further describe the evolution of the RABJ ecosystem and the development of peripheral tools augmenting the services made available by RABJ in Section 5. Finally, we report our observations and learnings and present areas which we believe offer the potential to amplify the usefulness of human com-

¹Redundant Array of Brains in a Jar

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

KDD-HCOMP '10, July 25, 2010, Washington, DC USA
Copyright 2010 ACM 978-1-4503-0222-7 ...\$10.00.

putation.

This paper contributes to the community a description of our implementation of a general purpose system for gathering human input. Second, we present empirical findings from our experience using RABJ as a human computation engine.

2. PREVIOUS WORK

Previous work in this area includes the work on Games with a Purpose [8], [10], [9], [6], which have demonstrated the use of collecting common-sense human judgments to solve problems which are hard to solve computationally. Sheng, Provost and Ipeirotis [5] investigate the use of multiple judgments for a questions and demonstrate how to effectively use the judgments collected. Snow et al [7] consider the relationship between the cost of judgments and the quality of the output produced.

3. RABJ APPLICATIONS

A variety of applications which use the RABJ platform have been developed at Metaweb. Here we illustrate the use of some applications as part of a process to add a large dataset of books and authors to Freebase. Consider Victor who is developing an automated process which distills data about 2 million books from an external library and weaves it into Freebase. He has developed a suite of tools that help in data-cleaning, matching the entities in his dataset with their Freebase equivalents and finally writing it all to the database. At several stages in this process Victor will use RABJ to provide oversight and input to his tools.

3.1 Inspection

Before jumping headlong into the task Victor samples a small portion of the dataset and submits it to an inspection application. Figure 2 shows the contents of a book edition from the OpenLibrary database. This task asks users to verify that the data is consistent and to ensure that was no mistake in the process which extracted and normalized the information.

Figure 2: Book Edition QA
Book Edition QA - Filler Editions Sample

Undo Is this book edition information correct?

Yes No Skip

The Duelling Handbook, 1820 (Dover Books on History, Political and Social Science) by Joseph Hamilton

Edition of: [The Duelling Handbook, 1820](#)

Pagination: [Dover Publications](#) 46p

Publisher: [Dover Publications](#) 1907

ISBN-13: [9780486214510](#)

Status: Created

Search: [Amazon](#) [Google](#) [Source Data](#)

Instructions

Each box on the left represents a Book Edition topic in Freebase that was created or modified by the OpenLibrary book edition tool. Please examine the data for any errors or inconsistencies and report them using the comment box. You should use the links provided to check this data against external sources. Here are some things to look out for:

- **Edition of** field should represent the book this is an edition of. It is OK for the book title to be slightly different than the edition title (e.g. missing a leading "The" or the full subtitle starting from a "?") as long as they are obviously the same book.
- When filled in, **Publisher** should be a Freebase topic for a publisher of books, and the right publisher for this edition.
- **Pagination** should be the correct number of pages for this edition.
- **Publication** should be valid for this edition.
- **ISBN-13** should be valid for this edition.

Status: Created vs. Existed

Please pay special attention to the Status field, which indicates whether the book edition or simply reconciled to an existing one. This is a potential source of error, and in the event of a bad reconciliation, you may see modified data.

3.2 Evaluation

As Victor develops techniques to link the records in his dataset with Freebase he needs oversight to evaluate how his algorithms are doing. Humans excel at *reconciliation* i.e. understanding if two pieces of information describe the same concept. For instance, determining that the OpenLibrary page about Eugene O'Neill describes the same concept as the Freebase topic about a person who was born on 16 October 1888 in New York is a reconciliation task.

Victor generates candidate pairings between the entities in his dataset and their likely Freebase counterparts. These pairings are submitted to the MATCHMAKER2 application (see Fig. 3) for evaluation. MATCHMAKER2 shows the target resource and the candidate topics most likely to relate to the same concept. Judges are asked to select one of the candidates (or indicate that no candidate was found appropriate). Victor uses the judgments to determine how well his classification strategy is doing. At the time of writing, Over 290K pairs have been judged through MatchMaker2.

3.3 Contribution

Most of the data in Freebase comes from automated processes (computer programs). There are many cases where human operators can easily and accurately find an answer while automated processes cannot deliver a confident result. This is quite like the common sense data collection done by games with a purpose [8], [9].

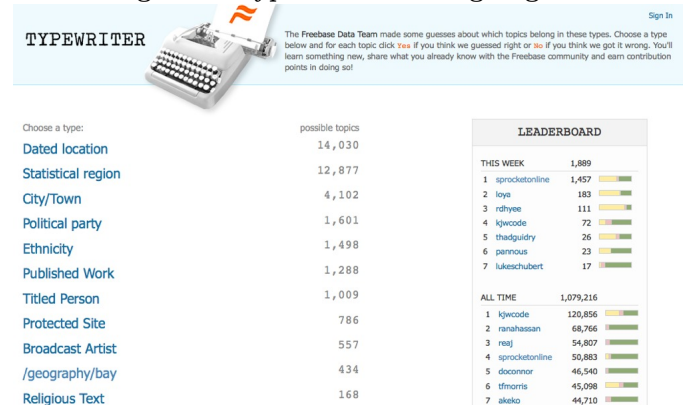
For instance, Victor believes that the automated process he has developed is working well and the evaluation step confirms his belief. However, there are still some parts of the dataset which could not be classified automatically. In this case Victor uses MATCHMAKER2 to manually match entities against Freebase. Judges are asked to find matches instead of evaluating candidates.

Below we describe some of the other applications developed for the purposes of eliciting user contributions to Freebase.

Typewriter

Our most successful RABJ application so far is Typewriter which asks judges to determine whether or not a given topic is part of a given class (for example, is Tom Cruise an actor?). This application alone received more than 1 million contributions over 14 months with the top contributor casting more than 120k judgments and the top 10 contributors casting half of the total judgments.

Figure 4: Typewriter Landing Page



Geographer

Geographer (Fig 5) is an application that helps people add or validate geographical coordinates associated to Freebase topics considered Locations. The applications makes it easy to enter this data by simply placing a pin on a map to acquire the numerical latitude and longitude values.

Figure 3: The MatchMaker2 Application

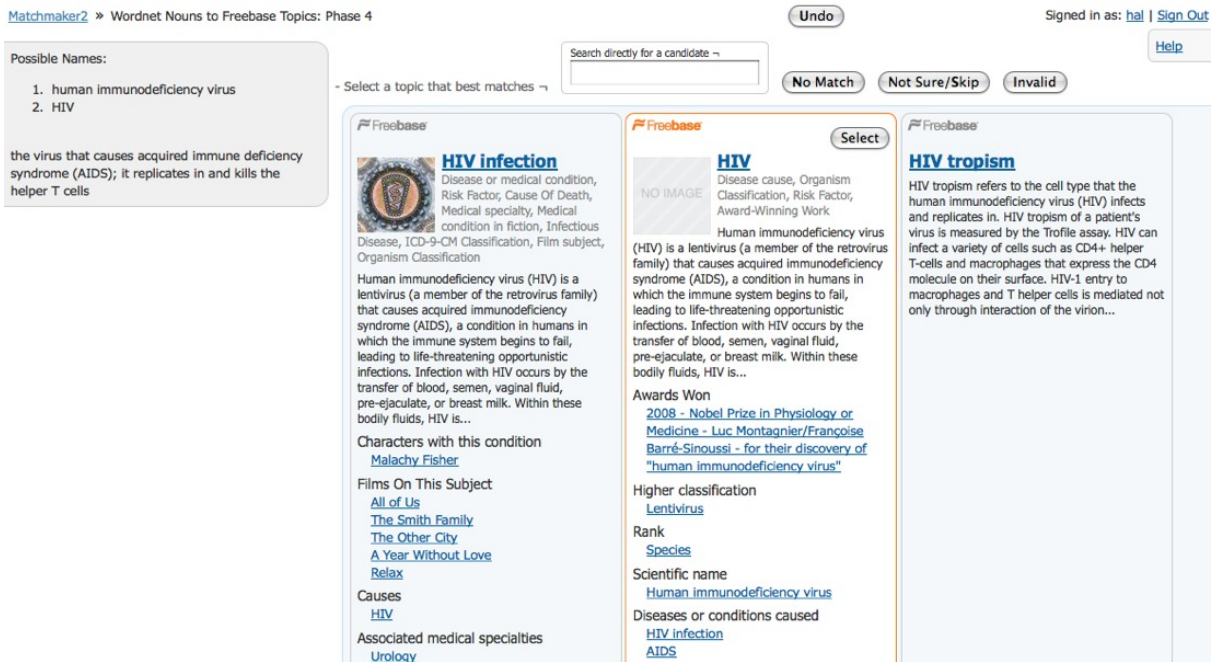
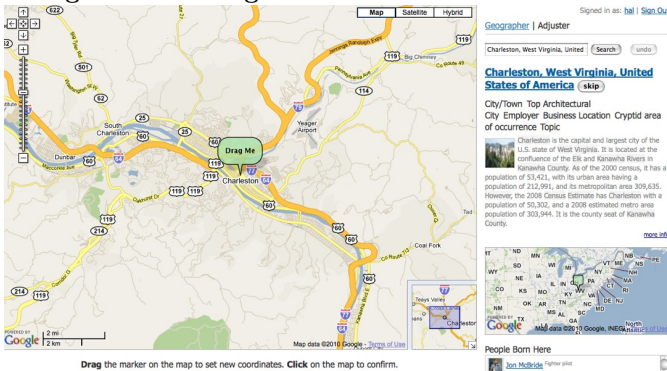


Figure 5: Finding Geo Coordinates for Places



Genderizer

Another successful application is Genderizer, which asks judges to evaluate the gender of Freebase topics which have been typed as a person, a fictional character or a notable animal, either by looking at a picture, or reading a description of the topic (which often contains indicating keywords like he, she, her, his, etc). This application added gender to roughly 135k topics.

4. SYSTEM DESCRIPTION

In this section we describe the entities, constraints and components that have guided the design of RABJ towards a flexible, repurposable system:

4.1 Entities

RABJ is organized around three kinds of entities: questions, judgments and queues.

Questions

A question is the smallest unit of work within the RABJ system. A question is typically a new assertion or some potential data which requires input from a human judge. Typical examples are factual assertions to be verified, multiple choice questions or fill in the blank questions. For instance, we may want to confirm Douglas Adams' pet raccoon's name with a question like, "Was Douglas Adams' pet raccoon named Beeble?"

Questions can also be tagged, which allows sub-dividing a queue along a variety of dimensions. For instance, the Typewriter application (Fig 4) tags questions by the possible type-categories and allows judges to focus on categories in which they are most interested.

Judgments

Judgments are the responses collected from judges along with some metadata describing the response. E.g. Yes, No or I Don't Know. The space of available responses varies according to the needs of the requester. Many applications provide binary voting mechanisms, while others expect more open ended input. For instance, the question, "Find the Freebase topic which best matches this Wikipedia topic." requires a specific Freebase topic identifier as a response.

Queues

Queues organize sets of questions and bind questions to applications. RABJ applications use queues as the primary basis by which to fetch questions, present them to human judges and collect responses. Queues are ordered which allows requesters to overlay priority on questions.

Queues specify default completion criteria for questions, e.g. number of votes required per question which can be overridden on a per-question basis. Queues also provide a container for declarative metadata which controls appli-

cation behavior. Requesters can configure their queues to limit the maximum number of votes or limit the individual response types. E.g. Stop after two *I Don't Know* votes. A few RABJ applications, notably MATCHMAKER2 (see Fig. 3), are in fact general purpose applications whose specific features are toggled based on flags present in the queue.

4.2 Constraints

RABJ was conceived as a general purpose question answering service to augment and enhance all our data processing abilities including data curation, entry, mining and verification. This imposes some constraints on the design of the system:

Content Agnostic

In order to be generally applicable to the problem of collecting human judgments RABJ is agnostic to the content of the question being asked. This is crucial to allow RABJ to support the diversity of use cases we encounter. RABJ imposes no structural requirements on questions treating them as a simple bag of properties. The downside is that RABJ services can make very few decisions based on the specific content of a question. We externalize content-specific actions back to RABJ applications.

Late-Binding of Data

A second need is for questions to reflect the latest data available to Freebase. In many cases, questions might be submitted many hours or days before they are viewed by a person. It is possible that in the intervening time an unrelated process has changed the underlying data or rendered the question moot. Consequently, RABJ clients are encouraged to represent their questions as structured references that can be expanded against the state of Freebase at the time when the question is answered. This of course comes at the cost of increased rendering times for the end-user applications.

Latency

As has been demonstrated in several contexts across the web [3], [2], adding a few hundred milliseconds to page load times impacts user engagement. Above we discuss how the dynamic nature of RABJ questions increases the time required to render them. Furthermore, treating the questions as opaque entities prevents RABJ from performing optimizations before sending questions to the client. To counteract these costs, we impose fairly strict latency requirements on the question serving infrastructure. Average request times are under 50ms and the most expensive 1% of requests complete under 100ms. Typically, a single instance is able to serve questions in under 10ms.

4.3 Components

RABJ is designed as a set of loosely coupled network-based server subsystems. A *store* records question and judgment data from users (both requesters and judges), a *broker* handles requests for questions and *applications* guide user behavior while collecting judgments. The *broker* is a fast application specific cache which maintains user session state while the *store* is a slower question archive maintaining global entity state.

Store

The RABJ store is the authoritative record of all questions and judgments. The store provides the broker and other applications with access to questions and records judgments returned. The RABJ store is responsible for ensuring that questions are served to the broker in a manner that speeds their completion i.e. serving partially completed questions before starting new ones when possible.

Broker

The RABJ broker serves requests for questions from a queue. The request may contain additional constraints on question tags. These requests are made by an application on behalf of a judge who wishes to answer a question. The broker finds questions from the queue which are incomplete and returns them to the client ensuring that they have not already been answered by the current judge.

The broker is also responsible for detecting that questions have received their requirement for votes and triggers external observers which are listening for changes in question state.

The broker exploits the spatial locality of judge behavior. Once work starts on a queue it is highly likely that more questions will be requested from the same queue. Keeping this in mind, the broker pre-fetches questions in bulk and actively works to match it's local cache fill-levels to the request rate.

Applications

Human judges never interact directly with RABJ, but always through applications that mediate the user experience while reading and writing data using the RABJ API.

The current RABJ applications are written in HTML and Javascript and are powered by Freebase's hosted application platform Acre².

Acre is a zero-installation platform allowing users to develop and run web applications in a hosted environment. Acre's enforced open source model allows users to freely fork and tailor applications to their needs. Finally, Acre contains a RABJ-specific library that abstracts protocol complexities and makes it easier for developers to write applications which interact with RABJ. Acre has been critical for the evolution and the success of the ecosystem around RABJ.

These factors, coupled with RABJ's agnostic data model, made it possible to rapidly explore a wide range of possible applications to collect human judgments for various tasks.

4.4 Question Lifecycle

A question begins its life when it is added to a queue. When a judge decides to answer questions, an application will fetch a question from the broker. If a judgment is made, it is recorded in the RABJ store, coming in via the application and the broker. The broker checks if the question is complete – and if so, removes it from service, ending its current life. Finally, the broker notifies any interested parties that the question has been completed.

Other task-specific processes periodically poll the RABJ store for completed questions and act on the judgments received. In cases where there are no conflicting judgments

²<http://acre.freebase.com/>

the action is straight-forward. In cases where conflicts arise, a decision needs to be made.

Conflicts

A conflict is the state where a question has judgments which are mutually incompatible. RABJ is unable to act on these conflicts autonomously since the semantics of a conflict are directly tied to the task at hand. In some cases the polling process will bring the question back to life for additional judgments. In other scenarios the question will be escalated to more skilled judges for conflict resolution. It is important to note that conflicts increase the turn-around time and eventual cost of acting on a question and are an important factor to consider when determining the efficiency of a RABJ application.

5. ECOSYSTEM

Here we describe some parts of the RABJ ecosystem which we have found most compelling to our purpose.

5.1 Judges

Our judges consist of 84 paid contractors and 555 volunteers from the Freebase user community. Some tasks were of interest to the Freebase community and expressly done purely by them, while other were done purely by paid contractors. Relatively few involved both groups of judges. As mentioned earlier, contract judges made 1.25 million judgments and the community made 1.05 million judgments.

Unlike Amazon’s Mechanical Turk [4] where the judges (*turkers*) are anonymous, the judges in our system are identified by an id and a profile ³. More importantly, RABJ contract judges are recruited and evaluated for long-standing engagements. This allows us to develop a relationship with them. The judges in the volunteer Freebase user community organize around a Wiki and a mailing list to communicate, and various leaderboards (Fig 6) provide incentives to the community to participate.

5.2 Judge Hierarchy

Contract judges typically start with simpler tasks, and based on their understanding and performance, are placed in a tiered hierarchy, where they work on harder problems, such as those which require a better understanding of the Freebase schema. Early on, we realized that to scale this to a large number of judges, Freebase employees could not directly manage the contracted user base. We systematically identified the most reliable amongst our contract judges and promoted them to managers. Today a significant portion of the administrative overhead of tasks is handled by our managers who are themselves contract judges.

When a queue is created with questions that seek judgment, the requestor of this task assigns it to a manager. The manager then creates queue-specific guidelines with canonical examples and pitfalls and assigns a pool of judges to the queue. Contract judges are paid per hour instead of per judgment, and qualitative assessments of their performance by their managers is used as a measure of their contributions.

³Freebase profiles are publicly visible to anyone and typically contain information which includes a person’s name, facts that they have added to Freebase and sometimes a picture.

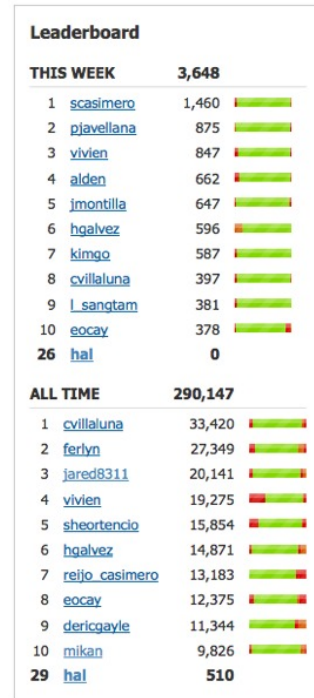
5.3 Visibility

In the course of operations we discovered the need for real-time visibility into the state of a RABJ task. To this end, we created an analytics subsystem which monitors RABJ activity for irregular behavior. We track conflict rates, queue throughput and user throughput. These are valuable metrics which allow us to switch course in the midst of a task. For instance, high conflict rates across all judges is a strong indication that the questions we are asking are poorly constructed or have ambiguous answers. Similarly, low throughput rates are warning that judges are struggling with a particular task.

5.4 Tools

An adhoc set of tools using bug tracking software, Google documents and email is used for collaboration by the managers and the judges. Web-based leaderboards (Fig 6), answer browsers, and a web based analytics tool that displays throughput, performance, conflict rates, etc. is used by managers to monitor and reallocate resources while the task is in progress ⁴.

Figure 6: MatchMaker2 Leaderboard



6. INCREASING THE RETURN ON HUMAN JUDGMENT

In the applications described thus far, human judgments are a fallback mechanism which is part of larger computational effort. We use human input when algorithms fail or report low confidence. Instead of viewing these as complemen-

⁴In the course of a task that required reconciling terms from WordNet synsets to Freebase topics, one of the managers noticed that the problems were hard as there were a number of anatomical terms, and decided to allocate a few judges who were nurses to this task.

tary tasks operating in a disjoint manner, it is worthwhile to consider this as a two phase process. The first, unsupervised step produces a small highly-likely confusion set for a given input. This confusion set is labeled through human judgments. We then use the original feature set (from the unsupervised learning step) in combination with the newly labeled examples to train classifier which predicts the label a judge might assign to a new question. We evaluate these predictions incrementally then rinse and repeat.

Consider a concrete example: mapping two ontologies, WordNet and Freebase. The closest analogue of WordNet is a dictionary, while Freebase is more encyclopedic. There is some overlap between the two, however, Freebase has a larger representation of concrete instances, like specific people, media and locations. A concept in WordNet is represented by synset – a set of synonymous words. It lacks instances of abstract concepts and has a small number of proper nouns, which is Freebase’s strength. Mapping the two ontologies has a manifold increase in the connectedness of the concept graph. For example, connecting the synset corresponding to *democracy* in WordNet to */en/democracy* the identifier for the same concept in Freebase makes it possible to find in Freebase the organizations, books, quotations, Nobel prizes won related to democracy; while WordNet captures the relationship that *democracy* is a *political orientation*, and therefore has sibling concepts of *monarchism*, *collectivism*, *communism* among many others, a piece of knowledge not currently modeled in Freebase.

WordNet has 117,000 synsets, out of which 85,000 are nouns. For each of these synsets, we generated a set of candidate matches based on different criterion including string matching on names and definitions, tfidf-weighted string matching, part-of-speech tagged matching, etc. The top three to five candidate matches were shown to human judges in a matchmaker application. We collected three judgments for each questions which gave us a set of 15,000 candidate matches for 5,000 questions. Each match was described by fifteen features that include the different string matching scores, the part of speech of the synset, the depth in the WordNet graph, the length of the description in Freebase, among others. We then used trained an SVM regressor to predict concordance of human judges on each candidate match. In a ten-fold cross validation, we found that this regressor could predict human votes with an precision of 90% and a recall of 75% in aggregate.

Using the regressor, we generated a batch of 15,000 questions for which the algorithm predicted a good match. This time, we asked for only one judgment per question, having got one judgment from our SVM model. There were 1500 mismatches which were sent to a different pool of human judges with higher skill levels. We verified that these were ambiguous or hard questions or places where the algorithm had erred.

In this example, we were able to use one judgment when we previously needed three. We are just beginning to explore the space of increasing the return on judgments.

7. LESSONS LEARNED

We have used the RABJ ecosystem to collect a large number of judgments for a variety of very different tasks. Through this empirical process, we have come to conclusions which were not obvious to us initially. Some of our observations are different from practices reported by others using crowd-

sourcing to similar effect.

7.1 Long-standing relationships with judges are beneficial

Anecdotal evidence indicates that a key difference between the RABJ ecosystem and one offered by services such as Amazon’s Mechanical Turk is the relationship between requesters and judges. We have an ongoing relationship with our contracted judges, they continue to work for us across tasks and there is significant out-of-band communication between judges and their managers (who are also contract judges).

In the time we have used RABJ we have not had to invest in engineering approaches to tackle spam, collusion and gaming. Instead, we have devoted resources towards training judges on the nuances of Freebase data and the applications they use. In many cases this training has been fundamental for them to answer questions accurately and efficiently.

Furthermore, once vetted, judges do not have to qualify for work again and continue working on other tasks. This continuing relationship provides a foundation for us to shepherd them through a hierarchy of skills required and also to promote them to higher levels of work.

The communication channels between judges have been invaluable. Through this a significant volume of documentation has been written. These documents are centered around our bug-tracking system and web-based collaboration tools. Generating this documentation internally would have required significant effort and impacted our data pipelines. Feedback from judges has also shaped application design and user flows.

7.2 Time-based compensation is effective

Many crowdsourcing implementations use a judgment-based payment model. This is attractive as it pays for units of work completed and provides incentives for minimizing idle time. Conversely, it places value on quantity of work done over quality. Such a model is prone to spam, collusion and gaming on the part of judges. Detecting *spammers* who provide a large number of poor quality judgments is an active area of research [5].

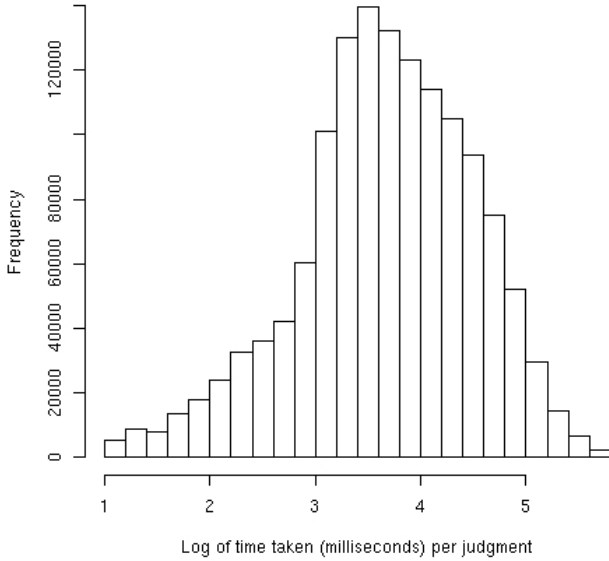
Figure 7 shows the distribution of time taken to make a judgment across all judgments in RABJ. Questions vary in difficulty from those that require a few hundred milliseconds to tasks that take over a minute each⁵. Even in the same class of tasks, questions are of varying level of difficulty: imagine a matchmaker question involving a famous celebrity that the judge knows, versus one that involves biological taxonomy and requires additional research. Compensating judges by unit of work equates these two difficulty levels and offers no additional reward for completing harder questions accurately. We do not have the luxury of lowering accuracy levels for more difficult questions and believe that a time-based compensation model honors the judges’ work more accurately.

A comparison of compensation strategies remains an open area for further investigation.

7.3 Fewer options lead to less conflict

⁵The x-axis in Fig. 7 shows the *log* of the dwell time. Some judgments were submitted automatically when applications detected that questions were no longer relevant. This accounts for the volume of judgments made in tens of milliseconds.

Figure 7: Distribution of Dwell-times for RABJ Tasks



Unanimous agreement from all participants is the ideal way to arrive at an answer from a sampling of human judgments. However, when judges disagree we find ourselves in a bind: we need a strategy for resolving the conflict. Our experience in acquiring contributions and direct user feedback has indicated that simple tasks which offer quick wins (*keeping the user in the flow*) is one of the most critical factors in making an application successful. Restricting each question to a few binary judgments increases worker throughput with the added benefit of reduced potential for conflict.

When disagreements do occur, they can be attributed to one of three sources, *viz.* one or more judges have made a mistake, the question is hard, or; the question is invalid. In order to monitor for these cases we need to add response options that allow judges to signal when a question was vague or simply invalid.

This trade-off between simplicity and capturing the right signal is at the heart of designing questions and making an application appealing. Conflate multiple decisions into a single choice and we lose some signal. Conversely, adding additional responses adds another decision point and with it, the possibility that responses will conflict along another dimension. Ultimately, our goal with RABJ is to have an actionable set of judgments collected per question, and the presence of conflicts makes it hard to do that without further investment of time.

7.4 Empirically derive the number of judgments required

In simpler applications like Typewriter (where a user answers a type classification question e.g. is New Harrisburg a City/Town?), we collected a single judgment on a large number of questions. Later analysis based on redundant judgments showed that the conflict rate was about 5%, higher than acceptable to us. This prompted an evaluation of the

judgments and an analysis of the errors which occurred. We observed that for a number of questions a single judgment was insufficient.

Furthermore, as Table 1 shows, the error rates varies by the “type” contained in the question. In many cases these errors are so prevalent (97% error rate for educational institutions) they indicate systemic problems in the questions. Indeed, we did find that for the most problematic types, their type definitions were vague and subject to misinterpretation.

As a corollary, this result also indicates that the number of judgments required can vary across the questions in a single task. For many types, a single judgment remains adequate for meeting our accuracy levels e.g. deceased people. It would be wasteful to require multiple judgments for such questions.

Table 1: Error-% for Single-Judgment Questions

Type	Questions	Error Rate
/education/institution	9065	0.977
/biology/organism_classification	24996	0.468
/biology/animal_breed	1061	0.368
/organization/organization	6458	0.160
/book/magazine	3113	0.094
/sports/sports_team	9982	0.085
/business/employer	35574	0.080
/people/person	106629	0.075
/location/location	94703	0.062
/geography/body_of_water	5406	0.062
/architecture/structure	24520	0.035
/transportation/road	5069	0.026
/geography/mountain	4881	0.023
/business/company	19393	0.021
/boats/ship	8329	0.018
/tv/tv_actor	10460	0.011
/music/song	17367	0.010
/people/deceased_person	85220	0.008

While RABJ supports dynamically adjusting the judgments required for a question, it remains to be seen if a suitable model can be derived to predict the number of judgments required. We suggest that for new tasks, a titration process be conducted to arrive at the optimal number of judgments needed. Adding descriptive meta data to questions is helpful in this process. This meta data can later be used to analyze and predict difficulty levels.

7.5 Incentives for volunteers are quite different than paid judges

There are two classes of judges that contribute to Freebase through RABJ-powered applications: volunteers and paid contractors. The economy of incentives for volunteers is different from those for paid contributors due to the fact that they get compensated with non-conservative resources (resources that can be shared without reducing their values, such as experience, knowledge, fun, gratification, etc) and non-conservative resources exhibit very different dynamics.

One of the most important results of this difference is that we have found it easier, more intuitive and ultimately more cost-effective to direct paid contractors toward a particular problem than to research and design applications that were both effective at achieving high data quality and be enticing

and rewarding enough to capture and retain volunteer labor.

We are aware that conservative resources put a limit to the efficiency and scalability of system targeted at paid providers. An application that can leverage volunteer human labor can scale dramatically better because of the non-conservative nature of the payback. We believe that generally-applicable behavioral designs for data-oriented games could radically change the nature of our business, yet, we also acknowledge that the cost of finding, designing and establishing reusable successful strategies for data acquisition is very high.

While *Typewriter* received most contributions in our system which largely came from volunteer labor, it did so by being general and covering a broad spectrum of data domains. It took several months for this app to be noticed and start generating a high volume of return. On the other hand, applications specifically designed for very targeted tasks (*Matchmaker* focuses on reconciliation tasks in narrower domains) do not entice volunteer contributions because contracted judges can finish a particular task (comprising even tens of thousand of judgments) before volunteers notice it.

Despite our focus on contract judges, we have gathered insights (by empirical direct observation or feedback from our volunteer contributors) suggesting generally applicable practices to encourage contribution and solidify compensation. While these represent widely established practices, we list them here for completeness.

7.5.1 Leaderboards

Each application comes with a *leaderboard* where users can rank against one another and compete for their rank on that list. Such systems provide gratification and reward as well as stoking competition among volunteers. Metrics like percentage of task completed are readily visible and provide the judges a sense of reward in completion of the task.

While we understand the concerns that social ranking systems can also lead to gaming and can increase the toxicity level of the volunteer contribution ecosystem, we have yet to experience these negative effects in practice.

7.5.2 Attribution encourages contribution

The most concrete way we reward our volunteer judges is by giving them a contribution score that reflects the amount of data they have contributed to Freebase. We attribute every assertion in Freebase to the user responsible for making it. Many of the one million judgments that came from the community led to an assertion, for example, identifying the gender of a person. However, we are yet to formalize attribution of partial contributions when multiple, potentially conflicting judgments support an assertion.

8. CONCLUSIONS AND FUTURE WORK

We presented RABJ, a system for human computation. RABJ seamlessly integrates human computation into data processing activities at Freebase, and has been successfully used at scale. We showed examples of different classes of applications built on top of RABJ, and the ecosystem around RABJ that has allowed it to flourish. We have described initial results on increasing the return on human judgments. Finally we demonstrated the effective use of a payment model for crowdsourcing applications that runs counter to conventional practice.

We are exploring the design of reputation and reliability

management systems that will allow us to accurately estimate the veracity of a judgment received by RABJ.

We have also noticed recurring patterns of multi-stage human judgment pipelines that require escalating a subset of questions to more skilled workers, or task decompositions that require multiple queues. We have not formalized this as a part of RABJ.

Human judgments increase the effectiveness and yield of automated processes manifold. Just as vast amounts of link data fundamentally changed web search, unfettered access to a stream of human judgments can change the landscape of data driven algorithms.

9. ACKNOWLEDGEMENTS

The authors would like to acknowledge Reilly Hayes and Mike Shwe for their continued efforts and support. Thanks to all who have assisted with the conception, design and operation of RABJ. We are indebted to the pioneers across Metaweb who built the initial applications that have guided RABJ's evolution.

10. REFERENCES

- [1] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, and J. Taylor. Freebase: a collaboratively created graph database for structuring human knowledge. In *SIGMOD '08: Proceedings of the 2008 ACM SIGMOD international conference on Management of data*, pages 1247–1250, New York, NY, USA, 2008. ACM.
- [2] D. Farber. Google's Marissa Mayer: Speed wins. *CNET Between the Lines*. <http://blogs.zdnet.com/BTL/?p=3925>, 2006.
- [3] D. F. Galletta, R. Henry, S. McCoy, and P. Polak. Web site delays: How tolerant are users? *Journal of the Association for Information Systems*, 5:1–28, 2004.
- [4] A. Kittur, E. H. Chi, and B. Suh. Crowdsourcing user studies with Mechanical Turk. In *CHI '08: Proceeding of the twenty-sixth annual SIGCHI conference on Human factors in computing systems*, pages 453–456, New York, NY, USA, 2008. ACM.
- [5] V. S. Sheng, F. Provost, and P. G. Ipeirotis. Get another label? Improving data quality and data mining using multiple, noisy labelers. In *KDD '08: Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 614–622, New York, NY, USA, 2008. ACM.
- [6] K. Siorpaes and M. Hepp. Games with a purpose for the semantic web. *Intelligent Systems, IEEE*, 23:50–60, 2008.
- [7] R. Snow, B. O'Connor, D. Jurafsky, and A. Y. Ng. Cheap and fast – but is it good? Evaluating non-expert annotations for natural language tasks. In *EMNLP*, pages 254–263. ACL, 2008.
- [8] L. von Ahn. Games with a purpose. *IEE Computer Magazine*, 39:92–94, 2006.
- [9] L. von Ahn and L. Dabbish. Designing games with a purpose. *Communications of the ACM*, 51(8):58–67, 2008.
- [10] L. von Ahn, M. Kedia, and M. Blum. Verbosity: a game for collecting common-sense facts. In *Proceedings of ACM CHI 2006 Conference on Human Factors in Computing Systems, volume 1 of Games*, pages 75–78. ACM Press, 2006.

Sellers' problems in human computation markets

M. Six Silberman, Joel Ross, Lilly Irani, and Bill Tomlinson

Department of Informatics
University of California, Irvine
msilberm | jwross | lirani | wmt@uci.edu

ABSTRACT

“Tools for human computers” is an underexplored design space in human computation research, which has focused on techniques for buyers of human computation rather than sellers. We characterize the sellers in one human computation market, Mechanical Turk, and describe some of the challenges they face. We list several projects developed to approach these problems, and conclude with a list of open questions relevant to sellers, buyers, and researchers.

Categories and Subject Descriptors

H.5.m [Information Interfaces and Presentation (e.g., HCI)]: Miscellaneous

General Terms

Design, Economics, Human Factors

Keywords

Human computation, Mechanical Turk, Markets

1. INTRODUCTION

“Tools for human computers” is an underexplored design space in human computation research.

With very few exceptions (notably [1], [2], [3]), human computation research has focused exclusively on problems facing users (or “requesters”) of human computation. Put shortly, the user of human computation wishes to get good data from “human computers” quickly, without paying much (e.g. [4], [5]). With the growth of paid crowdsourcing systems, most famously Amazon’s Mechanical Turk (“AMT”) platform, and interest in market mechanisms in human computation (e.g., [6], [7], [8], [9], [10] [11]) and human-computer interaction (e.g., [12], [13], [14], [15], [16]), the number of people doing human computation tasks for money has also

grown. Sellers¹ in the market for human computation, like buyers, have interesting and difficult problems to solve, some of which could benefit from the technical expertise of human computation researchers. In this note we first characterize sellers in the AMT labor pool and describe some of their problems. We present several projects, including one developed by the authors, that approach some of these problems. Finally, we list open design problems and empirical questions of interest to human computers to which human computation researchers might contribute.

2. THE CROWD AND ITS PROBLEMS

The AMT labor pool hosts a growing international population earning less than USD 10,000/yr., some of whom are reliant on their Turkling income to make basic ends meet. The uncertainty associated with HIT payment complicates human computers’ work and reduces their effective wage. This uncertainty is due in part to the apparent prevalence of fraudulent requesters, to whom certain design decisions have made AMT particularly attractive.

Intuitively, we might expect that, just as buyers of human computation aim to minimize expense at a fixed quality (or maximize quality within a cost constraint), sellers of human computation wish to secure payment with a minimum time expenditure, even if this means “gaming the system” by providing responses they know are of low quality. Fraudulent sellers do appear to optimize in this way, but a reading of survey responses and forum discussions reveals a concern for what is “fair” and “reasonable” rather than a desire to maximize short-term personal earnings at requester expense.

Ross et al. [17], [18], extending work by Ipeirotis [19], [20], present longitudinal demographic data on the “increasingly international” AMT. They find that AMT hosts a growing population of young, male, Indian workers earning less than USD 10,000/yr. Additionally, almost a third of Indian Turkers surveyed reported that they always or sometimes relied on their Turkling income to “make basic ends meet” (27% in May 2009 and 31% in Feb. 2010). Between May 2009 and Feb. 2010, the fraction of US Turkers surveyed reporting reliance held steady at 13±1%.

Many Turkers see themselves as laborers doing work to earn money. In survey data collected in Feb. 2009 ($n = 878$) [21], the most commonly reported motivation for doing HITs was making money. 91% of respondents mentioned

¹We use “sellers,” “workers,” and, when referring to AMT, “Turkers” interchangeably.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

KDD-HCOMP '10, July 25, 2010, Washington, DC, USA
Copyright 2010 ACM 978-1-4503-0222-7 ...\$10.00.

a desire to make money, whether as a form of supplemental income or in order to purchase extras. Turkling to pass the time, in contrast, was mentioned by only 42% of respondents. Feb. 2010 data ($n = 1000$) from Ipeirotis [22] confirms the importance of money compared to other motivations, with most respondents reporting they do not do HITs for fun or to kill time, and ~25% of Indian respondents and ~13% of US respondents reporting that AMT is their primary source of income.

What challenges, then, face these “professional” crowd-workers? We can begin to answer this question by examining the “mTurk Suggestions” board of Turker Nation (turker.nation.com), a free web forum operated by and for Turkers. The first thread² is called

*** IMPORTANT READ: HITs You Should NOT Do! ***

As of late April 2010, the thread has 57 replies and 15,048 views since it was started in late October 2009. User **spamgirl**, a Turker Nation administrator with 55,360 posts on the forum between Dec. 2008 and Apr. 2010, started the thread with the following suggestions:

Do not do any HITs that involve: filling in CAPTCHAs; secret shopping; test our web page³; test zip code; free trial; click my link; surveys or quizzes (unless the requester is listed with a smiley in the Hall of Fame/Shame); anything that involves sending a text message; or basically anything that asks for any personal information at all — even your zip code. If you feel in your gut it’s not on the level, IT’S NOT. Why? Because they are scams and you will not get paid for your work. Worse, your personal information will be used to send you spam, or steal your identity, or bill you fraudulently. [23]

She also advises Turkers to avoid doing search engine optimization (SEO) tasks, creating accounts on other sites (forums, Twitter, Facebook, etc.), posting a requester’s content to online classified listings (“or even your own blog”), downloading software to test, or accepting payment for a review or rating. Newcomers to the forum might read the thread as alarmist, but the prevalence of scam HITs and the difficulty of finding legitimate work on AMT is an ongoing discussion among Turkers. User **dreamingfree** responded to the thread, “I don’t understand why so many of the kinds of hits that you mention are still ubiquitous on Mturk. I have trouble finding any that aren’t in these categories! :-/” In early Nov. 2009, user **baubles** replied, “The black hats are out in full force today. Dozens of hits asking for email addresses after clicking on their links. Be careful out there.” A few days later, user **kaine13** posted, simply, “I’m so damn sick of these...” User **a4x401** offered a newcomer’s take:

Being a newbie and having relatively decent pc skills I have been checking all this stuff out and am somewhat upset [about] the things that I have discovered! It’s no wonder that people don’t trust the requesters, yes I did some of those hits that one should not do and found myself having to repair my pc and remove some pop-ups. After having done that I really got into checking out the program and realized that it’s too easy to manipulate it due the fact that work can be rejected

²The thread is “sticky,” appearing at the top of the thread list whether or not it has the most recent reply, indicating its importance according to moderators.

³These appear generally intended not to test functionality but to drive traffic, often to sites requiring email address registration.

*after it’s finished but the work is still done. All [a requester] has to say is “not to our satisfaction”!!!! The other way is to just leave the hits open; you still collect your work but don’t have to pay! My favorite part is hits that are way too complicated to complete in the time frame allowed! Why is there no control on any of this stuff? I’m still waiting for hits to be approved from Oct 31 [on Nov. 6], what’s up with that? After all this I discovered this forum and have seen the complaints from all the others; why is there no control?*⁴

spamgirl’s warnings pertain to HITs where (a) there is uncertainty about whether the requester *intends* to pay workers, (b) the requester intends to spam or defraud workers, and/or (c) the task is against AMT’s terms of use. (The private incentive to avoid HITs that violate terms of use, according to **spamgirl**, is that if the offending requester’s account is suspended, workers who did their tasks will not be paid.) **a4x401** corroborates the malware problem and highlights a few additional ones: (d) requesters can collect and use work without paying workers; (e) some tasks have prohibitively short time limits; and (f) requesters often take longer than seems reasonable to pay, especially given the small amount.⁵

Intrigued by earlier discussions of these phenomena, Irani in Oct. 2008 posted a HIT asking respondents to articulate a hypothetical “Turkers’ Bill of Rights” [24]. 67 respondents agreed to have their submissions displayed publicly. These were coded with an eye toward recurring topics of worker concern. 35 responses indicated a concern with unfairly or arbitrarily rejected work and 26 with slow payment. Seven explicitly mentioned a “minimum wage” or “minimum payment” per HIT, and 14 a minimum wage/payment and/or more “fair” compensation generally. Eight indicated a frustration with lack of response to email communication on the part of both requesters and AMT staff, often in the context of complaints about rejections perceived unfair or tasks in violation of the platform’s terms of service.

Respondents sought more information about requesters and the ultimate use of work supplied by Turkers. Several respondents, for example, indicated a frustration with tasks requiring third-party website membership or disclosure of an email address, and with tasks perceived as possibly implicated in the production of email or blog spam. Some were interested in a forum in which Turkers could air concerns publicly without censorship or condescension, and worker visibility and dignity more generally. Others were interested in a way to build long-term work relationships with prolific requesters, and worker-requester relations generally. Two respondents mentioned unionization.

Respondents expressed concerns about malicious or fraudulent requesters who systematically reject best-effort work in order to obtain it without payment; data privacy in survey tasks; technologically defective HITs or HITs with unclear or inadequate instructions;⁶ arbitrariness of bonuses; and tasks in which workers were asked to rate work submitted by others, presumably as a basis for approving or denying that work. Three respondents said they liked the platform just as it was and that its design was appropriate for “inde-

⁴Edited for punctuation.

⁵Workers are automatically paid after 30 days if the requester does not reject the work. Some workers have pointed out that a month is a long time to wait for 5 cents.

⁶These affect workers’ ratings negatively by forcing them to either return the task or leave it incomplete.

pendent contractors” who did the work by choice.

In Jan. 2010, Hartmann [25] asked Turkers about the biggest challenges facing crowdworkers. Respondents noted the difficulty of finding “credible work”; of communicating with requesters and Amazon staff; of finding “tasks that pay enough to be worth the effort”; the annoyance of “getting work rejected for no good reason”; and the frustration of technical errors by requesters or by Amazon leading to damaged worker reputations (leaving workers unable to accept work they could have otherwise done). These responses indicated the persistence of previously reported concerns.

In these responses and discussions we can identify at least eight recurring themes articulated by workers: (1) uncertainty about payment; (2) unaccountable and seemingly arbitrary rejections; (3) fraudulent tasks; (4) prohibitive time limits; (5) long pay delays; (6) uncommunicative requesters and administrators; (7) cost of requester and administrator errors borne by workers; and (8) low pay. There are of course others; for more discussion see the “Suggestions” boards on Turker Nation and mTurk Forum (mturkforum.com).

3. APPROACHES TO SELLER PROBLEMS

Software tools exist, some built by Turkers, that attempt to help Turkers manage these problems. Many are client-side scripts that add functionality to the AMT interface; some include a web database component. One platform aims to compete with AMT, in part by addressing some of these problems.

Workers and requesters on mTurk Forum have made a number of Turkling tools, including a list of all requesters [26], [27], a script for recording your own worker history (which is not preserved by AMT, but is useful knowledge for tax purposes) [28], [29], and, indirectly addressing some of the problems mentioned above, a client-side script to hide HITs posted by particular requesters [30].

Irani and Silberman built and maintain Turkopticon (turkopticon.differenceengines.com), a Firefox add-on and web database application that adds a drop-down interface element next to each requester’s name in the HIT listing displayed to workers. When the worker mouses over the name of a requester about which the database has data, averages are displayed for four requester attributes, ‘communicativity,’ ‘generosity,’ ‘fairness,’ and ‘promptness,’ motivated by problems 1-3, 5, 6, and 8 above.

CloudCrowd, launched in Sep. 2009, is positioned as a more “worker-friendly” alternative to AMT. In his post to mTurk Forum introducing the service [31], CEO Edelstein writes that CloudCrowd will offer “a more efficient [worker] interface,” payment through PayPal (allowing workers to collect currencies other than USD and INR, the only choices for Turkers), and “credibility” ratings (in place of acceptance rates as in AMT) as the measure of worker quality.

4. OPEN PROBLEMS AND QUESTIONS

These projects are only a tentative step toward addressing the problems facing Turkers and developing a richer understanding of the structure and dynamics of human computation markets. Many questions remain, including:

How does database, interface, and interaction design influence individual outcomes and market equilibria? This has been explored in online auctions (especially eBay; e.g., [32], [33], [34], [35]) but not human computation.

What are the economics of fraudulent tasks (scamming and spamming)? Work in this thread might draw on research on the economics of botnets (e.g., [36], [37], [38]).

What decision logics are used by buyers and sellers in human computation markets? Buyers who read HCOMP papers and workers who “game the system” may maximize financial return, while other buyers and sellers may satisfice. What are the various problems being solved (and with what strategies) by different actors in human computation markets and how do these shape market outcomes?

What’s fair in paid crowdsourcing? Economists Akerlof and Shiller [39] argue that “considerations of fairness are a major motivator in many economic decisions” that has been overlooked in neoclassical explanations that assume economic decision makers act rationally. They lament that “while...there is a considerable literature on what is fair or unfair, there is also a tradition that such considerations should take second place in the explanation of economic events” (pp. 20, 25). AMT requesters and administrators say tasks should be priced “fairly,” but fairness is difficult to define and thus to operationalize in practice. Concepts like a reservation wage (as in [2]) are useful here, but do not settle the matter, which is complicated economically and culturally by the global reach of human computation platforms.

Gaps remain in our demographic understanding of AMT, despite at least two ongoing parallel investigations. For example, US-based researchers (including the authors of the present paper) have designed surveys with income questions where the lowest bin is “less than USD 10,000/yr.” Most people in India make less than USD 10,000/yr., so these survey results are lossy. As Indians (and citizens of countries less affluent than the US generally) come to comprise a larger fraction of the Turkling population these surveys will need to be revised and redeployed. Additionally, language proficiency is an occasionally overlooked confound in quality control strategies; lack of English fluency, while perhaps a good proxy for unreliability in English language tasks, does not imply intent to “game the system” (cf. [16]).

As new platforms and tools come online and mature, comparative studies in all of these areas will become possible, and longitudinal studies more feasible. Much of the prevalence of the eight recurring themes is owed to two conditions that characterize AMT: information asymmetry and the power differential between requesters and administrators on one hand and workers on the other. How new human computation systems are designed (mostly through software) to make information available and distribute decision-making powers to different actors will be of practical interest to buyers and sellers, and empirical and theoretical interest to scholars.

Students in a joint Harvard/Stanford law schools course on “difficult problems in cyberlaw” [40] developed a list of hypothetical projects that might address some of the problems discussed above, including “Turkopticon-style MTurk plugins that achieve various ends, such as adding social features to [AMT] and increasing accountability” [41]. Human computation researchers are well-suited to evaluate such proposals, contributing to understanding of human computation market design and user experiences of buyers and sellers.

5. CONCLUSION

Fraud, spam, malware, and frustrating user experiences are endemic to the internet broadly, not just human compu-

tation markets. We do not suggest that human computation researchers should take it upon themselves to solve these problems, or even that they can be “solved” definitively. We do suggest however that it may be both practically useful and theoretically productive to ask how human computation markets can be made less hospitable to these “black hats.”

Most of us complain about our jobs, no matter how well-paid; that Turkers do also is hardly surprising. That their work takes place entirely within markets built in software presents a moderately novel situation, however, in which the difference between making rent and failing to do so can be made by a software (market) design decision. We believe the expertise of human computation researchers is well-suited to explore and address some of the problems raised by human computers, and that our understanding of human computation will be both broadened and deepened in the process.

6. ACKNOWLEDGMENTS

We thank the users of Turkopticon; the Turkers who completed our HITs; the members of Turker Nation and mTurk Forum, especially **numbers** and **spangirl**; Beatriz da Costa; Kavita Philip; Lukas Biewald; Paul Dourish; and Gloria Mark. This work was funded by the Donald Bren School of Information & Computer Sciences at UC Irvine, the California Institute for Telecommunications and Information Technology, the Sloan Foundation, and the National Science Foundation (Grant No. 0644415).

7. REFERENCES

- [1] Zittrain, J. Ubiquitous human computing. *Phil. Trans. R. Soc. A* **366**: 3813-3821, 2008.
- [2] Horton, J. J. and L. Chilton. The labor economics of paid crowdsourcing. arXiv:1001.0627v1 [cs.HC], 2010.
- [3] Horton, J. J. The condition of the Turkling class. arXiv:1001.1172v1 [cs.CY], 2010.
- [4] Snow, R., et al. Cheap and fast—but is it good? *Proc. EMNLP '08*.
- [5] Feng, D., et al. Acquiring high-quality non-expert knowledge from on-demand workforce. *Proc. ACL-IJCNLP '09*.
- [6] Hsieh, G. et al. Can markets help? *Proc. CSCW '08*: 535-544.
- [7] Hsieh, G. and S. Counts. mimir. *Proc. CHI '09*: 769-778.
- [8] Hsieh, G., et al. Why pay? *Proc. CHI '10*: 305-314.
- [9] Burnham, T. and R. Sami. A reputation system for selling human computation. *Proc. HCOMP '09*.
- [10] Jain, S. and D. C. Parkes. The role of game theory in human computation systems. *Proc. HCOMP '09*.
- [11] Mason, W. and D. J. Watts. Financial incentives and the “performance of crowds.” *Proc. HCOMP '09*.
- [12] Kittur, A., et al. Crowdsourcing user studies with Mechanical Turk. *Proc. CHI '08*: 453-456.
- [13] Heer, J. and M. Bostock. Crowdsourcing graphical perception. *Proc. CHI '10*: 203-212.
- [14] Jeon, G. Y., et al. Re-examining price as a predictor of answer quality. *Proc. CHI '10*: 325-328.
- [15] Seuken, S., et al. Hidden markets: UI design for a P2P backup application. *Proc. CHI '10*: 315-324.
- [16] Downs, J. S., et al. Are your participants gaming the system? *Proc. CHI '10*: 2399-2402.
- [17] Ross, J., et al. Who are the crowdworkers? *Proc. CHI '10*: 2863-2872.
- [18] Ross, J., et al. Who are the crowdworkers? Slides, alt. CHI, 14 April 2010, Atlanta, GA. www.ics.uci.edu/~jwross/pubs/RossEtAl-altCHI2010-slides.pdf.
- [19] Ipeirotis, P. Mechanical Turk: The demographics. behind-the-enemy-lines.blogspot.com/2008/03/mechanical-turk-demographics.html.
- [20] Ipeirotis, P. Turker demographics vs. internet demographics. behind-the-enemy-lines.blogspot.com/2009/03/turker-demographics-vs-internet.html.
- [21] Ross, J. and B. Tomlinson. Worker motivations in Amazon Mechanical Turk. Unpub. manuscript, 2009.
- [22] Ipeirotis, P. The new demographics of Mechanical Turk. behind-the-enemy-lines.blogspot.com/2010/03/new-demographics-of-mechanical-turk.html.
- [23] spangirl et al. HITs you should NOT Do! turkers.proboards.com/index.cgi?board=kgeneral&action=display&thread=5126.
- [24] Irani, L. Turk Work. turkwork.differenceengines.com.
- [25] Hartmann, B. Crowdsourcers respond to crowdsourcing. blogs.law.harvard.edu/difficultprobs/2010/01/07/crowdsourcers-respond-to-crowdsourcing.
- [26] techlist. MTurk Requesters. techlist.com/mturk/requesters.
- [27] techlist et al. Current requesters. mturkforum.com/showthread.php?t=1100.
- [28] lightdark et al. Recording your history, would you like to? mturkforum.com/showthread.php?t=1109.
- [29] lightdark et al. Scripts for user data. mturkforum.com/showthread.php?t=424.
- [30] ergo et al. Block (hide) requester script for Firefox+Greasemonkey. mturkforum.com/showthread.php?t=1088.
- [31] CloudCrowd. Intro from CloudCrowd, an alternative to Mechanical Turk. mturkforum.com/showthread.php?t=1169.
- [32] Ba, S., et al. The dynamics of the electronic market. *Inf. Sys. Frontiers* **2**(1): 31-40, 2000.
- [33] Hahn, J. The dynamics of mass online marketplaces. *Proc. CHI '01*: 317-324.
- [34] Resnick, P., et al. The value of reputation on eBay. *Experimental Econ.* **9**(2): 79-101, 2006.
- [35] Steiglitz, K. *Snipers, Shills, and Sharks: eBay and Human Behavior*. Princeton, 2007.
- [36] Franklin, J., et al. An inquiry into the nature and causes of the wealth of internet miscreants. *Proc. CCS '07*: 375-388.
- [37] Li, G., et al. Botnet economics: uncertainty matters. In M. Johnson, ed., *Managing Information Risk and the Economics of Security*: 245-267. Springer, 2008.
- [38] Namestnikov, Y. The economics of botnets. Kaspersky Lab Tech. Rep., 2009.
- [39] Akerlof, G. A. and R. Shiller. *Animal Spirits*. Princeton, 2009.
- [40] Zittrain, J. and E. Stark. Cyberlaw: Difficult Issues. cyber.law.harvard.edu/cyberlaw_winter10/Main_Page.
- [41] andrewj and spangirl. MTurk “App Store”? turkers.proboards.com/index.cgi?board=mturksuggest&action=display&thread=5543.

Frontiers of a Paradigm – Exploring Human Computation with Digital Games

Markus Krause^{*}
Research Group Digital Media
TZI, University of Bremen
28359 Bremen Germany
phateon@tzi.de

Aneta Takhtamysheva[†]
Research Group Digital Media
TZI, University of Bremen
28359 Bremen Germany
aneta@tzi.de

Marion Wittstock
Research Group Digital Media
TZI, University of Bremen
28359 Bremen Germany
wittstock@tzi.de

Rainer Malaka
Research Group Digital Media
TZI, University of Bremen
28359 Bremen Germany
malaka@tzi.de

ABSTRACT

Each day millions of people play digital games with different motivations. These motivations range from time beating to deep immersion into a narration or interacting with a community. To address all these different means, a range of game designs is necessary. Traditional human computation games cannot present all these aspects yet.

This work will give a game centered view on game design for human computation. To demonstrate the value of this view it will present a fast-paced action game called *OnToGalaxy* along with two different human computation tasks.

1. INTRODUCTION

Since human computation is integrated into digital games and has become relevant for science and industry, different fields of its application have been explored. The evolved designs have proved to be effective as well as efficient in many situations. An overview of current human computation games and relevant publications on game design will be given in section 2.

On the one hand human computation games provide a unique game play experience. On the other hand, they are focused on a special task and tightly aligned to it. This design focus makes them a game category on their own. Even though a range of appearances of human computation games has been presented, their core design is to some extent homogeneous. Many works on human computation with digital games are centered around a certain task or problem

^{*}Funded by the Klaus Tschira Foundation.

[†]Funded by the Klaus Tschira Foundation.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

KDD-HCOMP '10, July 25, 2010, Washington, DC, USA
Copyright 2010 ACM 978-1-4503-0222-7 ...\$10.00.

domain. Significantly fewer works are investigating possible game design aspects in the context of human computation.

This paper will focus on the paradigm of using digital games for human computation. Section 3 will demonstrate how to design a game comparable to other casual games that are played by millions of players every day. Benefits and drawbacks of using this design will be discussed as well in section 3.

2. RELATED WORK

Digital games with a human computation task have a range of designs. They can be puzzles, multi-player environments or virtual worlds. They also have many application domains. Common tasks for human computation games are for instance relation learning or resource labeling. A well known example in this regard is the *GWAP* [18] series. Other games try to use human computation for natural language processing like *Phrase Detectives* [6] or *TwinMind* and *Actionary* [16]. Yet others, like *OntoGame* [15] are used to build and learn ontologies. These games perform human computation by integrating their task into puzzle-like games.

Even though many games are very similar in their use of design elements, some are different. *HeardIt*, for instance, stresses user centered design as its core idea [3]. This makes it different from other human computation games that are mostly designed around a certain task as their core element. *HeardIt* also allows direct text interaction between players during the game session, which is prohibited in other games to prevent cheating [19]. *KissKissBan* [9] is another game with special elements. It is a game that involves a direct conflict between players. One player tries to prevent the “kissing” of two other players by labeling images.

Another game that is special according to its design elements is *Plummings* [17]. This game aims at reducing the critical path of FPGAs. Unlike other games the task is separated from the game mechanics. The game is about a colony of so-called *Plummings* who need adequate air supply. By keeping the length of the air tubes as short as possible the player saves the colony from suffocation.

Another interesting game is *FoldIt* [4]. This game aims on folding proteins, which is a complex task. It is a single player



Figure 1: Second version of *OnToGalaxy*. The human computation task is to populate an ontology. The blue ship on the right is a labeled freighter. The red fighter ship in the middle is controlled by the player. On the lower right is the mission description. It tells the player to collect freighters labeled with words of touchable objects like hammer. In the example the freighter is labeled with leaf which fulfills the relation of being a touchable object. The explosion close to the fighter originates from a freighter that was shot down by the player.

game which needs a downloadable client. It presents simplified three-dimensional protein chains to the player, and provides a score according to the predicted quality of the folding done by the player. *FoldIt* is special because all actions by the player are performed in a three dimensional virtual world.

As the presented works show, human computation games have already developed in different styles. To further enhance the design of human computation games it is necessary to investigate game design and related literature. One of the earliest works in this regard was written by Crawford [7]. His book deals with questions of what makes a real game, why people play them, and how to design engaging ones. Crawford gives a restrictive definition of the term “game” in his book, which is later analyzed by different works on this topic. Salen and Zimmerman for instance gave an interesting overview on definitions of games in their book “Rules of Play”[14]. They also describe the aesthetics of interactive systems, and define core concepts like play, design, and interactivity. They looked at games through so-called game design schemes, or conceptual frameworks, including games as digital interactive systems in the context for social play and as a medium for storytelling.

Hunicke, LeBlanc and Zubek describe a formal framework of Mechanics, Dynamics and Aesthetics to give guides on designing digital games. Mechanics are the particular components of a game, at the level of data representation and

algorithms. Dynamics is describing the run-time behavior of the mechanics acting on player inputs and outputs of the mechanics over time. Aesthetics is describing the desirable emotional responses evoked in the player, when he or she interacts with the game system [10]. Other works with an applied perspective on game design are for instance “Game Architecture and Design” [13] by Rollings or “Challenges for Game Designers” [5] by Brathwaite. Both books providing an interesting starting point for game design.

In the following section this work will look at human computation from the perspective of game design. It will describe how a game centered design can be applied and discuss the benefits as well as drawbacks of such an approach.

3. ONTOGALAXY

Every month 200 million people play casual games¹, spending up to two hours a day. The games they play can be of any genre and any type as claimed by the Casual Games Report [1]. As already argued in the previous sections, most human computation games provide a valuable, but specific sort of game play. To reach as many players as possible, it is necessary to expand the designs of games used for human computation.

Designing a game, that is different from traditional human

¹Since the evolution of social network games this number might be much higher. Cause games like Farmville have up to 10 millions of player per day.



Figure 2: Third version of *OnToGalaxy*. The human computation task is to find synonyms for German verbs. The mission tells the player to collect all freighters (blue ships) with a label being a synonym for “knicken” (to break). The female character on the bottom is the copilot of the player. She guides the player through his or her missions and gives necessary instructions. The player is controlling the red ship in the middle of the screen.

computation games and able to perform human computation, is a complicated and complex task. To give an impression of how such a design can look like this work will present *OnToGalaxy*. *OnToGalaxy* is a fast-paced, action-oriented science fiction game comparable to games like *Asteroids* [2] or *Starscape* [12]. Some participants also compared *OnToGalaxy* to games such as *Frontier* and *Wing Commander*, at least to a certain extent. *OnToGalaxy* provides a simple storyline and a progressive game-play. Progressive elements used in the design of *OnToGalaxy* are, for instance, different levels and agents, as well as upgrade elements of the players spaceship. This is comparable to other browser-based casual games. The design was chosen because of its stereotypical elements, seen in many games of its genre. It also allows a good comparison to other human computation games because of its difference in many aspects. To evaluate the outcome in regards to human computation *OnToGalaxy* was designed to reproduce different gold standards. The two different tasks are depicted in figure 1 and figure 2. A depiction of the first version of *OnToGalaxy* is not given because it is similar to version two.

3.1 Game-Play

Since *OnToGalaxy* is a space shooter game, it is not obvious how to integrate the desired relation extraction task into the game. In the game the player is the commander of a spaceship. He or she receives missions from an imaginary headquarter, represented by an agent, which is his or her copilot. Beside the storyline, these missions are present-

ing different tasks to the player like: “Collect all freighter ships that have a call sign that is a synonym for the verb x”. A call sign is a text label on top of each freighter. If a player collects a ship according to the mission description he or she confirms this label to fulfill the given relation. In the given example that means that the collected label is a synonym for x. Even though only two different tasks were integrated, the underlying game engine is capable of fulfilling other tasks where one needs to establish arbitrary relations between two elements. These elements could be words as well as resources like images or audio files.

3.2 Story Matters

A frontier that was explored very early during the design of *OnToGalaxy* was the coupling between task and game design. The first attempt towards a casual design was however not very successful, as player experience the decoupling of the task and the game elements to be “odd”. This happens because solving a task like populating an ontology in a game looking like a space shooter leaves a gap in the players experience. Players of *OnToGalaxy* made jokes about this disturbing effect and tried to give various stories to solve the perceived discrepancy.

Taking this observation, as well as the resulting stories into account, the second version of *OnToGalaxy* provides a storyline trying to close this gap. Even though the first attempt does not solve the problem of impression in total, it however enhances the experience of the players and closes the described gap. After introducing a storyline, no com-

ments regarding this gap were given in the interviews. As explained by some participants of the pre-evaluation interviews the story allows them to close this gap and therefore experience the task as an inherent element of the game play.

However the second version of *OnToGalaxy* was still perceived as “work” or an ordinary task by some players. They do not complain about the gap but still perceive the task as too dominant. Although only very few persons gave comments, the question was whether it would be possible to overcome this issue. Considering possible differences between “normal” games and *OnToGalaxy*, it turns out that the conflict situation was different from other games. In contrast to other games *OnToGalaxy* permits to shoot down freighters, but does not include equal reaction.

After integrating aggressive behavior from the system in version three, the game was accepted by the players. The players in the pre and final evaluation of the third version did not describe the task as distracting any more. Some persons still complained about missions perceived to be too challenging. This, however, is a problem of game balance and is not a problem specific to the human computation task.

3.3 Achieving Speed

As already mentioned a critical issue to some games is speed. This is true for games like *OnToGalaxy*, which like other games of this genre rely on fast interactions. As presented elsewhere [8, 11] fast interactions can be achieved by using binary verification. In contrast to traditional human computation games, the player is not asked to give his or her personal impression, but should decide whether a given relation is valid.

An extension to this is categorization where the player decides to which category a certain element or relation belongs as already presented by Siorpaes and Barrington [15, 3]. Both approaches can be faster than typing personal thoughts into a text field. In case of *OnToGalaxy* the chosen interaction form is binary verification. The game presents a certain relation to the player via the mission screen, and asks to find elements represented by game elements that fulfill the given relation. The major drawback of binary verification and possible solutions are also described elsewhere [8, 11].

4. CONCLUSION AND FUTURE WORK

Casual games are having different genres and styles. This paper has presented an example implementation: the action game *OnToGalaxy* which is comparable to casual action games. It demonstrates that human computation tasks can be integrated into such games. It also shows that the task can be hidden in the game that it is no longer perceived as a dominant element of the game. On the other hand this can prevent players from asking questions about the use and effect of their actions. In our opinion it is therefore mandatory to make sure the player is informed of the use of his or her actions. The next step is to integrate an unsolved task into *OnToGalaxy* and publish the game on a casual games website.

5. REFERENCES

- [1] Casual Games Market Report 2007, 2007.
- [2] Atari. Asteroids, 1979.
- [3] Luke Barrington, Damien O’Malley, Douglas Turnbull, and Gert Lanckriet. User-centered design of a social game to tag music. In *HComp’09 Proceedings of the ACM SIGKDD Workshop on Human Computation*, pages 7–10, Paris, 2009. ACM Press.
- [4] Laura Bonetta. New Citizens for the Life Sciences. *Cell*, 138(6):1043–1045, 2009.
- [5] Brenda Brathwaite and Ian Schreiber. *Challenges for game designers*. Charles River Media, Inc., Rockland, MA, USA, 1 edition, 2008.
- [6] Jon Chamberlain, Massimo Poesio, and Udo Kruschwitz. Phrase Detectives: A Web-based Collaborative Annotation Game. In *I-Semantics’08 Proceedings of the International Conference on Semantic Systems*, Graz, 2008. ACM Press.
- [7] Chris Crawford. *The Art of Computer Game Design*. McGraw-Hill/Osborne Media, Berkeley California, 1984.
- [8] Ali Dasdan, Chris Drome, and Santanu Kolay. Thumbs-up: A game for playing to rank search results. In *WWW ’09 Proceedings of the 18th international conference on World wide web*, pages 1071–1072, New York, NY, USA, 2009. ACM Press.
- [9] Chien-Ju Ho, Tao-Hsuan Chang, Jong-Chuan Lee, Jane Yung-jen Hsu, and Kuan-Ta Chen. KissKissBan: a competitive human computation game for image annotation. In *HComp’09 Proceedings of the ACM SIGKDD Workshop on Human Computation*, pages 11–14, New York, NY, USA, 2009. ACM Press.
- [10] Robin Hunicke, Marc LeBlanc, and Robert Zubek. MDA: A formal approach to game design and game research. *Lecture at Northwestern University*, 2004.
- [11] Markus Krause and Hidir Aras. Playful tagging: folksonomy generation using online games. In *WWW ’09 Proceedings of the 18th international conference on World wide web*, pages 1207–1208, Madrid, 2009. ACM Press.
- [12] Moonpod. Starscape, 2004.
- [13] Andrew Rollings and Dave Morris. *Game architecture and design: A new edition*. New Riders Games, 2003.
- [14] Katie Salen and Eric Zimmerman. *Rules of Play*. MIT Press, Cambridge, MA, USA, 2004.
- [15] Katharina Siorpaes and Martin Hepp. Games with a Purpose for the Semantic Web. *IEEE Intelligent Systems*, 23(3):50–60, 2008.
- [16] Aneta Takhtamysheva, Robert Porzel, and Markus Krause. Games for Games. In *HComp’09 Proceedings of the ACM SIGKDD Workshop on Human Computation*, pages 38–40, Paris, 2009. ACM Press.
- [17] Luke Terry, Vladimir Roitch, Shoeb Tufail, Kirit Singh, Omair Taraq, Wayne Luk, and Peter Jamieson. Harnessing Human Computation Cycles for the FPGA Placement Problem. In Toomas P. Plaks, editor, *ERSA*, pages 188–194. CSREA Press, 2009.
- [18] Luis von Ahn. www.gwap.com (Games with a Purpose).
- [19] Luis von Ahn and Laura Dabbish. Labeling images with a computer game. In *CHI ’04 Proceedings of the 22nd international conference on Human factors in computing systems*, pages 319–326, Vienna, 2004. ACM Press.

GiveALink Tagging Game: An Incentive for Social Annotation

Li Weng^{*}

Filippo Menczer

Center for Complex Networks and Systems Research
School of Informatics and Computing
Indiana University, Bloomington, IN, USA

ABSTRACT

Social tagging systems lead to inferred relationships among resources, tags and users from shared annotations in support of applications such as search, recommendation, and navigation. However, users share annotations largely for their own individual needs, such as bookmarking and involvement in online communities. Free tagging may also result in low-quality annotation data. In this paper, we introduce a tagging game designed as an incentive for users to share a large number of high-quality social annotation data while being entertained. Preliminary results suggest that playing the game leads users to classify resources into broad categories.

1. INTRODUCTION

In the GiveALink project (cnets.indiana.edu/groups/nan/givealink) we are working on several questions concerning the construction and applications of socially driven semantic annotation networks. One of them is the development of games as incentives for social participation in the process of tagging online resources. Here we discuss a tagging game that builds upon existing work in GiveALink. Previous research includes the design of effective similarity relationships in the semantic space of social tagging systems [4, 5], applications to page recommendation [6], exploratory navigation interfaces [1], bookmark management [9], and social spam detection [3]. We wish to explore the use of this prior work, especially similarity measures among Web pages and tags, and spam features, in our design of an effective game.

Social annotation is one of the ways in which Web users organize their bookmarks autonomously, by tagging them with keywords. By sharing their annotations online and viewing other people's annotations, users participate in a community. There are a number of successful Web applications of social tagging, such as Delicious.com and Last.fm. Most popular social annotation systems allow users to store, organize, and share resources.

There is little motivation for the majority of users to annotate many resources with sufficient numbers of accurate tags [2]. Motivations such the convenience of centralized bookmark manage-

ment do not work for everyone, leading to low participation rate and short-time user involvement in tagging. The number of new pages that are posted per day to social annotation systems is small compared to the rate of growth of the Web [2]. The free tagging model also has some disadvantages: overly general tags; personal tags that are not meaningful to others; and tag abuse such as spam.

Our motivation for designing a game largely comes from the need to collect high-quality social annotation data, with the goal of supporting many applications (as discussed in § 2). The GiveALink tagging game proposed here aims to encourage users to tag Web pages, generating new triples by making the process of tagging enjoyable. The game algorithm and scoring mechanism are designed to favor specific and uncommon tags, and prevent cheating behaviors. The design objectives are explained in detail in § 3. We conclude the paper by describing our implementation of the game (§ 4) and outlining some preliminary analysis (§ 5).

2. BACKGROUND

An *annotation* is defined as a *triple*, or tripartite relationship between a *user*, a *tag*, and a *resource*. In the remainder of this paper we will also refer to a *link* as a relationship between a resource and a tag, supported by the users annotating the resource with the tag.

The collection of social annotation data has been shown to be useful to improve social navigation [7], Web search [2], personalized recommendation services [6], and social links prediction [10]. Because of the important use of social annotation data to improve other Web services, we turn to games to help us generate more useful data.

The GiveALink tagging game leverages entertainment as an incentive for the contribution of useful annotations. The idea of *games with a purpose*, or crowdsourcing through play, was originally conceived by von Ahn [11]. The most famous instance is the ESP Game, asking players to label images. Games are based on the human desire to be entertained, and may help solve problems that are hard for computers [11]. *Serious games* are an older, similar idea in the context of training in military, education, and public health settings [12]. Games in ClubBing.com are designed to promote the Bing search engine. Players have to use Bing as a tool to collect hints for winning, thus trying functions that they might not have noticed before. Winners are awarded real products, which is a strong incentive for players.

Previous research in the GiveALink project lays the foundation of the tagging game. The Maximum Information Path (MIP) similarity measure has been shown to be efficient to compute and effective at capturing relationships among pages, tags, or users consistently with external benchmarks [4, 10]. We leverage MIP and social spam detection features [3] to set up the game, assist the user toward victory, and prevent cheating, as described below.

^{*}Corresponding author. Email: weng@indiana.edu

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

KDD-HCOMP '10, July 25, 2010, Washington, DC, USA
Copyright 2010 ACM 978-1-4503-0222-7 ...\$10.00.

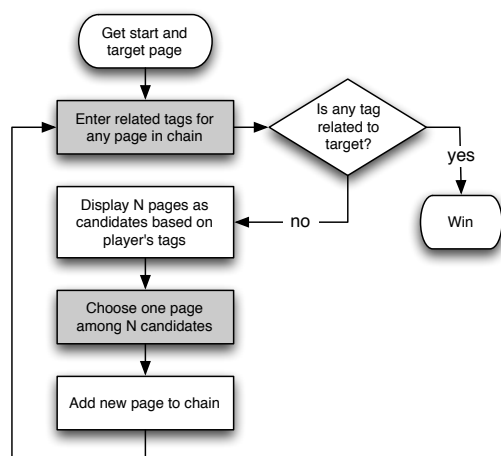


Figure 1: Flow chart of game logic. Gray boxes represent user actions, white boxes are system actions. Currently, $N = 3$.

3. GAME DESIGN

3.1 Basic Game Play

The basic idea of the game is to find a path (page chain) between two given pages by tagging. While players explore among online resources and try to build a path, new triples are generated.

The player is presented with a start page and a target page. The start page is the first node in the chain, and the player extends the chain by tagging any page inside the chain with one or more relevant tags. Each time the player enters tags for a page in the chain, the game displays a small set of pages based on those tags as candidates for the next node in the chain. The player is free to choose one of those candidates, and then select either this latest node or a previously tagged resource to be the next page to tag. The player wins the game when at least one of her latest tags are known to be relevant to the target. Therefore, once the player wins, a chain semantically connecting the start and target pages is successfully created. The logic is illustrated in Fig. 1.

3.2 Interface

The textfield for entering tags is placed between *target page* (top) and the *current page* (bottom), as shown in Fig. 2. The player needs to focus on tagging the current page and try to get it connected with the target. All pages in the chain are displayed in the cover flow style, so the player can easily switch between them. The url, title and thumbnail of each page are shown, so the player can try relevant tags based on that information or browsing the page by clicking on the thumbnail.

3.3 Mechanism Design

The data generated from the game is expected to be merged into the GiveALink triple store to decrease the sparseness of the semantics network of annotations and improve similarity calculations. Therefore, the game should be designed to strengthen relationships that appear weaker than they should be, or discover new relationships. At the same time, the game should be interesting enough to attract many players and should not be too hard to win.

The game mechanisms incorporate several design principles:

- Tags should be relevant to the current page in the chain; the more relevant they are, the more points the player can get.
- Links that exist in the GiveALink database are trusted, so they can be used as benchmarks.



Figure 2: Prototype game interface

- Links confirmed by many players can be trusted.
- New links are recorded until confirmed by other players.
- Once new links are confirmed by enough players, they can be added into GiveALink and treated as trustworthy later on.
- Candidate pages are selected based on the player's tags.
- Candidate pages should help the player approach the target, making the game easy to win.

The last principle requires an effective measure of similarity among pages. We use the Maximum Information Path measure from GiveALink [5]. MIP measures how similar a pair of tags or resources are. It has been implemented and integrated into the GiveALink system, and used as an important component in several applications, such as recommendation, bookmark management, and spam detection. The chain in the tagging game builds upon MIP similarities among pages, available via an API (GiveALink.org/api_doc); we select pages similar to the target to help the player.

Each tag for a given page (therefore each link) can be classified into one of three types:

1. **used**: tags that have been used *for this page* in GiveALink;
2. **suggested**: tags that have been suggested *for this page* by previous players, but not in GiveALink;
3. **new**: tags that have been used or suggested *for this page* by neither GiveALink users nor game players.

Because the **used** tags and the **suggested** tags are confirmed by either GiveALink or game players, they are more reliable. **New** tags can be relevant or irrelevant, so we need to wait for further evidence; they may turn into **suggested** tags when they are used for the same Web page more than once. **Suggested** tags may turn into **used** tags if they are confirmed by enough players. The players get more points with a larger proportion of **used** and **suggested** tags for a given page, as explained in § 3.4.

Once a player enters tags for the current page, a set of pages with any of these tags is selected based on their similarity to the target. The player should choose the next page that he believes is most similar to the target. The idea is that the similarity to the target increases as the chain is built, until the player can find a common tag between current and target pages, and win.

3.4 Scoring

Players are encouraged to provide more tags if possible, so more tags yield more points. The score of each tagging step is obtained by adding scores associated with each tag. Each tag is worth a number of points that depends on its specificity and the type of link it creates. Specific, trusted, and novel tags are most valued. The score for each tag t for the current page is given by two terms: $score(t) = \lambda(t) + \eta(t)$ where λ is a function of trustworthiness and novelty of t , and η is a measure of its *specificity*: $\eta(t) = \mu / \sum_{t' \neq t} \sigma(t', t)$ where σ is the MIP similarity between tags [5] and μ is a parameter. The denominator is a measure of *generality*. Tags that are similar to many other tags are general and therefore are worth fewer points.

According to the link classification, the trustworthiness and novelty functions are defined as follows:

- Discover trusted links: $\lambda(t_s) = \alpha \cdot f(t_s)$ where t_s is a **suggested** tag, f is the number of players who have suggested t_s for the current page, and α is a parameter. The player gets more points when more people have agreed with him in previous games.
- Find related tags: $\lambda(t_u) = \beta$ where t_u is a **used** tag for the current page and β is a parameter. We thus reward players who use relevant tags that have previously been used to annotate the same resource by GiveALink users.
- Add untrusted tags: $\lambda(t_n) = \gamma$ where t_n is a **new** tag for the current page and γ is a parameter.

Scores are added across all tags during each step of the game, and accumulated across the steps, yielding a cumulative score S . We set the parameters such that $\alpha > \beta > \gamma$ to prioritize based on the trustworthiness and novelty of links: among trusted links, novel ones (**suggested**) are more valued than known ones (**used**); untrusted (**new**) links are valued the least.

Additionally, winning the game yields a bonus that is larger if the player has won in fewer steps: $(S/\ell) + \delta$, where δ is a parameter and ℓ is the number of pages in the chain when the player wins.

3.5 Cheating

Cheating is an attempt to win the game without entering tags related to the current page. We define two types of cheating, each with its own solution.

Tag the Target. The first type of cheating is to tag the target page directly, without concern for the current one. This way the player can expect to win or at least get candidate pages similar to the target. To detect this behavior, the game determines whether the set of tags are relevant to the current page by computing the proportion of **suggested** or **used** tags. If most of the tags entered are **new**, the player is suspected of cheating. Specifically, a tagging step is deemed to be an instance of cheating if $\langle \lambda(t) \rangle_t - \gamma < \theta$, where $\langle \cdot \rangle_t$ denotes average over tags and θ is a threshold parameter.

Mixed Tags. The second type of cheating is to enter *some* tags that have been used for the target, but are irrelevant with respect to the current page. The player might use a mix of tags related to the current page and tags related to the target, such that the proportion of **used** or **suggested** tags is high enough to go undetected by the previous rule. We deal with mixed tags by noticing that tags aimed at different pages are likely unrelated to each other. We therefore use the *TagBlur* measure, introduced for social spam detection [3], to catch this condition. For a set of tags T entered by the player, *TagBlur* is computed from a measure of distance



Figure 3: Hints and score from the *Hintman* character

(dissimilarity) between tags, averaged across all the pairs of tags:

$$TagBlur(T) \propto \sum_{t_1 \neq t_2 \in T} \left(\frac{1}{\sigma(t_1, t_2) + \epsilon} - \frac{1}{1 + \epsilon} \right)$$

where ϵ is a small constant [3]. If the *TagBlur* of the player's tags is lower than the *TagBlur* of the current page in GiveALink, the player is suspected of cheating.

If no cheating is detected, the player can go forward toward the target (cf. § 3.3). If either cheating is detected, the player is not allowed to make progress; the candidate pages are selected among resources similar to the current page rather than the target.

3.6 Player-Friendly Design

A few game features are designed to make the game more fun. First, the target page and start page are not selected completely at random. To make a game easier to win, the player should be familiar with the two pages, and the two pages should be related. Players are registered users of GiveALink and may have already shared resources. If the player has annotated a sufficiently large number of pages (currently 50), we select one of these as the target, with some probability. Otherwise we select the page at random from the set of all GiveALink resources that have a minimum number of tags (5) and similar pages (5), and have been tagged by a minimum number of users not labeled as spammers (2). The start page is not chosen independently. We select it from the set of pages with small, non-zero similarity to the target. This way the start page is likely to be weakly related to the target — in fact they share at least one tag.

Spam can distract players and result in useless triples. Although the GiveALink API methods employ a spam filter, some spam can go undetected. The player can report a page as spam by simply clicking a button. Spam reports are stored, flagging suspicious resources and helping improve the accuracy of the classifier employed by the GiveALink spam detection system [3].

Finally, we set up a mechanism to display hints along with each player move, named *Hintman* (Fig. 3). Hintman helps the player review each move through different types of hints, telling him whether he is getting closer to or further away from the target, or whether his tags are related with the page in the previous step.

4. IMPLEMENTATION

When we combine the design principles, scoring rules, and anti-cheating mechanisms, we obtain the algorithm in Fig. 4. We have not yet implemented the specificity mechanism discussed in § 3.4.

The tagging game uses its own server and database for storing basic information on the players, triples generated during the games, scores, and other records.

The game implementation makes use of multiple GiveALink API methods. Initial pages are generated by the *url.generate* method. Then player u enters a set of tags T for page r , and the *url.getTags* method is used to judge whether tag $t \in T$ is **used** or not for r . If not, the triple (u, t, r) is added into the game database. A confirmed link (t, r) can be added into GiveALink via the *annotation.add* method. The similarity among pairs of tags in T is available through the *tag.similarity* method, necessary for the calcula-

```

Display start & target pages
UNTIL (win)
  Player picks start or next page as current page
  Player enters tags for current page
  FOREACH tag
    IF tag is suggested
      score += alpha * f(tag, current)
    ELSIF tag is used
      score += beta
    ELSE
      score += gamma
  IF [score >= (theta + gamma) * |tags|
    AND TagBlur(tags) < TagBlur(current)]
    IF win
      score += score/steps + delta
    ELSE
      Select pages similar to target as choices
  ELSE
    Select pages similar to current as choices

```

Figure 4: Pseudocode of the game algorithm

Table 1: Basic game data statistics

	Minimum	Average	Maximum
Score	10	194.4	775
Steps	1	2.7	9
Time	00:03	04:26	11:59

tion of *TagBlur*. For efficiency, *TagBlur* is precomputed and retrievable via the API. With *tag.geturls* and *url.targetSimilarity* we can collect resources annotated with tags entered by the player, and rank them by their similarities to the target or current page.

The *cold-start* problem can negatively affect the performance of the game when there are only a few annotations available to build the chain. Therefore in our initial implementation we employ the Delicious API to obtain recommended tags for the target and accelerate the confirmation of new triples.

5. PRELIMINARY DATA ANALYSIS

The game prototype is at GiveALink.org/giveagame. Since we started testing within our research group, we have collected a small dataset consisting of 1,079 suggested and 265 confirmed triples. These annotations were generated by 19 players in the course of 97 games. Basic statistics are shown in Table 1.

In Table 2 we report, for the tags that appear in the most suggested triples, the *generality* (cf. § 3.4) and the percentage of GiveALink tags that are more specific. These preliminary results suggest that general tags, which can describe many resources, are easier to discover through the game; game players tend to classify pages in broad categories. This confirms our intuition that to build a chain one must move to a general category encompassing the start and target pages. At the same time we see the need to promote the use of specific tags as discussed in § 3.4.

Table 2: Top 10 tags in suggested triples

No.	Tag	Freq.	Generality	Percentage
1	online	24	35591.1	99.98%
2	italia	16	112.9	89.41%
3	web	13	15576.4	99.97%
4	blog	12	4903.5	99.88%
5	music	11	4516.3	99.87%
6	italy	10	478.1	96.87%
7	code	9	2406.4	99.60%
8	service	9	10880.4	99.96%
9	design	9	12613.6	99.97%
10	computer	8	3984.1	99.82%

6. CONCLUSION

Improving participation in social tagging systems is an important goal, since user-generated annotations are valuable. We have described the design of a game as an incentive for quality tagging, to relieve the problem of sparse annotation networks and support other Web applications.

Our design principles lead to basic rules to make the game enjoyable. Novel tags are rewarded in the game, but they need to be confirmed by multiple players. When a user is playing, she needs to use fresh tags and guess what might be used by other players. Therefore, the data originated by the game provides new triples with confirmed relatedness between tags and resources in each link.

The game design and implementation are under continuous development and testing. We plan to incorporate specificity into the scoring mechanism to reward specific or fresh tags. During the current internal test, both the mechanisms and interface are being refined. We are also tuning the values of the various scoring parameters. Once the game passes this internal test, it will be advertised to the public to generate more annotation data.

Analysis of the annotation data from the game and its comparison with existing social tagging systems will allow us to explore some interesting questions. First, we will evaluate the quality of game data: whether tags are more or less specific, and more or less relevant, compared to those from bookmarks. Second, we will track changes in the structure of the folksonomy built from the relationships between tags, users, and resources when adding the game data. For example, we want to see if the game makes the similarity networks among tags and resources more “metric” [8]. Finally, we plan to compare the speed with which triples are generated via the game versus traditional bookmarking.

Acknowledgements. We thank C. Cattuto, R. Schifanella, and NaN members (cnets.indiana.edu/groups/nan) at the IU School of Informatics and Computing for helpful discussions and participation in testing. This work is funded by NSF award IIS-0811994.

7. REFERENCES

- [1] J. Donaldson, M. Conover, B. Markines, H. Roinestad, and F. Menczer. Visualizing social links in exploratory search. In *Proc. 19th Conf. on Hypertext and Hypermedia (HT)*, 2008.
- [2] P. Heymann, G. Koutrika, and H. Garcia-Molina. Can social bookmarking improve web search? In *Proc. 1st WSDM Conf.*, 2008.
- [3] B. Markines, C. Cattuto, and F. Menczer. Social spam detection. In *Proc. 5th AIRWeb Workshop*, 2009.
- [4] B. Markines, C. Cattuto, F. Menczer, D. Benz, A. Hotho, and G. Stumme. Evaluating similarity measures for emergent semantics of social tagging. In *Proc. 18th Intl. WWW Conf.*, 2009.
- [5] B. Markines and F. Menczer. A scalable, collaborative similarity measure for social annotation systems. In *Proc. 20th Conference on Hypertext and Hypermedia (HT)*, 2009.
- [6] B. Markines, L. Stoilova, and F. Menczer. Social bookmarks for collaborative search and recommendation. In *Proc. 21st Natl. Conf. on Artificial Intelligence (AAAI)*, pages 1375–1380, 2006.
- [7] D. Millen and J. Feinberg. Using social tagging to improve social navigation. In *Adaptive Hypertext Workshop on Social navigation and community-based adaptation*, 2006.
- [8] L. M. Rocha. Semi-metric behavior in document networks and its application to recommendation systems. In *Soft Computing Agents: A New Perspective for Dynamic Information Systems*. 2002.
- [9] H. Roinestad, J. Burgoon, B. Markines, and F. Menczer. Incentives for social annotation. In *Proc. 32nd SIGIR Conf.*, page 838, 2009.
- [10] R. Schifanella, A. Barrat, C. Cattuto, B. Markines, and F. Menczer. Folks in folksonomies: Social link prediction from shared metadata. In *Proc. 3rd WSDM Conf.*, 2010.
- [11] L. von Ahn. Games with a purpose. *IEEE Comp.*, 39(6):92–94, 2006.
- [12] M. Zyda. From visual simulation to virtual reality to games. *Computer*, 38(9):25–32, 2005.

Crowdsourcing Participation Inequality: A SCOUT Model for the Enterprise Domain

Osamuyimen Stewart
IBM Research
1101 Kitchawan Road
Yorktown Heights, NY
011+1-914-945-1137
ostewart@us.ibm.com

David Lubensky
IBM Research
1101 Kitchawan Road
Yorktown Heights, NY
011+1-914-3261
davidlu@us.ibm.com

Juan M. Huerta
IBM Research
1101 Kitchawan Road
Yorktown Heights, NY
011+1-914-9453378
huerta@us.ibm.com

ABSTRACT

In large scale online multi-user communities, the phenomenon of ‘participation inequality,’ has been described as generally following a more or less 90-9-1 rule [9]. In this paper, we examine crowdsourcing participation levels inside the enterprise (within a company’s firewall) and show that it is possible to achieve a more equitable distribution of 33-66-1. Accordingly, we propose a SCOUT ((S)uper Contributor, (C)ontributor, and (OUT)lier)) model for describing user participation based on quantifiable effort-level metrics. In support of this framework, we present an analysis that measures the quantity of contributions correlated with responses to motivation and incentives. In conclusion, SCOUT provides the task-based categories to characterize participation inequality that is evident in online communities, and crucially, also demonstrates the inequality curve (and associated characteristics) in the enterprise domain.

Categories and Subject Descriptors

H.1.2 [User/Machine Systems]: Human factors, Software psychology H.5.3 [Group and Organization Interfaces]: Computer-supported cooperative work, web-based interaction

General Terms

Design, Human Factors

Keywords

Crowdsourcing, online community, incentives, motivation

1. INTRODUCTION

Crowdsourcing is generally described as a web-based activity that harnesses the creative contributions of a diverse large network of individuals (the crowd) through an open call requesting for their participation and contributions [3, 4, 7, 8, 12, 13, and 14]. After the open call is issued, there are various views on crowd response or composition in terms of participation and contribution. Nielsen [9] based on existing works [5, 15] has analyzed the phenomenon of ‘participation inequality’ in online communities and advocates

a general 90-9-1 rule: (a) 90% of users are ‘lurkers’ (i.e., they read or observe, but don’t contribute), (b) 9% of users contribute from time to time, but other priorities dominate their time, (c) 1% of users participate very often and account for most contributions (it can seem as if they don’t have lives because they often post just minutes after whatever event they are commenting on occurs). Based on this, participation inequality is deemed to be a function of human behavior and one that is almost impossible to overcome [9]. This presents a major challenge for crowdsourcing inside the enterprise (within a company’s firewall) due to serious time constraints and work pressures that greatly inhibit employees’ participation and coupled with the fact that motivation and incentives are different from the public domain [11]. Thus, the only real choice is in how online crowdsourcing communities inside the enterprise can shape the inequality curve’s angle differently from the 90-9-1 distribution. This is the issue addressed in this paper.¹ We argue for a SCOUT (Super contributor, Contributor, OUTlier) framework wherein 33% are the (OUT)liers who do very little, 66% are the (C)ontributors who provide moderate contributions, and 1% are the (S)uper contributors who offer super effort. This is based on empirically motivated crowd categorization from quantifiable (measurable) task-based or effort-level features pertaining to the quantity of contributions and responses to motivation, correlated with attitudes towards incentives.

Crowdsourcing based on enterprise data from employee participation in translation tasks provides the background for our research, conducted inside a multinational company (IBM) with about 400,000 employees spread over 160 countries. We developed the community to help us with the data collection effort required for improving statistical machine translation algorithms, by harnessing the linguistic skills of worldwide bilingual employees for accomplishing the complex translation task that is typically done by professional translators. The open call to the crowdsourcing community was to help translate sentences from English to their native languages (or vice versa) including Arabic, Chinese (simplified), Chinese (traditional), French, German, Italian, Japanese, Korean, Portuguese, Russian, and Spanish. Participants are presented with text of relevant data e.g., news, technical content, history, etc., in a source language and asked to translate into a target language.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

KDD-HCOMP’10, July, 25, 2010, Washington, DC, USA.
Copyright 2010 ACM 978-1-4503-0222-7...\$10.00.

¹ Additional co-authors are: Julie Marcotte (IBM GBS), Cheng Wu (IBM Research), Andrzej Sakrajda (IBM Research)

2. THE SCOUT MODEL

As an emerging field in web-based, computer-supported cooperative work, there are very few fully developed models of crowdsourcing participation besides some foundational work [9, 14] which focus on online communities like Wikipedia, blogs, and social networking collaborations for which one can get substantial value from lurking rather than actually contributing. As far we know, there is no research that has attempted to formalize the differences in crowd behavior with respect to the *nature of the task*, beyond merely listing types of crowdsourcing [4, 7, 8, and 14]. We would like to propose an initial typology based on three relevant distinctions summarized as follows:

Collectivistic: this involves a task whereby several people are handling small parts of a larger problem (e.g., uTest) and the community exhibits the need to collaborate and network with each other using available social tools.

Individualistic: this involves a task whereby many people are contributing (individually) towards a single goal (e.g., iStockphoto) and do not need to “socialize” with each other.

Collectivistic-individualistic: a mixture of collectivistic-individualistic properties wherein the crowd is made up of subgroups who form (cooperate or collaborate) to compete against other subgroups in the quest of completing a task (e.g., 2009 DARPA experiment in crowdsourcing won by MIT²

Our implementation focused on the individualistic community in which the individual-participants are engaged in a translation task. This is similar to the iStockphoto community described in [3] as both of them share certain similarities: there is no need for social networking or collaboration amongst the participants.

Therefore, we propose a model of participation based on level of effort that can be measured in terms of quantity of contribution and the nature of participants’ motivation (incentives). We postulate three broad categories of crowd participation: (S)uper contributors, (C)ontributors, and (O)utliers (SCOUT):

Super contributors: This is an elite group of participants who give super effort by going above and beyond the ordinary. Super effort can be measured in terms of quantity (usually in the thousands). They are highly motivated members who participate for altruistic reasons as well as being overtly intrinsically motivated, and less concerned about the extrinsic motivation.

Contributors: This group of participants give moderate effort and contribute just enough to keep in pace to make it to the set goals for getting a prize. Moderate effort can be measured in terms of quantity (usually in the hundreds). They are heavily focused on the rewards (extrinsic motivation) and are always conscious of where they rank (at each point) relative to the reward and the end date of the task that is being accomplished.

OUTliers (crowd): This group of participants may be considered as leisurely contributors (they are not lurkers, because they do contribute). Leisurely effort can be measured in terms of quantity (usually in the tens). In terms of motivation, they appear to have enough interest in participating in the crowdsourcing community but are not sufficiently engaged to make any solid contributions.

3. MEASUREMENT METHOD

We set out to validate the proposed SCOUT model of crowd participation by issuing an open call to the worldwide employees (about 400,000 spread over 160 countries) to join the crowdsourcing data collection effort as volunteer translators. This call was issued in the form of a “Challenge” or a “Sprint” so that whoever accumulates the most points (from translations) at the end of an announced end date will be declared as the winner and recognized with some reward. A multi-tiered incentive structure was created to test the impact of both intrinsic and extrinsic motivation in crowdsourcing [1, 2, and 6]. For enterprise-specific constraints, participants did not personally keep the monetary awards but by virtue of winning were able to give it as a charitable donation to designated charities. Individuals in the top ten with the most points in a language received the highest amount which they could donate to a charity of their choice. The next top five participants (10-15) received the second highest amount for donation to a charity, while the next five winners (15-20), rounding up the top twenty winners in the language, received the minimum award that could be donated to a charity. It is important to note that the winners were given the option of either making a charitable donation or selecting a personal award (trinkets, bags, wrist watches, etc) that they could keep.

We set up low barriers for all participants to be able to participate in this Challenge. All that was required was a simple login (using the existing employee’s intranet id and password), and then a simple registration with just two mandatory steps that allows users to select the relevant language(s) and agree to the Terms and Conditions for participation.

The Challenge ran for eight weeks and during this time participants received real-time updates of their points and their relative ranking in the Challenge in terms of where they stood relative to the top 15 overall contributors. On a weekly basis, the relative ranking in each language community was sent by email to the members of a language group to know their status. A dynamic leaderboard showing real-time update of the top 15 overall contributors as well as the relative (real-time) contributions of each language were also provided.

3.1 Results

At the completion of the Challenge, the following table summarizes the contribution and overall participation.

Table 1. Summary of Challenge contributions

Activity	Totals
Words translated	2.4 Million
Sentences translated	123,646
Countries with participants	47

A total of 2.4 million words (from 123,646 sentences) were translated in eight weeks by 967 bi-lingual employees from 47 countries representing the eleven languages within the scope of the crowdsourcing translation task. Some important patterns emerge in support of the SCOUT model which we will now discuss.

² <https://networkchallenge.darpa.mil/default.aspx>

3.2 Validating the SCOUT model

The resulting overall contribution by the various participants (measured in sentences) was heavily skewed consistent with the Zipf curve [10].

Table 2. Distribution of total contribution in sentences

Contributor Rank	Total Sentences
Winner (1 st place)	9345
10 th place	2711
20 th place	2650
30 th place	727
40 th place	532
50 th place	341
100 th place	148
100++ place	Below 100

When we plot this data in Table 2 in a chart (Figure 1), we observe that a few participants have large contributions, many are in the middle, and the rest provide only small contributions.

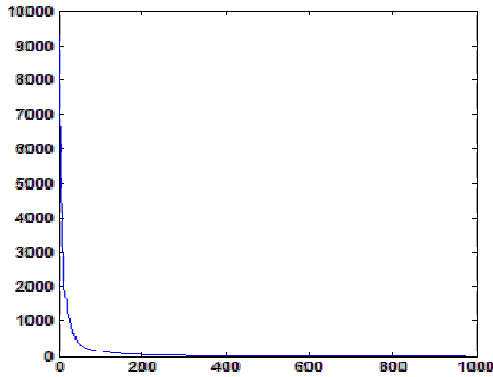


Figure 1. Plot of participant's contributions

In light of the SCOUT crowdsourcing model we are proposing, we can extrapolate some quantifiable conclusions relative to effort-level from the total contributions and surmise that:

- The top 20 contributors (out of 967) made contributions in the thousands (between 2,000 and 10,000 sentences).
- The top 20-100 contributors (out of 967) made relatively smaller contributions in the hundreds (between 100 and 700 sentences).
- The rest of the participants only made even smaller contributions in the teens (below 100 sentences).

4. DISCUSSION

We have hypothesized earlier on that crowd participation can be understood in terms of the consistent behavior of the three groups relative to their contribution (quantity) and their response to motivations (incentives). We will now discuss these respectively.

4.1 Impact of model on quantity

We can validate the SCOUT model from the relative impact of the contributions by the different crowd groups to the total effort (i.e., relative to the total number of sentences contributed).

Table 3. Impact of model on quantity

Participants	Sentences	Total data
1%	2,000-10,000	44%
66%	100-999	54%
33%	Below 100	2%

The following generalizations can be derived from Table 3: the contributions of 1% (Super Contributors) accounts for 44% of the overall contributions by the community, while the Contributors make up 54% of the total contributions and the Outliers make up the rest (which is a miniscule 2%). This approach to the crowdsourcing participation levels provides an empirical basis for better understanding how the 90-9-1 distribution rule applies in the enterprise domain. On the basis of quantity of contributions, the data points to a more equitable 33-66-1 distribution.

4.2 Impact of model on motivation

We can further validate the SCOUT crowdsourcing model by systematically examining the motivation (responsiveness to communication) of each group and then correlating the group behavior to their corresponding responses to crowdsourcing incentives. First, we present the participation trend during the eight-week period of the Challenge.

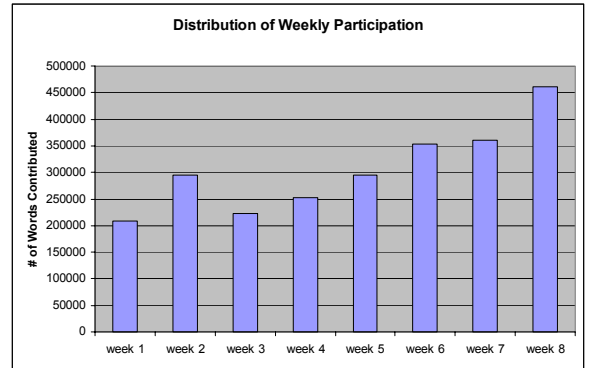


Figure 2. Weekly distribution of crowd participation

As the chart shows, aside from a slight aberration in the second week, we see a steady climb in the crowd contribution with each passing week during the Challenge. This steady trend in contribution was facilitated (triggered) via weekly communication with the crowdsourcing community: several strategies were used to communicate and interact with the participants during the Challenge including email, social media, blogs, newsletter, etc. each with varied and measurable impact on crowd response and behavior. By tracking the responsiveness by the crowd the day following each of these communications, we observe the following generalization regarding the three SCOUT groups:

Table 4. Crowd groups' response to communication

Group	% Response
Super Contributor	100%
Contributor	60%
Outliers	35%

If participation (based on the response the next day after each communication is released) is combined with the previous observation about a steady weekly increase in the contributions (Figure 2), then we can conclude that there is a hierarchy of distinctive motivation qualities that can be derived: Super contributors are highly motivated (100% response); Contributors are moderately motivated (only 60% response); OUTliers are very mildly motivated (only a meager 35% response).

The foregoing analysis is significant not only because it confirms the validity of the three SCOUT groups, but it also provides the basis for examining another contrastive behavior relative to the incentives (awards). At the end of the Challenge, the winners were asked to pick one option: either to make a charitable donation or select a personal award (trinket, wrist watch, etc). The following table summarizes how the groups responded:

Table 5. Group behavior regarding incentives

Group	% Response
Super Contributor	95% donation to charity
Contributor	55% donation to charity
Outliers	n/a

Based on these percentages of the preferences selected by the winners, we see that only 5% of the Super contributors chose personal awards. This can be taken as indicative of a key characteristic of this group: they participate for mainly altruistic reasons. By contrast, 45% of the Contributor category selected personal awards, thus confirming that majority of the members of this group participate for extrinsic reasons.

5. CONCLUSION AND FUTURE WORK

While there are many challenges with crowdsourcing participation inside the enterprise, however, we have observed a more equitable 33-66-1 distribution different from the 90-9-1 rule. This offers great hope for the future of crowdsourcing inside the enterprise. On the basis of quantitative and empirical evidence with respect to contribution levels and the nature of motivation, we formalized three components of the SCOUT model of crowd participation: (S)uper Contributors are the 1% who consistently give super effort in terms of quantity and are driven mainly by altruism (intrinsic reward); (C)ontributors are the 66% who provide moderate effort in terms of quantity and are mainly driven by extrinsic reward; and OUT(liers) are the 33% that only provide low-level effort not sufficient for receiving an award. In the future, we will attempt to replicate the SCOUT model outside

of the firewall, in the public domain, to see if the approach of using quantifiable effort-levels will yield similar results.

6. REFERENCES

- [1] Blumler, J.G. 1979. The role of theory in uses and gratifications studies. *Communication Research*, 6,(1), 9-36
- [2] Bonaccorsi, A. and Rossi, C. 2004. Altruistic individuals, selfish firms? The structure of motivation in open source software. *First Monday*, 9, (1),
- [3] Brabham, D.C. 2008a. Moving the crowd at IStockphoto: The composition of the crowd and motivations for participation in crowdsourcing application. *First Monday*, 13, 6
- [4] Brabham, D.C. 2008b. Crowdsourcing as a model for problem solving: an introduction and cases. In *Convergence: International Journal of Research into New Media Technologies*, 14, (1), 75-90
- [5] Brothers, L. Hollan, J., Nielsen, J., Stornetta, S., Abney, S., Furnas, G., and Littman, M. 1992. Supporting informal communication via ephemeral interest groups. In *Proceedings of ACM Conference on Computer-supported Cooperative Work, CSCW '92*, 84-90
- [6] Hars, A. and Ou, S. 2002. Working for free? Motivations for participating in open source projects. In *International Journal of Electric Commerce*, 6, (3), 25-39
- [7] Howe, J. 2006. The rise of crowdsourcing. *Wired*, 14, 6, <http://www.wired.com/wired/archive/14.06/crowds.html> (June, 2006)
- [8] Howe, J. 2008. *Crowdsourcing: Why the Power of the Crowd is driving the Future of Business*. Random House Publishers
- [9] Nielsen, J., 2006. Participation inequality: Encouraging more users to contribute. http://www.useit.com/alertbox/participation_inequality.html [accessed May 3, 2010]
- [10] Nielsen, J., 2006. Zipf curves and website popularity. <http://www.useit.com/alertbox/zipf.html> [accessed May 3, 2010]
- [11] Stewart, O., Huerta, J., Sader, M., Sakrajda, A., Marcotte, J., and Lubensky, D. 2009. Designing crowdsourcing for the enterprise. SIG KDD HCOMP Workshop, (June 2009)
- [12] Surowiecki, J. 2004. *The Wisdom of the Crowds: Why the many are Smarter than the few and how Collective Wisdom Shapes Businesses, Economies, Societies, and Nations*. Doubleday, New York
- [13] Surowiecki, J. 2005. *The Wisdom of Crowds*. Anchor
- [14] Viitamaki, S. 2008. The FLIRT model of crowdsourcing: planning and executing collective customer collaboration. MA thesis: Helsinki School of Economics.
- [15] Whittaker, S., Terveen, L., Hill, W.C., and Cherny, L. 1998. The dynamics of mass interaction. In *Proceedings of ACM Conference on Computer-supported Cooperative Work, CSCW '98*, 257-264

Mutually Reinforcing Systems

John Ferguson
Department of Computing
Science
University of Glasgow, UK
john@dcsgla.ac.uk

Marek Bell
Department of Computing
Science
University of Glasgow, UK
marek@dcsgla.ac.uk

Matthew Chalmers
Department of Computing
Science
University of Glasgow, UK
matthew@dcsgla.ac.uk

ABSTRACT

This paper demonstrates strategies for designing mobile games with by-products in order to allow the acquisition of specific data. A mobile game with by-products called *EyeSpy* and a photo website called *Realise* will be used as examples to demonstrate these strategies. The Realise website allows users to browse geographically tagged photos and make specific requests for new ones. In the EyeSpy game, players use mobile phones to tag geographic positions with photos and text. EyeSpy players can earn points from validating each others' tags by visiting tag locations and attempting to 'confirm' them. If players go to the correct location, both the player confirming the tag and the player who created it will gain points. This creates game content for EyeSpy and provides more refined results for the Realise website. In this way, both the systems mutually reinforce each other.

Categories and Subject Descriptors

H.5.m [Information Interfaces and Presentation]: Miscellaneous

Keywords

Human computation, mutually reinforcing systems, games with by-products, mobile multiplayer games, mobile photography.

1. INTRODUCTION

It has been demonstrated that 'human computation' games (such as the ESP Game [3]) can be effective in producing by-products as a side effect of play. These games often produce generalized by-products and are not designed for the benefit of a specific system. While this approach may maximize the potential usefulness of the by-products, there will be occasions when specific data is required that may not have been produced. This approach of creating general by-products often leads users to come up with the same solutions for similar problems. Players may see generalities in

the game play and feel they have solved the problem before. This can lead to a feeling of monotony. The similarity in the designs of many games with by-products means that a successful method of making specific requests for data could be adapted to work on many games.

General improvement could be achieved by designing systems which work more closely with each other. A system that is good at producing one kind of data can share that strength with other systems, or two systems that produce the same type of by-product can be used to form a larger dataset. If a system is bad at gathering one kind of data, it makes sense that it should be able to request that data from another system which is better suited to the task.

Mutually reinforcing systems have the ability to receive and act on requests and the ability to make requests of each other. By working together to improve the efficiency and quality of by-product creation, they would each do a better job of achieving their goals. The loop that they would form, where each system can make requests of the other, as well as responding to requests and sharing data, is a strong element of mutually reinforcing systems. This loop is the reinforcing element that allows the improvement of function.

In order to explore requesting specific data and making the games more diverse and enjoyable for the players, we developed two systems that can be described as mutually reinforcing. This means that each system provides a benefit to the other such that each of them better achieves its goals.

We have augmented a game called *EyeSpy* [1] so that it can deal with specific requests for data from a second system, called *Realise*. In EyeSpy, users created text and photographs, described as 'targets,' which were associated with geographic locations. The geo-tagged photographs would then have text associated with them (the text attached to the same location). Other users would then have to visit these locations to confirm the targets. However, they had no means of knowing where the targets were, other than the photographs or texts themselves. This design encourages people to produce targets with enough detail to be located easily. Realise is a website that allows users to browse the geo-tagged photographs produced by EyeSpy, see their location on a map and search them based on nearby text targets. It also allows users to make requests for new photographs by describing the photograph they want to the EyeSpy players. These appear in the game as a special target type that requires the players to create a photograph that matches the description they have been given.

By allowing users to request specific photographs from the EyeSpy players, Realise's data set is broadened and its

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

KDD-HCOMP'10 July 25, 2010, Washington, DC, USA.
Copyright 2010 ACM 978-1-4503-0222-7 ...\$10.00.

usefulness is increased. It does not have to rely solely on the players' decisions about what should be in the data set and can allow for any holes in the data set to be filled. Also, by introducing an element into the game that is not defined by the players themselves, the game will have more diversity and become less monotonous for the players.

2. DESIGN

The rules in EyeSpy encourage photographs that are created specifically for use by other people because the tags must be confirmed. In a sense, the photos are created specifically so that they will be easy for others to find.

The EyeSpy game was created such that players would create a set of photographs that were geo-tagged and had text associated with them. However, the text might not necessarily describe the photographs themselves, but the area in which they were taken. This would mean that searching the set of images on the Realise website for a specific term would return not only images that the term described, but also images from the surrounding area. It also means that a new photograph taken in an area which had previously been tagged with text would immediately be labeled with this information.

The Realise website allows users to make requests for photographs when the existing dataset does not satisfy their needs.

This most likely happens when users search for an image and do not get the results they want. They can then request that better results are produced. These requests can be very specific, but they can also rely on local knowledge or human decision making, such as, 'a tree,' or, 'the nearest subway to Glasgow University.' In this sense, the website is no longer just an image search engine with the option to request that 'holes' in the dataset be filled. An entirely new function has been added where labeling of images is descriptive in a subjective way. This ability would not have been possible without being part of a mutually reinforcing system loop. This is generally avoided in games with by-products because objective data is considered more valuable (leading to agreement rules between randomly paired players in order to create such objectivity).

While the Realise user may be happy with a photo because it fulfilled their particular needs, that does not mean it is labeled in a useful way for other users. The existing game mechanic can be helpful in this instance. Since the photo that was taken for the priority target had location information, it can still be used with the existing text targets for that location. However, we still need to know if the image is generic enough to be part of a general search for that area. This can be achieved by putting the image back into the game as a regular photo target. This also means that players who make priority targets will have the opportunity to get points from the photo a second time when it becomes a photo target. If enough players confirm the photo as a photo target, it means it is recognizable enough to be used as a search result on the Realise website.

Varied game experience

EyeSpy was designed to be a self contained system. An entirely new game mode needed to be added for it to function as part of a mutually reinforcing system loop. A new type of target was added to the game. This 'priority' target had different rules from the existing target types and was supplied

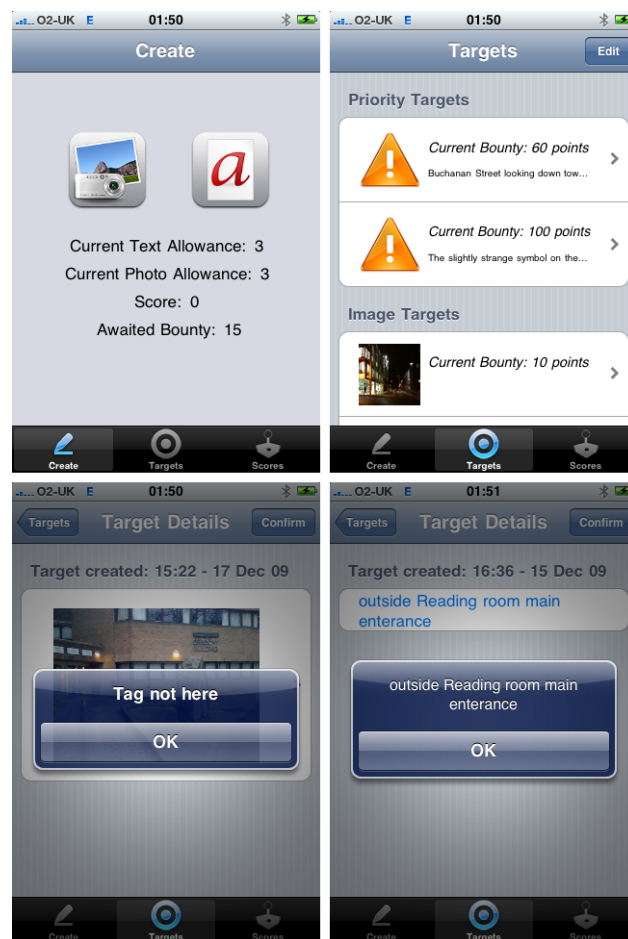


Figure 1: Screen shots from EyeSpy showing, clockwise from top left: the create screen (where players make image and text targets), the targets screen (where players browse the targets to be confirmed), a text target screen (where users can confirm a text target and see its full text), and an image target screen (where users can confirm an image target) showing an unsuccessful confirmation attempt.

by the website users rather than the game players. When the website users make a request, it comes into the game as a priority target. This is a textual description of the kind of photo for which the website user wants a picture. Unlike the confirm procedure for the other target types (where the player is checking to see if they are in the same place the target was created), the priority targets require the players to create a photo that matches the priority target's description. Since the Realise website supplies no location information with a request, priority targets can be confirmed in any location. The photo is then sent back to the Realise user who requested it to see if it is what they wanted. The EyeSpy player will not receive any points until the Realise user has accepted the photo as being what they requested.

Allowing the EyeSpy players to fulfill the requests of the website users creates a different game dynamic than when they are creating and confirming targets amongst themselves. The website users make requests for photographs that are not already in the system. This means there is a

greater likelihood that the requests will be for things that the players would not pick on their own, which makes the game more varied. Additionally, the website users may make requests which call upon the EyeSpy players to give subjective opinions which are not supported by the existing game dynamic.

Equilibrium between systems

The website users create bounties for their requests. Each day that a Realise user logs into the website, they are given points which are added to a total. These points can be assigned to requests for photos. EyeSpy players will receive these points if they successfully create the required photo.

The number of points that a Realise website user is assigned is based on the overall system activity for the previous day. This means that if a smaller percentage of website requests were dealt with by the game players, the number of points that the website users are assigned will be less. This means that users will want the same number of photos, but will have fewer points to spend. This measure is used to encourage people to request fewer photos until the EyeSpy players can catch up with the requests already in the system. In assigning requests to players, the system will also favor requests that have more points assigned to them. This adds more value to the points as it indicates that the more that Realise users assign to a request, the more quickly they are likely to receive a response. This should encourage more points being used on a smaller number of requests.

The system also gives each Realise user an allowance of requests that can be made each day. This allowance does not accumulate and is reset every day. This allowance is also adjusted automatically based on the overall activity in the system. If there are greater numbers of requests awaiting photos, there will be fewer requests that are allowed to be made. In some instances where there are too many requests awaiting photos, this may mean that a Realise website user is not allowed to make any requests on a given day.

These measures mean that each Realise user has a fair chance of getting photos for their requests. This method of valuing the work between systems in a mutually reinforcing system loop is important. The systems should form a partnership without one becoming too dominant, or else the equilibrium will collapse and the benefits of mutually reinforcing systems might be lost.

Using multiple systems for validation

If a requested photograph is satisfactory, the Realise user will pay the bounty (giving the EyeSpy player the points). This allows the Realise user to access the full sized photograph (they must make their determination of whether a photograph is satisfactory based on a smaller thumbnail version). This has the double effect of validating that the photograph matches the request text (which can now be used as a label for it) and giving the player and website user what they wanted (respectively, points and a full sized photo).

The photograph itself will return to the game as a regular photo target so that it can be confirmed by other players as well before it is added to the usable data set.

The website users and game players can also monitor each other for inappropriate behavior. The website users can report inappropriate photographs that appear on the website, and the game players can report inappropriate requests made of them. Any person who has enough targets or re-

quests reported can then be banned from using the systems. This banning is not automatic. Once the threshold has been reached, an administrator is alerted to review the activity and determine if a ban is warranted. This prevents game players from colluding to try and get another player banned by flooding the system with reports of inappropriate behavior when there is none. It is, however, unlikely that this would happen because only a target or request can be flagged, not a person. Since the person who creates a target is kept anonymous, it would be hard for other players to target anyone specifically. Admittedly, banning people will not prevent them from creating a new account and repeating their abusive behavior, but they will have to start with no points to spend on the website, or at the bottom of the leader board in the game. More extreme methods of identification could be included to make people accountable for their actions (such as sending an authentication code in the post to someone's home address, making it difficult to register multiple times). However, unless a serious problem of abusive behavior is demonstrated in the system, this would merely detract from the experience of the users.

In addition to getting targets which are more interesting to confirm, using the second system to validate requests adds a new scoring system to EyeSpy that would not have been possible without mutually reinforcing systems. The new priority targets act like a bonus in the game. The number of priority targets received and how many points they are worth will appear to be random to the EyeSpy players, making the game play less monotonous. Additionally, the requests themselves will not always be the same objective type as in the rest of the game. They may now include non-location based requests for specific objects, requests which have potentially more than one correct answer and requests which may require the player to give an opinion.

3. IMPLEMENTATION

The systems each have a client and a database. The EyeSpy game is currently implemented as a phone-based game using the phone's GPS hardware and camera to create geo-tagged photographs and text. The EyeSpy database contains all these geo-tagged photographs and text so that other systems can access the dataset through a web service.

The Realise website accesses the EyeSpy web service so that it can display the photographs to the website users. It also contains its own database storing user and request information. In addition to this, the EyeSpy web service allows the website to add requests to the EyeSpy game.

The Realise website was implemented as a search engine. When a user searches using a keyword, the website sends a request to the EyeSpy web service telling it the keyword and how the results should be ordered. The EyeSpy web service then checks to see if that keyword exists in any of the text labels in the dataset. For every matching label that is found, the EyeSpy web service returns images that were taken in the same area as the matching label.

In order to allow the Realise website users to see where a photo was taken, the Google Maps API was used to create a pin showing the location. The main search results page for Realise can be seen in figure 2.

4. TESTING THE SYSTEMS

Three trials were carried out to test the augmented version

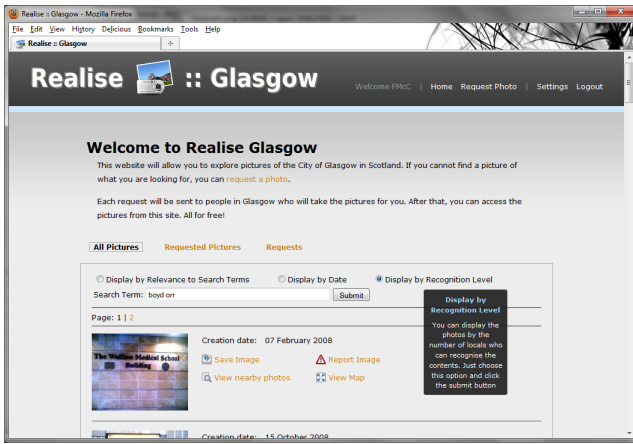


Figure 2: The Realise website showing results for a user entered search. For each result, users can see the full sized image, save the image, report the image, see the image’s location on a map or see photos that are near it. The results can be ordered by relevance, date or recognizability (the number of times the image was confirmed by EyeSpy players).

of EyeSpy and the Realise website, however, the addition of a bounty system was not included in the first trial. Initially, all priority targets were worth the same number of points, but the Realise website users were still required to confirm them. However, the Realise users quickly flooded the system with requests and confirmed almost all the images that were submitted whether they were accurate or not. This led to the inclusion of the bounty system in the second and third trials. In these latter trials, the problem of flooding was reduced and photos were only confirmed when they represented what the Realise users actually wanted.

The three trials consisted of a total of 46 people (13 female and 33 male): the first trial consisted of 10 EyeSpy players and 5 Realise users, the second trial consisted of 18 users split into two groups of 9 (one group of users who had lived in the game area for more than three years and one group of users for less than three years), and the third trial consisted of 13 players split into two groups of 7 people and 6 people (7 users who had lived in the game area for more than three years and 6 for less than three months). This selection process gives a broad range of users with varying levels of local knowledge. The first trial lasted two weeks and the users maintained their roles for the whole trial. In the second and third trials, the two groups each took on the role of EyeSpy player or Realise user for half the trial (11 days), then the two groups swapped for the second half. The second and third trials lasted three weeks each. The EyeSpy game was not seeded with targets in any of the trials, but the Realise website was seeded with all the data that had been gathered from the previous trials, as well as the data gathered from earlier EyeSpy trials [1]. In each trial, the participants got paid £10 a week with winners of the game being paid double. In the first trial, the participants came from a mixture of professions, but most were University students studying a mixture of subjects. In the second and third trials, the participants were all University students with most coming from Computing Science with only four studying non-computing

subjects. In total, there were five groups of people in the trials who knew each other well: three groups of two users, and two groups of three users. This means that 34 of the trial participants (74%) did not know the other participants.

The trials produced 427 images, 290 text labels and 212 requests. 123 of the images (29%) were produced as the result of requests made to EyeSpy from the Realise website. Out of a total of 717 images and text labels, 60% were images and 40% were text. On average, users produced 16 targets during the game: 9 images and 7 text labels. However, on average, about a third of those images were produced to fulfill the direct needs of a Realise website user. This strong level of directed effort is a major benefit of mutually reinforcing systems as it ensures that the by-products produced are those which are needed.

5. DISCUSSION

While the Realise website is currently limited in ways for the user to interact, it shows the potential in creating a photograph website where users can request new photographs. The website could be used to gather local information about a place before you visit it (such as asking for a picture of a hotel so that you know it when you see it), or to get a photograph of a place you visited before when you forgot to bring your camera. This idea of pre-visiting and post-visiting locations is an existing concept in e-tourism [2].

Adding EyeSpy to a mutually reinforcing system loop has resolved some of the problems that the game suffered from: the by-products produced are now specifically requested by the system that uses them (so they are closer to what is required and less broad), the game is less monotonous because of the randomness and challenge of the new priority targets (these are like bonus targets because the players cannot guarantee that they will always have them during play), and players feel more competitive because the priority targets are worth a variable number of points depending on what the Realise website user assigned to the request. This means that the game encourages finer grained results where required and is now more enjoyable to play because it is less monotonous. By working together in a mutually reinforcing system loop, Realise can now request specific data from EyeSpy, and EyeSpy becomes more diverse and enjoyable to play.

6. ACKNOWLEDGMENTS

This research was funded by UK EPSRC (GR/N15986/01, EP/E04848X/1, EP/F035586/1).

7. REFERENCES

- [1] M. Bell, S. Reeves, B. Brown, S. Sherwood, D. MacMillan, J. Ferguson, and M. Chalmers. Eyespy: Supporting navigation through play. In *CHI 2009*, April 2009.
- [2] B. Brown and M. Chalmers. Tourism and mobile technology. In *ECSCW 2003: Proceedings of the eighth european conference on computer supported cooperative work*, pages 335–355, Helsinki, Finland, 2003. Dordrecht: Kluwer Academic Press.
- [3] L. von Ahn and L. Dabbish. Labeling images with a computer game. In *CHI '04: Proceedings of the SIGCHI conference on Human factors in computing systems*, pages 319–326, New York, NY, USA, 2004. ACM Press.

A Note on Human Computation Limits

Paul Rohwer
Indiana University
School of Informatics and Computing
150 S. Woodlawn Ave.
Bloomington, IN 47405
prohwer@indiana.edu

ABSTRACT

In 2007, Penguin Books and De Montfort University used a wiki to crowd source a novel. The result was deemed a failure because of the many un-integrated elements; however, in the last year a model utilizing Twitter has achieved success: BBC Audiobooks America has produced two audio books from an iterative progression, singular integration model. With an initial seed Twitter message, a real-time editor chooses the progression from submitted messages—achieving contextual integration—and continues the iterative process until the editor chooses to conclude. The model achieves success by controlling variability and progression, demonstrating that human computation can achieve limited success in un-collaborative literary domains.

Categories and Subject Descriptors

H.5.3 [Group and Organization Interfaces]: Computer-supported cooperative work; H.5.m [Information interfaces and presentation (e.g. HCI)]: Miscellaneous; I.2.6 [Learning]: Knowledge acquisition;

General Terms

Design, Human Factors, Languages, Theory

Keywords

Human Computation

1. INTRODUCTION

“As a permanently ex-contributor to this project, I just want to express total disappointment and lack of confidence in it. It’s just a big wall of virtual graffiti... I wish this thing would just go away now, instead of encouraging idiocy.” - Contributor to “A Million Penguins” project. [1]

Human computation accelerates by division of labor; however, time and context limit unit labor power. Even with task differentiation, there are particular problems ill-suited for computers and ill-suited for human computation in the crowd. In general, these are problems where human creativity is focused for

a long period to create an intricate artifact for human consumption. For example, there is a wild rumor that with enough time, a room full of monkeys could produce Shakespeare. Wikipedia catalogs this wild rumor as a theorem--the infinite monkey theorem. True to science, a media arts lab was awarded a grant, experimented and discovered none of the experimental subjects, six Celebes Crested Macaques of the Paignton Zoo, contained either the mind of Shakespeare or, the experimenter's conclude, the random generative powers necessary to produce Shakespeare. [2]

Is the literary novel too much for human computation? A human computation model of iterative progression and singular integration points a way forward.

To read a book, we assume the task of reading a sequence of words organized in a hierarchy: the chapters, paragraphs and sentences. Without the sequence and structure, the words alone do not have context. [3] It is a singular exercise to read a book, and so too has been the creative production of the award-winning novel. It is a different exercise than solving for the best chess move, labeling all the Martian craters, or verifying the letters within an image.

And yet, the creation of the novel has been crowd-sourced. “A Million Penguins” project, launched by Penguin Books and De Montfort University in February 2007, seeded the production of a novel with one sentence, and then discovered the tragedy of the commons [4]:

“The existence of so many un-integrated elements in ‘A Million Penguins’ may well be why the wiki novel is often claimed to fail as literature.” - A Million Penguins Report [5]

Five months, and more than 11,000 edits later, the project leaders concluded the project did not produce a novel. The lack of collaboration did not produce the necessary integration and structure. In the project leader’s analysis of the public commentary about the project, they note the fascination and immense effort the “A Million Penguins” project generated. And the conclusion by critic Ben Vershbow, that “a novel is probably as un-collaborative a literary form as you can get, while a wiki is inherently collaborative... Here that tool is misapplied.” [5]

Smart devices suggest words as letters are typed, leaving the decision to the smart device owner to accept or to decline. Can a similar technique be used to continue the plot? And if the novel literary form is so difficult for the wiki, and difficult for computers, how close can human computation come to literary production?

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

KDD-HCOMP’10, July 25, 2010, Washington, DC, USA.
Copyright 2010 ACM 978-1-4503-0222-7 ...\$10.00

2. TWITTER

In the last year, BBC Audiobooks America has found a successful model to produce a literary story from crowd-sourced text elements: iterative progression, and singular integration. The story is created tweet by tweet—140 characters at a time—with the successive tweet chosen by the integrator, a real-time editor at BBC Audiobooks America. At a time of the integrator's choosing, the compiled tweets are transformed into an audiobook manuscript by the featured author. BBC Audiobooks America has completed this process twice: first with *Hearts, Keys, and Puppetry* by Neil Germain and the *Twitterverse*; Narrated by Katherine Kellgrin and the forthcoming *Fashionably Undead*, by Meg Cabot and the *Twitterverse*; Narrated by Sarah Drew. [6]

The first part of the model, the iterative progression portion of BBC Audiobook America's model, is not new to Human Computation, as several noted the importance during the 2009 Human Computation workshop.[7] In addition, MIT's Amazon Mechanical Turk toolkit, TurKit, experimentally explores iterative tasks.[8]

The second feature of BBC Audiobook America's usage of Twitter is singular integration. That is, a single editor providing coherence at each progressive step. [9] As noted by Von Ahn, human computation requires verification. [10] And a plethora of voting mechanisms exist.[11] MIT's TurKit toolkit uses an iterative voting mechanism to achieve sufficient votes and straightforward majority voting has been used to collaboratively choose the next chess move in *Kasparov vs. The World*. [12] However, as critics of *The Million Penguin* project noted, the novel literary form does not lend itself to collaboration. The single real-time editor may be the natural requirement to achieve a sufficiently coherent narrative.

3. GROUP MECHANISM MODEL

In two letters-to-numbers matching problem experiments, small groups of individuals out-performed the best individuals, [13] and was further confirmed by experiments that small groups of 3 or more participants out-performed 2-person groups. [14] The result led Professor Robert Cialdini and his co-authors to suggest that the cause was the diversity of opinion and the power of parallel processing. And that rather than use a vote-counting strategy for the selection decision, “the final choice is always for the leader to make.” [15] So the Iterative Progression, Singular Integration model is not un-discovered—group decision researchers have found success in small problem-solving exercises—yet, the promising implementation from BBC Audiobooks America does show a practical real-world application.

3.1 Iterative Progression

For both BBC Audiobooks America and “The Million Penguins” project, the initial seed was the first sentence of the story. By limiting the next contribution to 140 characters, creators have a better hold on context, at least initially in the BBC Audiobooks America project. For in the limit, each selected addition requires greater contextual awareness by the collaborators. And, one can reason, the marginal rate of successful new contributors decreases over time because the required contextual knowledge imposes a growing burden from prior contributions.

Thus, the successful model works like a sieve. Those who start initially and continue to invest are more likely to remain successful contributors, while a greater burden is placed on late-arriving new participants and those who intermittently participate, implicitly filtering the iterative progression.

In contrast, the wiki model of the “The Million Penguins” project allowed contributions, edits, and deletions at any time. And “The Million Penguins” project gained contributors as publicity about the project attracted new interest, yet a fraction ultimately contributed more than once. [5] The initial contributors for the BBC Audiobooks America project were cleverly encouraged to participate from the fan base of an established author. On her blog, Meg Cabot appealed to her fans, not just to join, but appealed to a shared effort: “Now remember: This is the big time, baby, so make it good.” [9]

3.2 Singular Integration

A singular source of repeated decisions is the requirement for written context because, as the wiki model demonstrated, without such control the contributed fragments are unconnected to the whole. At each stage, the singular decision maker for BBC Audiobooks America decided the progression. The iterative progression is similar to a chess game where turn-by-turn the game progresses from a common blank slate.

However, the iterative progression of a chess game is different. The move choices at each step are fixed and relatively few in comparison to the open-ended Twitter messages submitted to the singular real-time editor of BBC Audiobooks America. This limited move choice allowed open majority voting to be successfully used in *Kasparov vs. The World*. [12] And even in the *Kasparov vs. The World* one can argue the expert chess panel providing recommendations throughout the match served as a singular integrator by significantly influencing each move choice because only once during the middle game did the expert panel differ from the voted move choice.

Without any control, a voting strategy for selection decisions leads to greater variability because of the inherent randomness in the sample size and voter composition. So while variability is the creative spark for the collaborated narrative, variability must be balanced with cohesive decision makers into a singular integration. In a naïve writer model, a division into a stream of consciousness and a prose editor, the purpose of singular integration is to integrate the fragments of controlled variability from the stream into the construction of cohesive prose.

4. LIMITATIONS

The product produced by “The Million Penguins” projects was widely deemed a failure. [5] And the BBC Audiobooks America project can be considered a success in comparison. However, BBC Audiobooks America's finished product did go through a final edit by the well-established author who partnered and publicized the project and the final text was then narrated by an actor. The audio narration may hide written flaws. And the subjective nature of literary work presents a limitation for wider study.

However, the human computation model can be compared in a limited fashion. With the Wiki model versus the Iterative Progression, Singular Integration model, a comparison can be

made of the interaction mechanisms in use, the human computation composition, and the control imposed by the model to achieve the output type. By comparing models, we can conclude the Wiki model is a poor choice for literary production because it lacks sufficient control.

5. CONCLUSION

BBC Audiobooks America has implemented a successful model for human computation production of a literary work. After a noted failure of a Wiki-model to produce literary work by “The Million Penguins” project, the human computational limits have been expanded in an iterative progression, singular integration model. The audiobook, *Hearts, Keys, and Puppetry* by Neil Germain and the Twitterverse; Narrated by Katherine Kellgrin is a noteworthy achievement for human computation.

6. ACKNOWLEDGMENTS

My thanks to Indiana University’s School of Informatics and Computing for continued research support and my thanks to the reviewers for valuable comments and suggestions.

7. REFERENCES

- [1] “Infinite Monkey Theorem.” *Wikipedia, The Free Encyclopedia*. Wikimedia Foundation, Inc. 22 July 2004. Web. May 1, 2010.
http://en.wikipedia.org/wiki/Infinite_monkey_theorem
- [2] “A Million Penguins” Project Blog Comment. 18 Feb 2007.
<http://amillionpenguins.com/blog/?p=22>
- [3] Adler, M. J. and Van Doren, D. 2007. *How to Read a Book*. Simon & Schuster, Inc. New York, NY.
- [4] Hardin, G. 1968. The Tragedy of the Commons. In *Science*, Vol. 162, No. 3859, pp. 1243-1248.
- [5] Mason B. and Thomas, S. (2008) *A Million Penguins Research Report*. Institute of Creative Technologies, De Montfort University, Leicester.
- [6] BBC Audiobooks America
<http://www.bbcaudiobooksamerica.com/Trade/TradeHome/tabid/60/Default.aspx>
- [7] P.G. Ipeirotis, R. Chandrasekar, P.N. Bennett, E. Law, M. Chickering, A. Mityagin, F. Provost, L. von Ahn (2009). A report on the Human Computation Workshop (HComp 2009). To appear in *KDD Explorations*. 2009.
- [8] Little, G. Chilton, L. B., Miller, R. C., and Goldman, M. 2009 *TurKit: Tools for Iterative Tasks on Mechanical Turk*. Not yet published.
- [9] Meg Cabot Blog. <http://www.megcabot.com/2010/02/>
- [10] von Ahn, L. *Human Computation*. Ph.D. Dissertation. Computer Science Department, Carnegie Mellon University, Pittsburgh, PA, December 2005.
- [11] Conitzer, V. *Computational Aspects of Preference Aggregation*. Ph.D. Dissertation. Computer Science Department, Carnegie Mellon University, Pittsburgh, PA, July 2006.
- [12] “Kasparov versus the World.” *The Free Encyclopedia*. Wikimedia Foundation, Inc. 22 July 2004. Web. May 1, 2010.
http://en.wikipedia.org/wiki/Kasparov_versus_the_World
- [13] Laughlin, P. R., Bonner, B. L. and Miner, A. G. Groups perform better than the best individuals on Letters-to-Numbers problems. In *Organizational Behavior and Human Decision Processes* (2002), Vol. 88, 605-620.
- [14] Laughlin, P. R., Hatch, E. C. Silver, J. S., and Boh, L. 2006. Groups Perform Better Than the Best Individuals on Letters-to-Numbers Problems: Effects of Group Size. In *Journal of Personality and Social Psychology* (2006), Vol. 90, No. 4, 644-651
- [15] Goldstein, N. J., Martin, S. J. and Cialdini, R. B. 2008. *Yes!: 50 Scientifically Proven Ways to Be Persuasive*. Simon & Schuster, Inc. New York, NY.

Reconstructing the World in 3D: Bringing Games with a Purpose Outdoors

[Extended Abstract] *

Kathleen Tuite¹, Noah Snaveley², Dun-Yu Hsiao¹, Adam M. Smith³, and Zoran Popović¹

¹Center for Game Science
Department of Computer Science
& Engineering
University of Washington
{ktuite,dyhsiao,zoran}@cs.washington.edu

²Department of Computer
Science
Cornell University
{snaveley}@cs.cornell.edu

³Department of Computer
Science
UC Santa Cruz
{amsmith}@cs.ucsc.edu

ABSTRACT

We are interested in reconstructing real world locations as detailed 3D models, but to achieve this goal, we require a large quantity of photographic data. We designed a game to employ the efforts and digital cameras of everyday people to not only collect this data, but to do so in a fun and effective way. The result is PhotoCity, a game played outdoors with a camera, in which players take photos to capture flags and take over virtual models of real buildings. The game falls into the genres of both games with a purpose (GWAPs) and alternate reality games (ARGs). Each type of game comes with its own inherent challenges, but as a hybrid of both, PhotoCity presented us with a unique combination of obstacles. This paper describes the design decisions made to address these obstacles, and seeks to answer the question: Can games be used to achieve massive data-acquisition tasks when played in the real world, away from standard game consoles? We conclude with a report on player experiences and showcase some 3D reconstructions built by players during gameplay.

Categories and Subject Descriptors

K.8.0 [Personal Computing]: General – Games; I.4.8 [Image Processing and Computer Vision]: Scene Analysis – Shape

Keywords

3D reconstruction, alternate reality, ARG, computer vision, games with a purpose, GWAP, game, photography, virtual world

1. INTRODUCTION

*A full version of this paper is available as *Reconstructing the World in 3D: Bringing Games with a Purpose Outdoors* at *FDG 2010*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

KDD-HCOMP'10, July 25, 2010, Washington, DC, USA. Copyright 2010 ACM 978-1-4503-0222-7... \$10.00

In the past few years, there has been increasing interest in building 3D models of entire cities for navigation, visualization, and planning applications. Recently, computer vision algorithms have been developed for completely automatic 3D reconstruction of buildings and landmarks from large photo collections [1], including images scraped from photo sharing sites such as Flickr. However, the resulting models are incomplete, as only “popular” viewpoints of popular landmarks are well-represented on Flickr. Reconstructing complete, high-resolution models of contiguous neighborhoods and whole cities requires many more photos, distributed more uniformly, than currently exist on such sites.

In PhotoCity, a hybrid alternate reality game with a purpose, we exploit the mobility of everyday players to collect carefully composed photographs of locations in an area of interest. Player photos include not just the standard, front-facing views of a location, but photos from a wide variety of viewpoints, allowing us to build a complete reconstruction. Bridging active collection of imagery with the human desire to playfully compete has required the design of a new gameplay form and forced the development of new computer vision technologies that can interact with players and usefully direct their effort.

We favor models reconstructed from photos because of the accuracy and realism such techniques provide. These techniques, borrowed from computer vision, are also cheaper and more automatic than hiring a 3D modeling artist.

Digital reconstructions are sensitive to the camera placement of the original images and resulting models often contain gaps and holes where additional views are needed. The data from a single pass with a vehicle-mounted camera array is often not dense enough to admit the quality of reconstructions we seek and is limited to where the vehicle can drive.

In PhotoCity, we have approached the problem of collecting useful data by actively involving players in an iterative growing and refining process. To play, players go to locations in the real world that correspond to locations under construction in the virtual world and take photos. We direct players to focus their attention on the gaps and fringes in a partial reconstruction in exchange for in-game rewards.

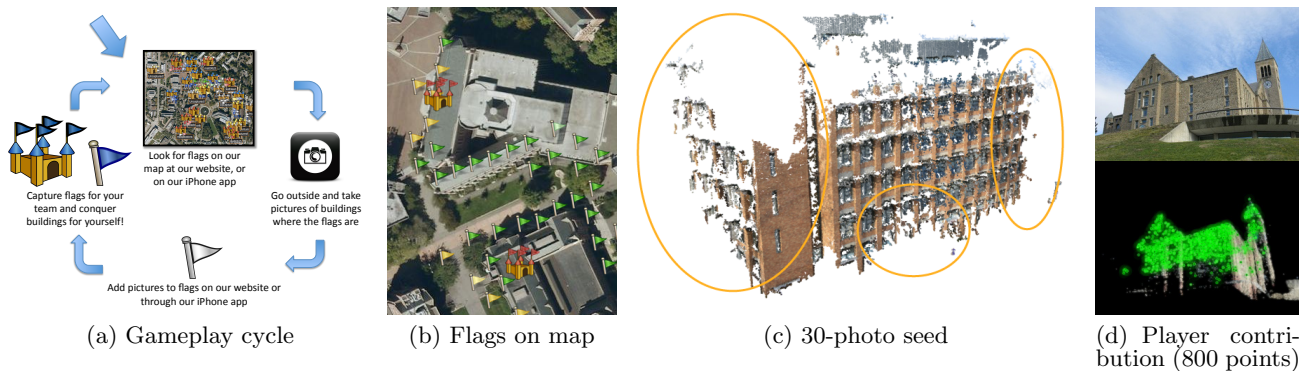


Figure 1: PhotoCity aspects of play

Unique to PhotoCity are the combined challenges of designing an effective game with a purpose (GWAP) and an alternate reality game (ARG), using the real world as its gaming platform. This pairing requires both getting skilled players out in the world and communicating game rules and feedback to them.

2. GAME MECHANICS.

With the purpose of our game being to collect the kinds of images that would yield a high-quality reconstruction, we were faced with the challenge of translating this task into an inviting gameplay experience. In our players' eyes, PhotoCity is about capturing virtual flags anchored in real-world locations and vying for nominal ownership of familiar landmarks.

The core mechanic of our game involves players inspecting the state of the game world on a map, taking photos at locations of promising in-game value, and uploading the photos, and then observing the results of their play. Through repeated cycles of this process, summarized in Figure 1(a), players introduce new geometric points in the reconstructions that correspond to points of detail in the real world.

Accumulating points lets players control flags and own models in the game's fictional world. We will briefly go through each element of the game: where the models come from, where the flags come from, how exactly a player receives points, and the requirements for capturing flags and taking ownership of models.

A model is a partial building that starts off as a seed generated from a batch of photos of the real building. A seed can be generated from about 50 photos. As a result, models in their starting state only span one face or one corner of a building and have rough edges and large holes where data for the building has yet to be captured. Figure 1(c) shows the size of a seed made from thirty photos. The seeds are added either by the game designers or the players themselves, but go through a manual approval process that rejects offensive material and low quality reconstructions.

To anchor the model to the world, we manually align an overhead view of the model with a satellite map. Flags are automatically placed on the model so that each flag has a map location corresponding to a position on a wall, or

sometimes a tree or cluster of non-building points. To take a photo of a flag, one must face the building the flag is on and stand far enough away to get distinctive features of the building in the shot. As the models grow, new flags automatically appear at the edges of the models for players to capture. If the flags at a model's edge are already captured, a player can work to grow the model enough to spawn a new flag.

The typical player looks at the map of the game, identifies the flags she wishes to capture, and then takes photos of that portion of the building. For a photo to earn points, it must (1) overlap with enough existing points in the model and (2) overlap the empty space next to a model. The first requirement allows the photo to connect with the model and have its position within the model automatically calculated. The second requirement, that the photo look beyond the existing model, allows new points to be added to that void as soon as there are enough other photos to triangulate the 3D positions of those features. A photo can add up to several thousand new points if it fills in a large patch of the model with new features.

To earn points at a flag, a player simply generates new 3D points in a column around a flag with a radius of about thirty feet. 3D points translate directly into game points. The team with the majority of points at a flag captures that flag. Flags often accumulate tens of thousands of points, so flags that one team owns but that have fewer than 2,000 points total are considered "disputed" because they are relatively easy for another team to steal.

While teams control flags, individual players control individual models. If one player has over 10,000 points at a model (roughly equivalent to making a major contribution at five different flags) and has more points than any other player, that player owns that model.

The photography must happen outdoors, in the vicinity of certain buildings, but the game can be played in two different ways. Players can bring their cameras inside and use a high-bandwidth link to upload photos to the website, or players can play using a custom iPhone application that lets them submit photos and get feedback in one seamless process. While high-end digital cameras yield better results (and more points) than cell phone cameras, anyone can play and contribute with the camera he or she already has.

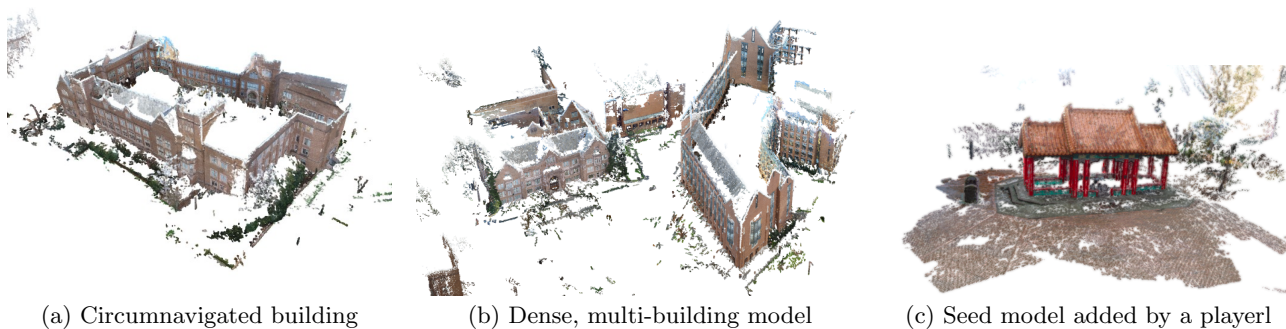


Figure 2: Reconstructions

These mechanics result in a virtual world, populated by 3D reconstructions, which grows organically through continued player interaction.

3. RESULTS

To evaluate how well we overcame these design challenges, we ran two game trials, one in May 2009 that lasted for one week, and one beginning in December 2009 that lasted for two months. Both trials took place on the University of Washington campus using the same set of building seeds, which were reset to their initial states before each trial. We report player statistics for both trials and present the reconstruction results from the second trial.

3.1 Reconstructions

We started with 12 seed models at the beginning of the second trial, listed in Table 1. The 13th model was added during the game by one of the players. The first two columns show the number of photos and points in the initial seed models and the last two columns show the growth of each model.

Three very exciting things happened with the reconstructions that demonstrated the success of the game. First, as shown in Figure 2(a), players circumnavigated several buildings, despite obstacles like trees and hedges at the corners of buildings that were hard to reconstruct. Second, players learned how to expand models onto other physically disconnected buildings by taking photos of one building with the second building in the background. Players actually expanded seeds to touch and overlap nearby seeds. Figure 2(b) shows a seed that started as just the corner of the angular building expanding to include several adjacent buildings. Third, players went out and photographed buildings of persona importances and submitted them as seeds. One example of a player seed is shown in Figure 2(c).

Towards our goal of harnessing player effort to build 3D reconstructions by taking photos, the 25 players in our second trial submitted over 11,000 new photos generating over 3,000,000 new sparse-model points. Since the publication of the full paper [2], over 130,000 photos have been collected, most of which actually provide useful information for the 3D reconstructions.

3.2 Reactions to Game Elements

Did the PhotoCity players have fun? We interviewed several of them and also observed their interactions with other players.

New players who managed to score points in their first set of photos felt good about themselves, even if not all of their photos were successfully added to a model. The positive effect of a matching photo was greater than the negative effect of a failed photo.

Players who started the game when there were a large number of white (uncaptured) flags were very excited to see the flags change to their team color. However, players who came to the game when many photos had already been taken and most of the flags captured were overwhelmed and felt less able to make a difference (a common occurrence in multi-player online games).

Despite the game being driven by the mere acquisition of points, flags, and buildings, players discovered “expert strategies” that both made them more interested in the game, and allowed them to acquire vast quantities of points. For example, when one player had built a sufficiently dense model (so dense that new photos would match but not add new points), a second player came up with a strategy to reclaim the flags by expanding nearby models into the flag-space already inhabited by the first model. This interesting emergence was a direct result of attaching abstract mechanics to locations in the physical world.

4. CONCLUSION

We have described PhotoCity, a game with a purpose that draws its players outdoors to participate in the construction of a virtual world. In creating a game at the intersection of two genres, alternate reality games and games with a purpose, we encountered challenges on a new scale. PhotoCity is both vastly more data intensive than other GWAPs and more process intensive than traditional ARGs, even requiring the development of new algorithms and deployment of them in a distributed system to maintain interactivity. Initial trials of our game have already resulted in a collection of highly-detailed 3D models, demonstrating that hybrid ARG-GWAPs, played out in the real world, can be effective at collecting vast quantities of data, provided the games are interesting, fun, and equip players with the right tools and training to accomplish the task at hand.

Model	Starting Photos	Starting Points	Ending Photos	Ending Points
CSE Front Entrance	58	18,131	515	148,189
Electrical Engineering	30	13,133	1,230	380,102
James J. Hill Statue	48	51,461	406	224,783
Southeast Corner of CSE	27	9,597	935	304,030
Fountain-facing Corner of EE	68	38,439	979	321,805
Commodore Apartments	41	9,973	581	66,777
Allen Library	92	28,945	840	131,558
Engineering Library	74	8,598	584	62,095
Guggenheim	101	85,239	1,394	622,355
Mechanical Engineering	46	27,022	741	273,889
Mary Gates (West Side)	32	36,891	1,488	761,777
Suzzallo Library	41	36,566	1,526	492,871
Hing Hay Park*	73	46,965	116	54,333

*Seed added by player during trial

Table 1: Model Growth

By sharing the design of our game, we hope to stimulate further discussion on games with a purpose that extend into the real world. A great many computational problems can be solved in the presence of masses of data, so that cheaply and effectively collecting this data quickly becomes the primary issue. The reader is invited to visit the game’s reconstruction gallery¹ and join our ongoing effort to reconstruct the world in 3D by submitting his or her own images.

5. ACKNOWLEDGMENTS

Many thanks to the efforts of Sylvia Tashev, Nadine Tabing and Juliet Bernstein, and to our star players, Erik, Daniel, and Yuki. We also thank Intel, Google, and Microsoft for their support.

6. REFERENCES

- [1] N. Snavely, S. M. Seitz, and R. Szeliski. Photo tourism: Exploring photo collections in 3d. In *SIGGRAPH Conference Proceedings*, pages 835–846, New York, NY, USA, 2006. ACM Press.
- [2] K. Tuite, N. Snavely, D. Hsiao, A. M. Smith, and Z. Popovic. Reconstructing the world in 3d: Bringing games with a purpose outdoors. In *FDG ’09: Proceedings of the 5th International Conference on Foundations of Digital Games*, New York, NY, USA, 2010. ACM Press.

¹<http://photocitygame.com/reconstructions.php>

Improving Music Emotion Labeling Using Human Computation

Brandon G. Morton, Jacquelin A. Speck, Erik M. Schmidt, and Youngmoo. E. Kim
Department of Electrical and Computer Engineering, Drexel University, Philadelphia, PA 19104
{bmorton, jspeck, eschmidt, ykim}@drexel.edu

ABSTRACT

In the field of Music Information Retrieval, there are many tasks that are not only difficult for machines to solve, but that also lack well-defined answers. In pursuing the automatic recognition of emotions within music, this lack of objectivity makes it difficult to train systems that rely on quantified labels for supervised machine learning. In recent years, researchers have begun to harness Human Computation for the collection of data spanning an excerpt of music. *MoodSwings* records dynamic (per-second) labels of players' mood ratings of music, in keeping with the unique time-varying nature of musical mood. Players collaborate to build consensus, ensuring the quality of data collected. We present an analysis of *MoodSwings* labels collected to date and propose several modifications for improving both the quality of the gameplay and the collected data as development moves forward.

Categories and Subject Descriptors

E.0 [Data]: General; H.1.2 [Information Systems]: Models and Principles

General Terms

Human Factors, Measurement

Keywords

human computation, music, mood prediction, moodswings, mood labels, emotion, valence, arousal

1. INTRODUCTION

The problem of automated mood detection of music is particularly challenging in that it attempts to quantify highly subjective human perceptions, requiring collection of ground truth data directly from humans. In recent years, Music Information Retrieval (Music-IR) researchers have attempted to harness human computation to collect ground truth data

more efficiently by developing games similar to those designed for other purposes, such as computer vision [1]. Music labeling games, however, involve unique challenges due to the time-varying nature of audio.

In this paper we present an analysis of *MoodSwings*¹, our game designed to collect “ground truth” data labeling music by mood [2]. The game is targeted at collecting dynamic (per-second) labels of users' mood ratings in real-time using a two-dimensional space of emotional components. We use the Arousal-Valence (A-V) space, a well-validated representation of mood from psychology research proposed by Thayer and Russell [3]. As in other collaborative games, players are partnered in order to verify each others' results, providing a strong incentive for producing high-quality labels. Accordingly, scoring is designed to maximize consensus-building between partners.

We propose several improvements to *MoodSwings* suggested by an analysis of data collected thus far. These improvements could both produce more informative ground truth data and increase the amount of data collected by encouraging more people to play *MoodSwings*.

2. BACKGROUND

The success of data collection games like the *ESP Game* for image labeling has inspired Music-IR researchers to develop games for labeling audio. Note that the games described below are designed for collecting static labels, such as musical genre or artist, and do not account for time-varying labels. When working with audio data, which is by nature time-varying, this consideration can be crucial.

One of the earliest games developed for music label collection was *MajorMiner* [4]. While it does not emphasize collaboration as much as its successors, *MajorMiner* rewards consensus on the “tags” (i.e., labels) it collects by awarding points based on how often the tags players generate are used by others. Agreement between players can occur asynchronously, as players continue to earn points between games if others use their tags. An added benefit is preventing what is commonly referred to as the “cold start” problem, resulting from both an initial lack of data and lack of players to produce data. New song clips can be introduced systematically, reducing the number of clips in the corpus from which no useful inferences can be drawn due to lack of data.

Listen Game expands on *MajorMiner*'s model of entirely free-form label assignment, including both a free-form “freestyle” mode and a “normal mode” in which players are

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. *KDD-HCOMP'10*, July 25, 2010, Washington, DC, USA. Copyright 2010 ACM 978-1-4503-0222-7 ... \$10.00

¹*MoodSwings*: <http://schubert.ece.drexel.edu/moodswings>

presented a predetermined tag vocabulary [5]. Players mark randomly chosen labels from this vocabulary as either “good” or “bad”, simplifying analysis by removing the need to consider redundant or nonsensical labels assigned by players. *TagATune* collects free-form tags, but employs a mechanism to discourage players from assigning nonsense labels. Partnered players describe the songs they hear to each other, then determine whether or not they are listening to the same song [6].

Herd It, the most recently developed Music-IR ground truth collection game, emphasizes agreements within a large group rather than between partnered pairs of players [7]. It consists of several mini-games in which groups of players select one of several labels provided for each song clip. Mini-games are designed to collect ground truth for a variety of Music-IR tasks, including genre and mood classification (e.g.: “which of the following colors best fits this song clip?”) *Herd It* is also unique in that it runs on the popular Facebook social networking website, which attracts user traffic. Facebook also facilitates the collection of statistics describing demographics of the *Herd It* user base, adding additional context to data collected based on agreements between players.

3. MOODSWINGS

MoodSwings is a collaborative game incorporating two players’ judgments of the moods of songs into gameplay [2]. The fundamental difference between *MoodSwings* and other Music-IR games is its collection of time-varying labels, which is particularly significant in the realm of audio because the signals themselves are time-varying. Players are partnered anonymously over the internet, with the goal of dynamically and continuously reaching agreement on the mood of five 30-second song clips drawn from a music database. The game board, shown in Figure 1, is a representation of the A-V space. It presents a continuum of mood ratings: valence, reflecting positive vs. negative emotion, is displayed on the vertical axis and arousal, reflecting emotional intensity, is on the horizontal axis.

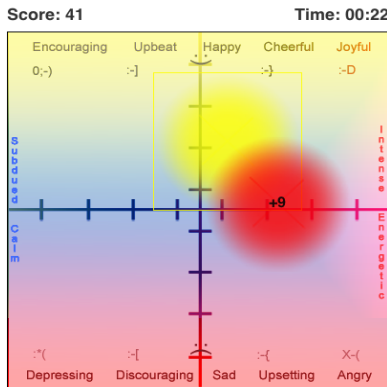


Figure 1: Screenshot of MoodSwings gameplay. The red and yellow orbs represent each player.

3.1 Gameplay and Scoring

A *MoodSwings* game consists of 5 rounds, each with a different 30-second song clip. Partners are paired at the beginning of each game for its entire duration. Each player

positions his or her cursor on the game board to indicate judgment of the mood of the song at each instant during the game. Scoring is based on the amount of congruency between the two cursor positions, with greater overlap resulting in more points earned. Partners’ cursors are visible only once every 3 seconds. However, cursor size decreases over time, increasing the difficulty of overlapping cursor positions. Players can also accumulate bonus points by “convincing” partners to agree with their mood assessments, providing incentive against capricious movements. If a player keeps his or her cursor position stationary for more than 1 second and their partner moves to the same position, 5 bonus points are awarded, indicated by a change in cursor color. The point system is designed so that a “good” score for a single round is about 100 points, or about 500 points for a game. High scores for the top five players and the top individual match scores are visible when a player first logs onto the *MoodSwings* website.

3.2 Music Corpus and Data Collected

MoodSwings draws from 7,135 songs from the *uspop2002* database, developed by the the NEC Research Institute and the Laboratory for the Recognition and Organization of Speech and Audio (LabROSA) [8]. The database contains songs from 400 artists representative of best-selling music in the year 2002.

To date, a total of 5,177 *MoodSwings* games have been played. Over 150,000 mood labels have been collected for 1,158 songs, producing about 16% coverage of all available songs.

3.3 Application of Data

Some of our recent work uses MoodSwings data to predict a 2-D Gaussian distribution of emotional ratings within the A-V space for each 1-second time slice. We have explored several methods for multi-variate parameter regression as well as a variety of acoustic features. Shown in Figure 2 are the collected A-V labels and distribution projections resulting from regression analysis. Performance of the regression can be evaluated by the amount of overlap between a projection and its ground truth. This work has also been used to improve to the single player version of MoodSwings, which

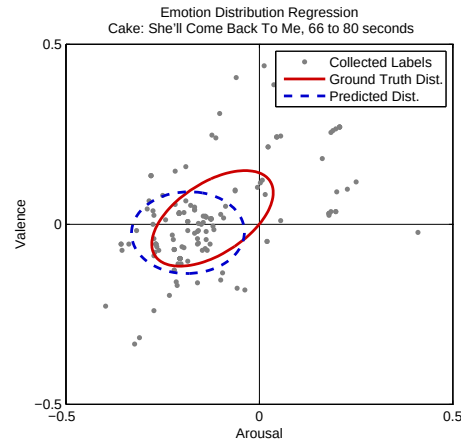


Figure 2: Collected A-V labels and distribution projections from regression analysis.

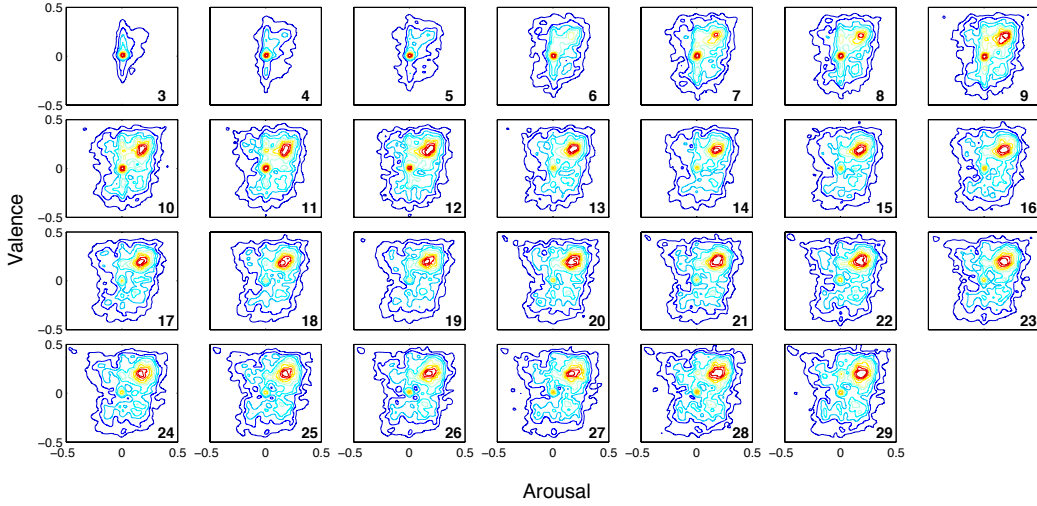


Figure 3: Distribution of MoodSwings labels over length of songs

is described in later sections of this paper.

4. ANALYSIS OF DATA

MoodSwings was designed to be played by two people simultaneously, which required accounting for the possibility of only one player being online at a given time. *MoodSwings* originally paired players with either a past recording of an earlier game or a simulated AI player, which randomly follows a real player’s movements. We hypothesized that enough data collected from human players would negate the effects of simulated players, but overcoming the cold start problem has proven unexpectedly difficult. It is necessary to attract many new players and encourage past users to play again, which we hope to accomplish through improvements to make the game more challenging and exciting. To better understand which aspects of the game should be improved, we examine data collected during each gameplay scenario.

4.1 Analyzing Labels vs. Time

Figure 3 shows the second-by-second distribution of all labels collected from *MoodSwings*. Most movement of players’ cursors occurs at the beginning of a song clip, decreasing over time. We see the distribution becomes less clustered around the origin between about 7-8 seconds. Figure 4 shows the mean distance from the origin of the A-V space to the collected labels, with a square marker indicating 85% of the total distance traveled at about 8 seconds. We infer that

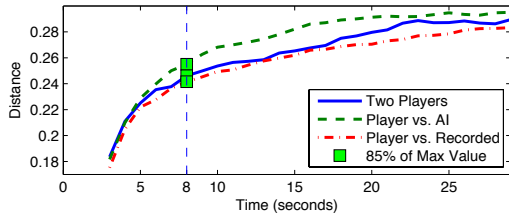


Figure 4: Average normalized distance from origin of the player’s cursor in each gameplay scenario

most players are unable to instantaneously judge the mood of a clip at its onset. This is consistent with other research by Bachorik, which concluded that most music listeners require about 8 seconds to judge the mood of a song [9].

We also see a difference in comparing games with real players vs. the random AI. More distance is traveled in games against the AI, whereas in head-to-head games players tend to compromise, pulling each others’ cursors more towards the origin.

4.2 Analyzing Live vs. Recorded Players

Figure 5 shows the mean distance between the two cursors for two possible gameplay configurations: head-to-head (two players online) and human-vs.-recording (one person plays against a recording of a past game). On average, head-to-head games have a smaller distance between the cursors, which indicates greater agreement. There is a 3-second periodic oscillation, particularly pronounced in the human-vs.-recording case. The amount of agreement here appears to be heavily dependent on whether or not the recorded cursor is visible to the human player (every 3 seconds). In contrast, the head-to-head case shows a less pronounced oscillation, more fluid agreement, and overall greater consensus between live players. All of these characteristics demonstrate agreement being reached faster and in a more consistent manner, further supporting preference for the head-to-head scenario.

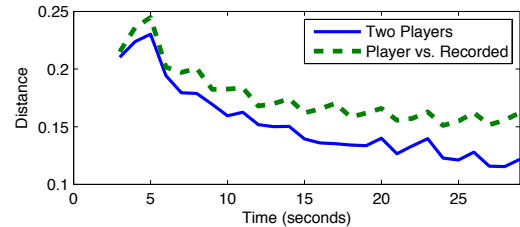


Figure 5: Average normalized distance between opponents’ labels over time in two gameplay scenarios

4.3 Prior Distribution of Pop Music

The second-by-second distributions seen in Figure 3 also reflects a prior distribution of mood labels associated with popular music. Player responses tend to be clustered in the upper-right quadrant. While this prior distribution is not necessarily problematic when only considering one particular musical genre (it can be useful for optimizing classification), collecting ground truth data on pop music alone will not necessarily lead to inferences about the wide range of moods expressible by all musical genres.

5. PLANNED IMPROVEMENTS

We hope to use the presented analysis to supplement the use of recent, ongoing work on emotion prediction to improve the game itself. Our aim is to collect more useful data and produce a more enjoyable gameplay experience.

5.1 Encouraging Replay

The cold start problem is partially caused by the fact that many people play *MoodSwings* once, then never again. We hope to encourage replay by making the game more fun. Fun is difficult to quantify, but we hope that improving the AI and increasing the level of challenge will increase game traffic.

In single player games where players were paired with a simulated player, moves made by the original AI made it obvious to the human player that he or she was not partnered with another person. Recent work to predict time-varying distributions of *MoodSwings* labels has been used to implement a predictive AI in the single player version of the game, *MoodSwings Single Player*². The AI chooses its mood labels for a given song clip from a predicted mood distribution, making it behave more realistically [10]. *MoodSwings Single Player* is designed as a feedback system to address the cold start issue: predicted mood distributions shape moves by the AI, which is used to obtain more data from human players to be fed back into the training of the predictive AI.

We are also testing a version of the game using shorter song clips with no breaks in between, for more fast-paced and exciting gameplay. We must, however, be careful not to shorten song clips to the point where players are unable to determine the mood of the song before the end of the clip (> 8 seconds).

5.2 Expanding the Music Corpus

As described earlier, the music comprising the *MoodSwings* corpus does not include the most current popular music (the most recent tracks are from 2002). We eventually hope to incorporate more current music into the corpus to make the game more entertaining, particularly for younger players.

Other interesting problems related to mood detection could be explored by incorporating other genres besides popular music into the corpus. Pop music is somewhat limited for two reasons: first, the moods of pop songs tend to change seldom or not at all over their durations. Second, pop songs almost always contain lyrics, which influence the perceived mood. We hope to use *MoodSwings* to collect ground truth for instrumental music, including jazz and classical pieces. We hypothesize that mood labels for instrumental music pieces would more evenly span the A-V space.

²*MoodSwings SP*: <http://music.ece.drexel.edu/mssp>

6. DISCUSSION AND FUTURE WORK

We have proposed improvements to address the cold start problem and enhance the gameplay experience. In the future, the game could benefit from updated animations to produce a more modern-looking interface. Displaying players' statistics during gameplay would allow score comparisons, adding a competitive aspect to the game. One possibility for increasing the challenge level is to increase the number of players collaborating during gameplay. The level of challenge is increased due to multiple players striving for consensus. Finally, more players could be attracted to the game if it were available on a wider variety of platforms. Facebook could be an attractive platform for enhancing the social and collaborative aspects of *MoodSwings*. The game could also be successful as a mobile application on handheld devices.

7. ACKNOWLEDGMENTS

This work is supported by National Science Foundation award IIS-0644151.

8. REFERENCES

- [1] L. von Ahn, "Games with a purpose," *IEEE Computer*, vol. 39, no. 6, pp. 92–94, 2006.
- [2] Y. Kim, E. Schmidt, and L. Emelle, "MoodSwings: A Collaborative Game for Music Mood Label Collection," in *Proceedings of the 9th International Conference for Music Information Retrieval Conference*, 2008, pp. 231–236.
- [3] R. E. Thayer, *The Biopsychology of Mood and Arousal*. Oxford, U.K.: Oxford Univ. Press, 1989.
- [4] M. Mandel and D. Ellis, "A Web-Based Game for Collecting Music Metadata," in *Proceedings of the 8th International Conference on Music Information Retrieval, Vienna, Austria*, 2007, pp. 365–366.
- [5] D. Turnbull, R. Liu, L. Barrington, and G. Lanckriet, "A Game-Based Approach for Collecting Semantic Annotations of Music," in *Proceedings of the 8th International Conference on Music Information Retrieval, Vienna, Austria*, 2007, pp. 535–538.
- [6] E. Law and L. von Ahn, "Input-agreement: a new mechanism for collecting data using human computation games," in *Proceedings of the 27th International Conference on Human Factors in Computing Systems*, 2009.
- [7] L. Barrington, D. Turnbull, D. O'Malley, and G. Lanckriet, "User-Centered Design of a Social Game to Tag Music," in *ACM KDD Workshop on Human Computation*, 2009.
- [8] A. Berenzweig, B. Logan, D. P. Ellis, and B. Whitman, "A Large-Scale Evaluation of Acoustic and Subjective Music- Similarity Measures," *Computer Musical Journal*, vol. 28, no. 2, pp. 63–76, 2004.
- [9] J. Bachorik, M. Bangert, P. Loui, K. Larke, J. Berger, R. Rowe, and G. Schlaug, "Emotion in Motion: Investigating the Time-Course of Emotional Judgments of Musical Stimuli," *Music Perception*, vol. 26, no. 4, pp. 355–364, 2009.
- [10] E. Schmidt, D. Turnbull, and Y. Kim, "Feature Selection for Content-Based, Time-Varying Musical Emotion Regression," in *Proc. ACM SIGMM Intl. Conf. on Multimedia Information Retrieval*, 2010.

Webpardy: Harvesting QA by HC

Hidir Aras
University of Bremen
TZI, Digital Media Group
28359 Bremen, Germany
aras@tzi.de

Markus Krause
University of Bremen
TZI, Digital Media Group
28359 Bremen, Germany
phateon@tzi.de

Andreas Haller
University of Bremen
TZI, Digital Media Group
28359 Bremen, Germany
ahaller@tzi.de

Rainer Malaka
University of Bremen
TZI, Digital Media Group
28359 Bremen, Germany
malaka@tzi.de

ABSTRACT

We present a human computation online game for enabling users to contribute to the creation of a corpus of question-resource pairs for harvesting web-based question answering. Our idea was motivated by the popular ‘jeopardy’ quiz.

Categories and Subject Descriptors

H.1.2 [User/Machine Systems]: *Human information processing*;
H.3.3 [Information Search and Retrieval]: *Information filtering*;
Relevance feedback.

General Terms

Algorithms, Design, Human Factors.

Keywords

Human computation, web question answering, natural language.

1. INTRODUCTION

Understanding natural language is still a challenging field for artificial systems. Answering questions given in a natural language or finding relevant search results to these questions are problems tackled by different researchers. Their approaches range from statistic analysis based on term occurrences to natural language processing, knowledge or logic-based techniques that utilize formal ontologies.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

KDD-HCOMP’10, July 25, 2010, Washington, DC, USA. Copyright 2010 ACM 978-1-4503-0222-7 ...\$10.00

Furthermore, the Semantic Web¹ – an initiative and a bundle of technologies for encoding and processing web data in a machine-understandable way – allows us to answer complex question by utilizing knowledge-inference and reasoning over available web data. Unfortunately, users do not contribute on a massive scale in “weaving” the Semantic Web by using knowledge representation languages such as RDF-S², OWL³, etc.

On the other side, lessons learned from the Web 2.0 movement how to incorporate human engagement in to the annotation processes are encouraging. As conceptual modeling and semantic annotation are demanding tasks human intelligence in the process of semantic content creation and meta-data generation is required. So far, missing incentive structures and motivation are the reasons for unsatisfactory user participation. Nevertheless, promising new ideas come from the Human Computation research and approaches inspired by the *Wisdom of Crowds* [9] principle from the Social Networks theory that was successfully adopted in Web 2.0 for enabling user participation for providing meta-data (user tags) for organizing and indexing user generated content (UGC).

In this paper, we present a system that uses a human computation (HC) approach to support artificial systems to provide relevant and high quality responses to natural language questions. Our online game, called *Webpardy*, is a single player online game for creating a corpus of question-resource pairs, which can be used for web-based question answering (QA).

2. RELATED WORK

Natural language processing is an interesting application domain for human computation. A variety of different sub domains were

¹ <http://www.w3.org/2001/sw/>

² <http://www.w3.org/TR/rdf-schema/>

³ <http://www.w3.org/TR/owl-features/>

explored by, e.g. Phrase-Detectives [4] as well as Actionary and TwinMinds [5]. Other approaches are based on paragraph analysis and named entity recognition for extracting appropriate answers for given natural language questions.

A sub-system of SmartWeb⁴ was able to provide answers and images concerning persons, locations and other names entities that are found in web documents, retrieved utilizing a keyword-based search engine such as Google. Cramer et al. [2] describe how to build an evaluation corpus for German question answering by harvesting Wikipedia. Other approaches such as the Alyssa represent statistically-inspired question answering systems. Shen et al. [3] describe their experiments for the TREC 2006 question answering track. Besides this, approaches that are based on ontology-based information extraction [1] have been developed. Here, the extracted data entities are represented as ontological instances and matched against semantic query representations. The latter realize domain-dependent question answering systems, e.g. for soccer resources from the web.

3. A HUMAN COMPUTATION ONLINE GAME FOR HARVESTING QA

The basic principle of Question-Answering – in contrast to document level search – is to provide exact or related answers to user given questions.



Figure 1. A sample web page.

So far, existing systems try to extract their answers from a collection of documents, e.g. web pages applying various techniques ranging from statistical methods, e.g. tf^{-idf} scheme [7], co-occurrence analysis, etc. to natural language processing, e.g. paragraph analysis for matching named entities such as persons, institutions, dates; or analyzing Hearst-patterns that rely on part-of speech, etc. Figure 1 shows an example for extracting answers related to facts from a web page.

⁴ <http://www.smartweb-project.de/>

⁵ term frequency

⁶ inverse document frequency

Questions such as: “Who is the current chancellor of Germany?” could be answered by matching the terms “current”, “chancellor” and “Germany” and looking for named entities of the type person in the proximity of the matches, e.g. whether all three terms are matched, whether the corresponding part-of speech order is correct, etc. For our example question, typical pattern such as <TEMPORAL_WORD>-<NOUN/PROFESSION>-<NOUN/LOCATION>, etc. could be used.

In general, depending of the term matching and paragraph analysis a confidence score is calculated for each answer candidate.

The drawbacks of such systems are that they are not able to answer complex questions. Even facts-related answers can hardly be extracted with high precision from a huge collection of documents utilizing, e.g. search-engines.

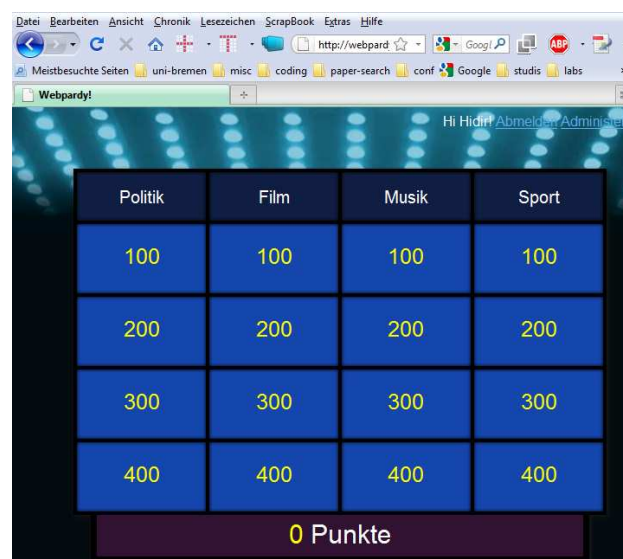


Figure 2. The Webpardy “wall” UI for selecting the category/difficulty level and the respective resource.

This is where the idea to use a human computation online game for generating question-answer pairs arises. Our goal is to enable users to contribute to the generation of questions for existing (web) resources on a large-scale by implicitly labeling them with ‘questions’ that they have to create for getting scores depending of the chosen category.

The idea behind the Webpardy online game is based on the successful 1960s quiz show. The user is asked to select a category as well as a level of difficulty from a matrix as depicted in the left screenshot of Figure 2. The game then chooses a resource, e.g. a web page fragment for the player by utilizing web page segmentation. After that, the player has a limited amount of time to enter a question for this resource as depicted in Figure 3. In order to create the underlying document corpus for a particular domain that corresponds to a category from where the resources are retrieved focused or topic crawlers could be used [11].

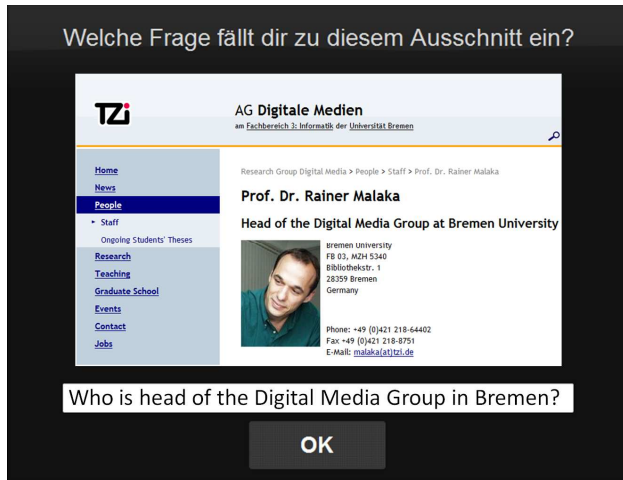


Figure 3. Entering questions for a given web resource.

3.1 Conceptual Framework of Webpardy

Webpardy leverages the wisdom of crowds-principle by using a similar tripartite model as used in collaborative tagging [10], a process of indexing resources of any kind using a set of keywords from an uncontrolled user vocabulary, i.e. a community of users are contributing to the system by collaboratively tagging Web resources, such as pictures, web pages or any other available data. In our case, we use ternary associations represented by edges that connect a given user with a certain resource using a certain question. Equivalent to the social tagging approach, co-occurrence analysis on question terms could be applied for generating good and compact reference questions for validating the quality of provided input.

The basic architecture of our system realizes a two steps approach; firstly, *generating questions*, and secondly, *answering questions*. Figure 4 illustrates the generation of questions.

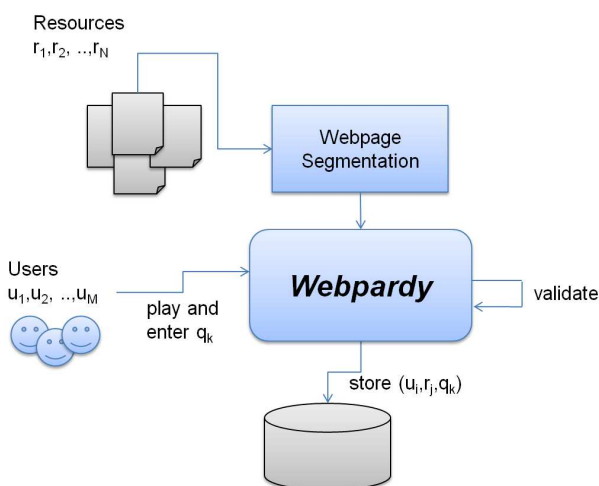


Figure 4. Step 1: Generating questions

Existing web pages are segmented into reasonable fragments that are loaded by the Webpardy game to be presented to the user. Users once registered to the system can start playing by selecting a given category and a level of difficulty, which is chosen randomly for the bootstrapping. We then, estimate difficulty levels depending from implicit user feedback such as, duration for entering the question, length and complexity of the sentence. After clicking the desired cell on the wall a screen with the resource and an input field is presented. Afterwards the user has to think of a suitable question during a countdown, e.g. 10 seconds. The provided sentence is validated according to a multi step procedure before storing the question-resource pair in the database together with its confidence metric as described in the next section.

In the second step, the data store with the set of question-resource pairs can be queried and matched against a given user question. Here, sentence similarity analysis [6] can be applied for gathering previously stored (q, r)-pairs, hence, the desired resources or answers.

3.2 Verification of Questions

The validation of questions in Webpardy follows a two phase approach. In the first phase, a *trust score* is generated for each user. The score is calculated upon a set of existing relevant resource-question pairs. A similarity comparison of player given questions with these ground truth questions allows estimating the quality of the questions collected. The resulting trust values for a particular user indicate whether the provided questions for unknown resources are valuable or not.

In the second phase, where unknown resources need to be annotated with questions, the following procedure is applied for calculating a confidence score:

1. Check whether every entered <word> is really a word in a given language, e.g. using a thesaurus.
2. Check whether the entered sentence contains a question word, e.g. why, who, etc. as well as a question mark.
3. Check the tf-idf value for each term of the sentence. Verify whether the individual terms are relevant for the entire corpus from where the original page was retrieved and in particular, for the web page itself. Additionally, WordNet⁷ could be used to verify the relevance of the provided terms.
4. Match Hearts-pattern [8] for typical question templates.

In addition, it is possible to use a human computation online game for validating entered questions. Several players could play Webpardy in reverse mode, where a list of alternative resources is presented for a particular question. The users then need to select the best fitting resource. The quality of the given question-resource pairs can be evaluated based on the accuracy of provided responses. This mode could be easily integrated into Wepardy as a bonus level.

⁷ <http://wordnet.princeton.edu/>

4. SUMMARY AND FUTURE WORK

We presented a human computation approach for harvesting web-based question answering. Integrating our proposed online game *Webpardy* into a gaming web portal would allow us to ensure sufficient user participation for conducting our proof of concept evaluation. In future work, besides sophisticated algorithms to evaluate the quality of the user-provided questions it is desirable to analyze and develop various types of assistance for entering question sentences to boost the generation of high quality question-resource pairs.

ACKNOWLEDGMENTS

This work was partly funded by the Klaus Tschira Foundation.

REFERENCES

- [1] P. Buitelaar, P. Cimiano, S. Racioppa, M. Siegel. *Ontology-based Information Extraction with SOBA*. In Proc. of LREC, Genoa, Italy, May 2006.
- [2] I. Cramer, J. L. Leidner and D. Klakow (2006): *Building an Evaluation Corpus for German Question Answering by Harvesting Wikipedia*. In: Proc. of the 5th Int. Conf. on Language Resources and Evaluation (LREC 2006).
- [3] D. Shen, J. Leidner, A. Merkel and D. Klakow. 2006. *The Alyssa System at TREC2006: A Statistically-Inspired Question Answering System*. In Proc. of the TREC 2006.
- [4] Chamberlain, J.; Poesio, M. & Kruschwitz, U. *Phrase Detectives: A Web-based Collaborative Annotation Game*. Proc. of the Int. Conf. on Semantic Systems, 2008.
- [5] Takhtamysheva et al. *Games for Games*. HComp'09 Proc. of the ACM SIGKDD Workshop on Human Computation, 2009.
- [6] Higgins, D. and J. Burstein. *Sentence similarity measures for essay coherence*. Proceedings of the 7th International Workshop on Computational Semantics (IWCS), January 2007, Tilburg, The Netherlands.
- [7] R. Baeza-Yates, B. Ribeiro-Neto: *Modern Information Retrieval*, pp 29-30. Addison Wesley / ACM Press, New York 1999.
- [8] Hearst, M.A. *Automatic acquisition of hyponyms from large text corpora*. In Proceedings of COLING, Nantes, France, 1992.
- [9] Surowiecki, James. *The Wisdom of Crowds: Why the Many Are Smarter Than the Few and How Collective Wisdom Shapes Business, Economies, Societies and Nations* Little, Brown, 2004.
- [10] Mika, P. *Ontologies are Us: A unified model of social networks and semantics*. In the Proceedings of the Fourth International Semantic Web Conference (ISWC 2005), Ireland, 2005.
- [11] Bing Liu. *Web Data Mining - Exploring Hyperlinks, Contents and Usage Data*. Springer, December, 2006.

Measuring Utility of Human-Computer Interaction

Michael Toomim and James A. Landay

University of Washington

DUB group, Computer Science & Engineering

{toomim,landay}@cs.washington.edu

ABSTRACT

Human Computation, along with much of the Internet, only works when humans find tasks fun, enjoyable, or valuable enough to outweigh the time and effort they require to complete. The more value, or *utility*, that a task and interface provides, the more “work” humans will do. However, we do not yet know how to objectively measure the fun, enjoyment, or value of a user interface applied to a particular task. This demonstration presents the UTILISCOPE, a system that empirically measures the utility of a user interface for a task by putting multiple versions of a user interface together with a task on Mechanical Turk and measuring the amount of money required to convince humans to use them.

1. INTRODUCTION

The ESP game makes image labeling fun. Wikipedia works by making it easy to edit pages and contribute to a body of knowledge. Facebook and Twitter work by making it easy and rewarding to upload your life online for others to see. However, designing for “fun”, “enjoyability”, or “value” is currently a black art. Fun is subjective. Laboratory studies can detect usability errors, but cannot measure the *utility* users find in the tasks or associated user interfaces, or determine if there is enough utility to outweigh the effort required to complete the task. As such, the only way we know how to validate a social or Human Computation system is to completely build, deploy, and market it to real users and see if it reaches critical mass. This requires a lot of effort and time, and without incremental feedback, is prone to failure.

We present the UTILISCOPE, a system that measures an interface and task’s *interaction utility*: its ability to convince a human to use it. Web designers can use it to prototype and evaluate subtasks in user interface designs, and identify and correct utility bottlenecks, before building and deploying the entire system. Scientists can use it to experiment with game and incentive mechanism designs, and then infer and validate general models of motivation and utility that can be later used by other scientists and designers.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

KDD-HCOMP’10, July 25, 2010, Washington, DC, USA. Copyright 2010 ACM 978-1-4503-0222-7 ...\$10.00

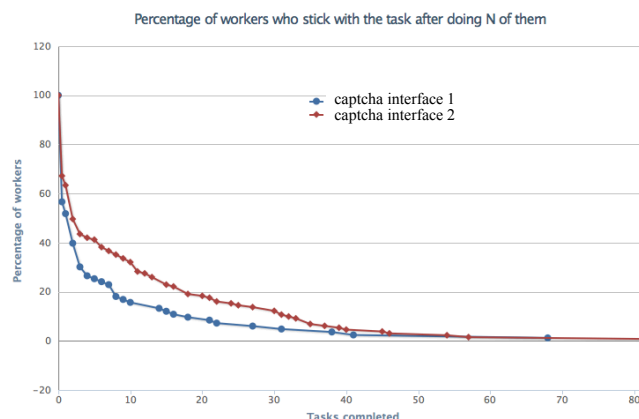


Figure 1. Users get bored of doing a task over time, decreasing its utility. This UTILISCOPE plot shows how quickly they bore of the repetitive task of answering captchas. Interface 2 is more usable than interface 1, and users stick with it for longer, giving it a higher utility curve. By varying the price on Mechanical Turk, our system calculated that the utility difference between these interface graphs is equivalent to 1 cent of pay.

2. MEASURING INTERACTION UTILITY

To use the UTILISCOPE, one uploads to it the URL of a web interface, a task description, and a payment, and in a matter of hours receives a number measuring the *cost* of the interface for accomplishing the task. To obtain this, the UTILISCOPE posts the task and interface to Mechanical Turk at a variety of prices, and records for each price how many people look at the task, choose to complete it, how many times they complete it, and other statistics of behavior and user demographics. A UTILISCOPE study typically takes 3-5 hours and costs \$20-\$200. The UTILISCOPE automatically runs a variety of statistical analyses on the data and displays graphs of the results.

3. CONCLUSION

Although Human Computation systems depend on their interaction utility, we do not yet know how to objectively measure it. The UTILISCOPE lets us measure and experiment with the utility of web-based user interface and social systems in a more principled way than traditional, lab-based user studies. Utility as a metric for user interface evaluation may become as important as time on task or error rate.

Translation by Iterative Collaboration between Monolingual Users

Chang Hu^{1,2}, Benjamin B. Bederson^{1,2} and Philip Resnik^{1,3,4}

¹Department of Computer Science, ²Human-Computer Interaction Lab,

³Institute for Advanced Computer Studies, ⁴Department of Linguistics,

University of Maryland

changhu@cs.umd.edu

ABSTRACT

In this paper we describe Monotrans, a new iterative translation process designed to leverage the massive number of online users who have minimal or no bilingual skill.

Categories and Subject Descriptors

H.5.m. [Information interfaces and presentation (e.g., HCI)]: Miscellaneous.

Keywords

Monolingual, translation, translation interface, human computation, distributed human computation, wisdom of crowds, crowdsourcing, machine translation.

1. INTRODUCTION

An enormous potential exists for solving certain classes of computational problems through rich collaboration between humans and computers. Take translation for example. Humans alone are expensive and can be surprisingly slow; despite significant recent advances, machine translation (MT) remains a crucial problem and fully automated high quality translation remains a distant dream for the vast majority of the world's language pairs. Usable translation quality can sometimes be obtained by statistical MT systems, but only for a minority of language pairs, and only in use cases where sufficient training text is available and the material being translated is reasonably similar to the material on which the system was trained.

Using the Web to reach non-professional human translators holds promise, and there has been some initial success with distributing translation over a crowd of bilingual users. However, compared to the total user population, the potential translator population is still small. For example, while Wikipedia currently has about 75,000 active contributors, there are fewer than 800 translators. With a much larger number of potential human helpers who speak only the source or target language, but not both, it seems natural to ask whether some combination of machine translation with volunteer monolingual speakers could result in high quality translation.

We propose a rethinking of the translation problem to bring

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

KDD-HCOMP'10, July 25, 2010, Washington, DC, USA. Copyright 2010 ACM 978-1-4503-0222-7 ...\$10.00

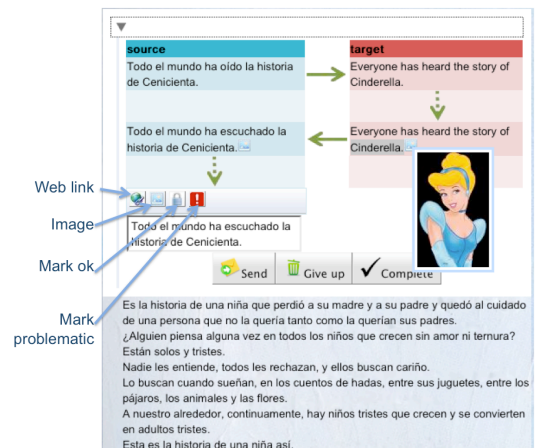


Figure 1 In this prototype UI showing a page, a sentence is expanded, with an image attached to a word.

together translation technology and human-computer interaction, producing a framework for translation that will exploit imperfect technology and limited human abilities in tandem to achieve capabilities neither can achieve alone.

The core of this framework is Monotrans, an iterative protocol in which the human participants work together to make sense of machine translations, and introduce redundant information to make their intended meanings clearer (see Figure 2). This protocol makes it possible to detect and correct some translation errors, and to at least identify some passages that have errors even if they are not correctable given the available information. For example, “has cheeseburger” is a detectable error, even if it is not clear whether the intended meaning was “has cheeseburgers” or “have a cheeseburger”. Back-translating a refinement and carrying along redundant information, e.g. a picture of multiple cheeseburgers, might help convey which of those alternatives the English speaker guessed, presenting the opportunity for confirmation or further correction.

2. RESEARCH PROTOTYPE

We built a research prototype as a multi-user web application (Figure 1). When a user logs in, the UI displays a book page in the user's language. Every sentence in the page is displayed with the most up-to-date translation hypothesis (or corresponding back-translation). The user can navigate through all available pages with navigation controls, or expand a sentence translation to edit it.

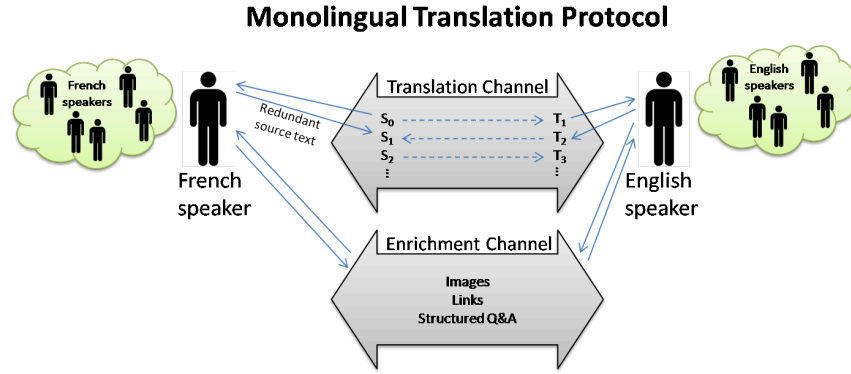


Figure 2 Round-trip protocol. Dashed arrows show machine translation and solid arrows show human editing.

When a sentence is expanded, the UI shows the most up-to-date translation hypothesis with all previous translation hypotheses of this sentence and a rich editor where the sentence can be edited and annotated.

The rich editor currently includes the following elements for the enrichment channel, aimed at enhancing redundancy and communicating shared context.

- Image annotations
- Web link annotations
- Annotation of correct parts of a sentence
- Annotation of incorrect parts of a sentence

Word level alignments necessary to perform annotation projection can be obtained from our own machine translation engines. Also, some machine translation services make word alignment information publically available to researchers along with the translation hypotheses. The Google Translate Research API (open to university research projects) is one such example.

3. PRELIMINARY RESULTS

We used Monotrans to translate part of a children's book from Russian to Chinese. Chinese and Russian are commonly spoken languages in the world. However, they make good experimental candidates because they are very different from the perspective of linguistic typology.

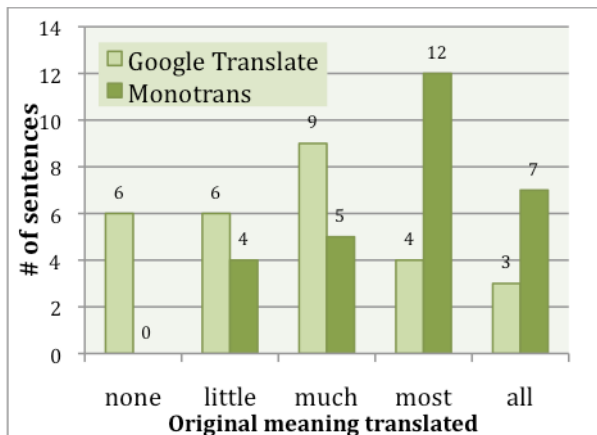


Figure 3 Number of sentences in each adequacy category, rated by a professional translator.

In the experiments, two Russian speakers and four Chinese speakers formed four pairs to use the prototype. (One Russian speaker participated three times, with different content.) The participants were all native speakers of one language and had no knowledge of the other. They were all computer-literate and fluent speakers of English. While most of the participants were computer science students and researchers, none of them work in the area of machine translation directly, and none of them were familiar with the details of this project. They were not linguists or linguistic students.

Participants worked on 6 pages (a total of 44 sentences) and finished translating 28 of them. This works out to approximately seven sentences per hour between any given pair of participants. It is about five times faster than the earlier “Wizard of Oz” experiment. With a standard rating procedure, sixteen of the 28 sentences translated with the prototype were rated as fully fluent and nineteen sentences of the 28 were rated as mostly or fully translated, by a professional translator not connected with the project.

The shift in adequacy is especially notable among these results. Completely inadequate MT outputs (none of the meaning preserved) dropped from 6 to 0. This means that the protocol helped the target language participants understand at least some of the meaning even when the original MT output quality was especially low and they had little to go on. In a coarse-grained way of thinking, if the adequacy rating could be categorized so that {none, little}=bad and {most, all}=good, then there would be a drop in bad (meaning) from 12 to 4 out of the 28, and there would be an increase in good from 7 to 19 of 28. That represents a factor of roughly 3 in each of the desired directions.

4. ACKNOWLEDGMENTS

This work is supported by the National Science Foundation.

5. REFERENCES

- [1] Chang Hu. Collaborative Translation by Monolingual Users. *CHI Extended Abstracts 2009*: 3105-3108
- [2] Chang Hu, Benjamin B. Bederson and Philip Resnik. Translation by Iterative Collaboration between Monolingual Users. *Graphics Interface 2010, Ottawa, Canada*

Sentence Recall Game: A Novel Tool for Collecting Data to Discover Language Usage Patterns

Jun Wang
School of Information Studies
Syracuse University
Syracuse, New York 13244
junwang4@gmail.com

Bei Yu
School of Information Studies
Syracuse University
Syracuse, New York 13244
byu@syr.edu

ABSTRACT

Recently we ran a simple memory test experiment, called *sentence recall*, in which participants were asked to recall sentences that they had just seen on the screen. Many participants, especially non-native English speakers, made various deviations in their recalled sentences. Some deviations represent alternative ways to express the same meaning, but others suggest that there are missing pieces in the participants' language knowledge. The deviation data, on the one hand, can provide individual users valuable feedback on their language usage patterns that they may never notice, on the other hand, can be used as training data for automatically discovering language usage patterns in a subpopulation of language learners. This paper presents our attempts to create an enjoyable sentence recall game for collecting a large amount of deviation data. Our results show that the game is fun to play and the collected deviation data can reveal common language usage patterns among non-native speakers.

Categories and Subject Descriptors

H.5.m [Information Systems]: Information Interfaces and Presentation (I.7)—*Miscellaneous*

General Terms

Design, Experimentation, Human Factors

Keywords

Human computation, Games with a purpose, Computer-assisted language learning, Language and memory, Educational gaming

1. INTRODUCTION

Common language usage patterns in a subpopulation of language learners have proven to be a very valuable language learning resource. For example, the misuse of English articles like “a” and “the” is very common among speakers who do not speak western European language (e.g. Russian or

Chinese speakers)[6]. These common error patterns span all aspects of language usages, from syntactic level article and preposition usage to deep semantic level collocations and idiomatic expressions. Capturing these common error patterns of particular language learner groups can help language teachers prepare more targeted teaching materials as well as increase language learners' awareness of their own habitual usage patterns.

A traditional way to identify these patterns is to build large-scale annotated corpora of non-native speakers. Because of the high cost to build such corpora, only a few of them have been made available[4], including the proprietary ones like the Cambridge Learner Corpus. The high cost and low accessibility prevents regular language researchers, educators, and learners from using them as research or learning materials.

Motivated by the ESP game and generally the idea of games with a purpose[7, 8], we introduce a game, called *sentence recall game*, to help collect large-scale data for discovering language usage patterns. In a sentence recall task, players are asked to recall sentences that they have just seen on the screen. During the recall, a player may generate a different sentence from the original one. The deviations between the recalled and the original sentences, such as word omission, insertion, or substitution, could mean random noises. However, if the similar deviations occur again and again among different players, we have reason to believe that these deviations can reveal common language usage patterns among them.

We have developed a game prototype and ran a pilot data collection experiment using the Amazon Mechanic Turk. Our results demonstrate that the game is fun to play and the collected data can reveal common language usage patterns among non-native speakers.

2. GAME DESIGN

A game consists of N rounds, and each round involves one or more sentence recall trials. In each trial, a sentence is briefly presented to the player, and then the player is asked to type it by recalling the sentence from his memory. If the player makes a successful recall, he will enter into the next round. A recall is successful if its normalized string edit distance from the original sentence is less than a threshold. If a player fails to recall a sentence, it could mean he does not pay enough attention or the sentence is difficult for him

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

KDD-HCOMP'10, July 25, 2010, Washington, DC, USA.
Copyright 2010 ACM 978-1-4503-0222-7 ...\$10.00

to grasp the meaning or the wording. In this case, he will be given another recall trial, on a different sentence. The game is over when the player successfully completes the last round or fails K times in one round.

To increase the enjoyability of the game and to elicit as much as possible useful recall deviations from players, we have employed the following techniques.

Animating sentences.

A sentence is displayed to a player with the trace of its words gradually fading with time (see Fig. 1). This way of dynamically displaying a sentence is used to mimic the sound effect of speaking a sentence, and in particular, to make it challenging for the player to memorize the specific wording of the whole sentence. As a side effect of the challenge, we could collect more deviation data because a player may remember the meaning of the sentence but forget the exact linguistic form, and thus he has to reconstruct the sentence using the language patterns stored in his long-term language knowledge[5].

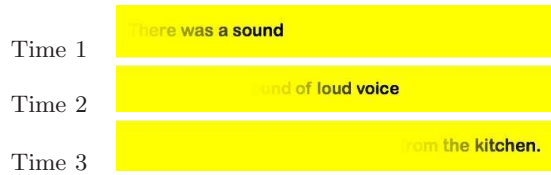


Figure 1: An illustration of sentence animation.

Varying sentence display speed.

Controlling the display or animation speed of a sentence is an important design factor. If the speed is too slow, players may find the game boring and can easily make a perfect recall; if it is too fast, players may not understand the sentence at all and thus generate very poor recalls, and as a result, they might get frustrated. Also, from the perspective of the goal of our game design, we are neither interested in collecting those perfect recalls that are identical to the originals, nor those poor recalls like random noises.

In our design, in the first round of a game, a sentence is displayed at a speed of 400 words per minute (wpm)¹. When a player enters into the next round, the sentence display speed will increase by 20%. If he fails, the speed will decrease by 10%. We have noted that when a player switches back and forth between successful and failed recalls, such abrupt changes in speed could make one really confused. To avoid this problem, we will consider in the future the use of a sigmoid function to adjust the speed[2].

Immediate deviation feedback.

Immediately after a recall attempt, a player will be shown both the original and the recalled sentence with deviations

¹Words per minute is a common measure of reading or writing speed. For the purpose of measurement, a word is standardized to be of the size of five characters, including the space key. The reading speed of an average American adult is around 300 wpm, and with the use of RSVP (rapid serial visual presentation), the speed can easily exceed 400 wpm. Considering that the way of animating sentences in our game bears a resemblance to the RSVP technique, we set 400 wpm as the starting speed.

highlighted. For example, Fig. 2 illustrates that when playing the sentence “*He seems to have washed his hands of the job*”, a player recalled the preposition “*of*” as “*off*”. Such feedback could help players, especially for non-native speakers, to discover their habitual language usage patterns and increase their awareness of the need to improve their language knowledge.

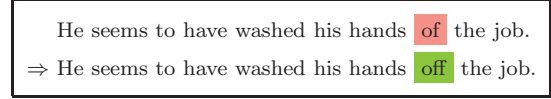


Figure 2: Immediate deviation feedback.

Scoring.

A player will get points for each successful recall. The amount of points that a player can get depends on the string edit distance of his recall from the original sentence.² It also depends on the sentence display speed: the faster the speed, the more the points. The scoring mechanism is designed to encourage players to try their best to make good recalls in the setting of fast sentence display speed—a perfect recall in a low speed setting is less worthy than a near perfect recall in a high speed setting.

3. EVALUATION

We seek answers to the following three questions: (1) will people enjoy playing such a simple sentence recall game? (2) will people like to use the game for language learning purposes? and (3) can the game play data reveal the common language usage patterns among the players especially those non-native speakers? In this section we focus on the first two questions, and leave the third one to the next section.

Our evaluation is based on the game play data and the survey data, collected from 33 native English speakers and 57 non-native speakers (including 46 Indian players³), who were mainly recruited through Amazon Mechanical Turk. We recruited twice. In the first time we put no restriction on the eligibility of the players, and in the second time we required that the players be from India since we wanted to collect more data from non-native English players—according to a recent report by Ipeirotis (2010), one third of turkers are from India[3].

About 300 sentences are used in the game. They were culled from the Google English Dictionary, with each sentence containing a number of between 8 and 12 words.

Enjoyability.

A simple analysis of the game log data shows that each player played for an average of 8.9 minutes. Over 1 out of 4 non-native players played the game at least twice—note that a turker only needed to play once to get paid, and about 1 out of 7 native players played twice. Specifically, 3 Indian players played for more than 40 minutes, including one who

²Considering that some word pairs, such as ‘can’ and ‘could’, are exchangeable, the distance between them will be 0.

³According to their self-reports, our Indian players speak the following 6 (native) languages: Gujarati, Hindi, Kannada, Malayalam, Tamil, and Telugu.

was referred to the game by his (her) turker friend.⁴ We believe these numbers show the game was fun to play.

In addition, many turkers also left comments saying that the game was fun to play. Here are a few examples:

“The words moved across the screen faster as the proficiency rose, and I was able to fill in some words that I did not catch clearly due to the connotation of the sentence. It’s a very addictive game.”

“Great simple game. Glad there was a ‘top 5’ at the end to gauge my score. Liked the increasing challenge.”

“This was actually one of the more entertaining psych-survey-type games that I’ve encountered. I don’t think I’ve ever seen a memory-style game use that particular method of obscuring the information to be remembered; it made it much less samey than other similar games.”

“It is fantastic playing the game. It is a game to increase your memory as well. Very interesting to do this even without earning money.”

Designing for learning purposes.

An important feature of our game is that it is also designed as a language learning tool. We asked turkers whether the game would be helpful for improving their language skills, and the responses are summarized in Table 1: 80% of non-native English speakers responded positively, while only 50% native speakers thought so. After receiving comments from an anonymous reviewer, we realized that a better survey question might be: *would you like to use the game for language learning purposes?* Regarding the evaluation of whether the game is effective for improving players’ language skills, we need to run a controlled experiment, but this remains to be future work.

Table 1: Responses about whether the game could be used for improving language skills.

	yes	maybe	no
native English speakers (NS)	81%	6%	13%
non-native English speakers (NNS)	49%	27%	24%

We also want to point out that quite a few non-native speakers specifically mentioned that the game is good *because* it can help improve their English. This is an encouraging sign since the game is mainly designed for discovering habitual language patterns among non-native speakers. We believe that, in the long run, an enjoyable game with educational purposes would be more likely to be adopted in a sustainable way.

4. LANGUAGE USAGE PATTERNS

Now we move on to the third question, raised in the last section, of whether the collected data can reveal common language usage patterns, especially common usage errors among non-native speakers. We have collected some 2,000

⁴We know this because the turker left a comment: *“I have bookmarked it and will certainly refer my friends to play it.”* And then, 20 minutes later, a player from India, unidentified, played for about an hour, with two short breaks each of a few minutes.

recalled sentences, about 40% of which are perfect recalls. Given that we have 300 source sentences in the pool, so each sentence, on average, only introduces a few recalls. Luckily, thanks to the use of the strategy of random-sampling sentences during the play, we have some 20 sentences being played 20 times or more. By generally analyzing the recalls on all the sentences and specifically examining the recalls on those frequently played sentences, we will demonstrate that some common language patterns can be revealed.

First, the collected data shows that non-native players tend to make various grammatical mistakes, which native players rarely did. For example, Table 2 demonstrates that non-native players have a tendency to neglect articles—a result consistent with what happened in the Japanese Learners of English corpus[4]. This shows that the omission of articles poses a serious problem for those non-native English speakers who do not speak western European languages[6].

Table 2: Non-native speakers tend to omit articles. NS: native speakers; NNS: non-native speakers.

omission	insertion		substitution	
	NS	NNS	NS	NNS
the	0	10	the → a	4 7
a	1	8	a → the	2 2

The 9 sentences that were involved in the omission of the article “the” by non-native speakers:

He believes food prices will go up in ~~the~~ future.
 Next door to ~~the~~ garage is a small orchard area.
 We shall take ~~the~~ treasure away to a safe place.
 The bullet had entered ~~the~~ top part of the brain.
 She went into her bedroom and lay down on ~~the~~ bed.
 He seemed to be just ~~the~~ right sort for the job.
 He dreams of changing the world for ~~the~~ better.
 Amy stood quietly in ~~the~~ doorway watching him.
 One of the office girls was down with ~~the~~ flu. (2 NNS)

Second, the game play data shows that non-native players are subject to making mistakes on idiomatic expressions. For example, for this sentence:

“To all intents and purposes the case was closed”,

all native speakers recalled the phrase “to all intents and purposes” perfectly, and all non-native speakers got it wrong, mutating it to: “to all the intents and purpose”, “to all intent and purpose”, “to all intends and purposes”, and “to all intentions and purposes”.

Another striking example is the common mistake made by non-native speakers when recalling the sentence

“He seems to have washed his hands of the job.”

It turns out that 7 among 11 of them recalled ‘of’ as ‘off’ while no English speaker made this mistake.

These examples reflect exactly how idiomatic expressions are processed and produced in human memory: they are holistically stored and accessible from memory as units, but for non-native speakers, the expressions are stored in pieces and therefore subject to mutation when reassembled[9].

Third, the data also shows that the players often provide alternative ways to express the same meaning or paraphrases (see Table 3 for a few examples). If a sentence was recalled by a large number of players, we would be able to show the distribution of alternative expressions and see which ones are more popular than others. Even though this game does not tell apart the right and wrong expressions, demonstrating various expressions and their occurrences would induce the players to compare them and think over which are more appropriate.

Table 3: Examples of paraphrase.

The boss covers all bases when he sets up a job.
⇒ covers all the bases (1 NS, 5 NNS)
⇒ a new job (3 NNS)
The man sat at his regular table near the window.
⇒ by the window (3 NS, 1 NNS)
⇒ at the window (2 NS, 1 NNS)
Sandman is a cult figure on the local music scene.
⇒ in the local music scene (2 NS, 2 NNS)
I had a pretty good idea what she was going to do.
⇒ a pretty good idea of (2 NS, 3 NNS)
It looked as though it might rain at any minute.
⇒ it might rain any minute (3 NNS)
So this could be the perfect time to buy a home.
⇒ to buy a house (2 NNS)
Within a matter of days she was back at work.
⇒ back to work (1 NS, 4 NNS)
I used to write during my free periods at school.
⇒ in school (3 NNS)

Currently, we have not employed any text mining algorithms to discover the language patterns in the collected data—only some simple statistics techniques were used. In the future, when a large amount of data is available, we would need to develop text mining algorithms to automatically summarize the common patterns.

5. DISCUSSIONS AND FUTURE WORK

The present work is closely related to the field of *computer-assisted language learning*. A problem for second language learners, especially for those intermediate and advanced learners who use the second language on a daily basis, is that many occurrences of their habitual mistaken language usages never get corrected. Even worse, these habitual usages are hard to detect in that the speakers or writers are not fully aware of them. Our game can be used as a tool to help learners improve their language skills. We believe recall deviation patterns, such as the recall of “*wash one’s hands of the job*”, as “*wash one’s hands off the job*”, would assist language teachers/learners to be better aware of the trends and nuances in language learning.

In addition, the game can also be used as a language assessment tool in that the recall deviations can be used to gauge the language proficiency of the players or language learners[10]. It is also worth to note that the emerged common (error) patterns or paraphrases can be used by language

educators to create fill-in-the-blank (cloze) tests.

Our work also contributes to the growing field of automatic error detection in the writings of non-native speakers[1]. Our game, if adopted by a large number of language learners, can generate large-scale training data that can be used in the development of writing error detection systems.

The sentences used in our current experiment were culled from the Google English Dictionary. An interesting extension of the game is to build a game platform that allows users to easily create, customize, and share their collections of sentences for use in different language use scenarios, such as academic writing and everyday conversation. For example, a teacher can load a customized game with sentences tailored to her teaching purposes, and then uses the game play data to find the common patterns of her students.

A limitation of the current form of the game is that it is only applicable to short sentences. One solution could be that a player only needs to recall part of a sentence, like the fill-in-the-blank tests. Good candidates for the blanks could be those formulaic phrases that are used regularly by native speakers[9]. The new approach also has the potential to speed up the pace of game play and greatly reduce players’ burden of typing a whole sentence.

6. ACKNOWLEDGEMENTS

We thank Veronica Maidel for helpful discussions and help setting up the Amazon Mechanical Turk task. We also thank the anonymous HCOMP 2010 reviewers for their insightful comments and suggestions.

7. REFERENCES

- [1] M. Gamon. Using mostly native data to correct errors in learners’ writing: A meta-classifier approach. In *HLT/NAACL*, pages 163–171, 2010.
- [2] S. Hacker and L. von Ahn. Matchin: eliciting user preferences with an online game. In *CHI*, pages 1207–1216, 2009.
- [3] P. G. Ipeirotis. Demographics of mechanical turk. <http://hdl.handle.net/2451/29585>, March 2010.
- [4] J. Lee and S. Seneff. An analysis of grammatical errors in non-native speech in english. In *IEEE Workshop on Spoken Language Technology*, 2008.
- [5] M. C. Potter and L. Lombardi. Regeneration in the short-term recall of sentences. *Journal of Memory and Language*, 29:633–654, 1990.
- [6] M. Swan. *Practical English Usage*. Oxford University Press, 2005.
- [7] L. von Ahn and L. Dabbish. Labeling images with a computer game. In *CHI*, pages 319–326, 2004.
- [8] L. von Ahn and L. Dabbish. Designing games with a purpose. *CACM*, 51(8):58–67, 2008.
- [9] A. Wray. *Formulaic language and the lexicon*. Cambridge Univ Press, 2002.
- [10] A. Wray and T. Fitzpatrick. Why can’t you just leave it alone? Deviations from memorized language as a gauge of nativelike competence. In *Phraseology in foreign language learning and teaching*. Amsterdam: John Benjamins, 2008.

Word Sense Disambiguation via Human Computation

Nitin Seemakurty, Jonathan Chu, Luis von Ahn, Anthony Tomasic

School of Computer Science
Carnegie Mellon University
Pittsburgh, PA

{nseemaku,jechu}@andrew.cmu.edu {biglou,tomasic}@cs.cmu.edu

ABSTRACT

One formidable problem in language technology is the word sense disambiguation (WSD) problem: disambiguating the true sense of a word as it occurs in a sentence (e.g., recognizing whether the word "bank" refers to a river bank or to a financial institution). This paper explores a strategy for harnessing the linguistic abilities of human beings to develop datasets that can be used to train machine learning algorithms for WSD. To create such datasets, we introduce a new interactive system: a fun game designed to produce valuable output by engaging human players in what they perceive to be a cooperative task of guessing the same word as another player. Our system makes a valuable contribution by tackling the knowledge acquisition bottleneck in the WSD problem domain. Rather than using conventional and costly techniques of paying lexicographers to generate training data for machine learning algorithms, we delegate the work to people who are looking to be entertained.

Categories and Subject Descriptors

I.2.6 [Learning]: Knowledge Acquisition H.5.3 [HCI]:
Web-based interaction

1. INTRODUCTION

The human language is ambiguous. That is, words can be interpreted with different meaning depending on their surrounding context. Take, for example, the following two sentences [4]:

- (a) I can hear *bass* sounds.
- (b) They like grilled *bass*.

The word *bass* refers to a low-frequency tone in one sentence while it refers to a type of fish in the other. Although this sense recognition seems intuitive to humans, it is a much more sophisticated task for a machine, which has to cope with the unstructured nature of the data (language). This computational identification a word's meaning in a given context is called *Word Sense Disambiguation (WSD)*.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

KDD-HCOMP'10, July 25, 2010, Washington, DC, USA. Copyright 2010 ACM 978-1-4503-0222-7 ...\$10.00

The relevance of WSD is becoming clear as advancing information/web technologies are catalysts for the production of enormous amounts of textual data, including articles, blogs, status messages, digitized books, etc. There is a growing need to introduce structure to this data in order to make it consumable and manageable by machines.

Current WSD algorithms use collections of data and machine learning algorithms to create models that determine the sense of the target word in the sentence. Generally supervised algorithms perform better than unsupervised algorithms [4]. These facts made human computation an ideal technique for this problem – with sufficient knowledge supervised algorithms can be used for almost all applications.

Currently knowledge acquisition for WSD is very expensive. Manual creation of a training dataset for a WSD system involves taking a large set of textual data, isolating words to disambiguate, and hand labeling each of these words with their gold label word sense. This process is an arduous and consequently an expensive one [3].

But what if we make this labeling process a pleasant one? This paper explores a new system: a game that is designed to capture human knowledge in a distributed fashion via an enjoyable game. Our study involves *assessing the effectiveness of this game in tackling the knowledge acquisition bottleneck*. Many elements of our game, named *Jinx*, are derived from a predecessor: the ESP Game [6].

1.1 Open Mind Initiative

Like the ESP Game, our game is much in tune with the efforts of the Open Mind Initiative [5], which focuses on collecting data from internet users in order to train machine learning algorithms. Our game is similar in that it attempts to use the efforts of regular internet users to tag the senses of words. However, as with the ESP Game, we place particular emphasis on the playability (i.e. viability) of our system.

2. GENERAL GAME PLAY

Jinx is an online cooperative two player game. When a player begins the game, he/she is anonymously paired with another random player. The anonymous pairing of players in most cases prevents any form of (cheating) communication between the two players. Each player interacts with the game independently. The players share only one aspect of the game: the current round. At any given time, both players view the same round, where a round is defined by a context (e.g. a sentence), and a highlighted word within that context.

The players are encouraged to rapidly type replacement words/phrases for the highlighted term. They are given incentive to type words that their partner is likely to type because both players are awarded points if and only if they both type the same string. As with the ESP Game, these players do not need to type their matching string at the exact same time, but both must have independently typed this string at some point during that round to receive points (see Figure 1 below).

We call the matching of two guesses for the same challenge a “tag”. Once a tag is collected, the game awards points to each player and then proceeds to the next round. In the case where agreement cannot be reached, the round expires after 30 seconds. Players are presented rounds for exactly 3 minutes, and then they are taken to a summary page that recaps their performance and offers to restart the game with another anonymous player.

3. GAME DESIGN

Our game was originally designed to be more of a quiz comprised of a series of multiple choice questions. The player would be presented with a highlighted word in context, and then given multiple definitions to choose from. These definitions were intended to reflect different interpretations of the highlighted term. The player would be rewarded if their choice matched with the partner’s choice. This setup, however, was inherently flawed.

- (a) **Random guessing.** This design allowed players to collect points by blindly selecting answers and rapidly progressing through rounds, hoping for a lucky match with the partner. One solution could be to penalize for mismatch, but that would mean that a normal player would be deducted points due to the misbehavior of their partner.
- (b) **Rigidity.** Limiting answer choices to a set of dictionary definitions also had the potential to create confusion if none of the definitions “worked”. Players would eventually choose an answer that only weakly approximates the word’s true meaning.
- (c) **Playability.** This design requires a player to read through dictionary definitions. These definitions are surprisingly complex, subtle, and lengthy. Playing a single round required minutes of concentrated work that would quickly exhaust a player.

For these reasons, we adopted a more open-ended approach that allows players to rapidly guess word replacement strings. This approach minimizes a player’s ability to match by randomly guessing. At the same time it makes the game more challenging and engaging by requiring that the players guess cooperatively, despite not being able to communicate. This cooperation emerges automatically from the task at hand and results in the generation of valuable tags.

3.1 Tag Quality

Because there are only a few possible word replacements that are reasonable given the sentence in any given round, players quickly recognize that making guesses from this limited set drastically increases their chances of matching with their partner. Consequently, the tags collected from the game are typically relevant word replacements.

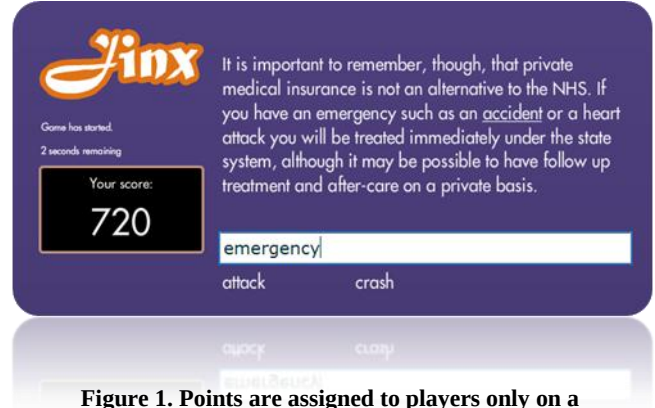


Figure 1. Points are assigned to players only on a match. The number of points rewarded depends on several different factors.

3.2 INTERACTION DESIGN

In designing the point system for Jinx, we had several goals in mind [7]. We wanted to keep the game fast paced while still allowing for high quality input from players. Fast pace is encouraged because prior dry runs of the game indicate that the matching tag (i.e. the best replacement for the word) is commonly a very early guess during the round (one made quickly after reading the provided textual context). Giving players a sense of urgency encourages them to guess what is most intuitive to them, and this tends to be a successful tag. To generate this urgency, we reward each player $P = f(t)$ points upon a matching guess, where t is the number of seconds remaining in the round when the matching guess was made, and f is an increasing function (we currently use $f(t) = 10 * t$). The faster a player generates a tag, the more points the player is awarded. Notice that one of the players will inevitably make the matching guess before the other player does; each player is rewarded accordingly.

Players are also awarded bonus points for successfully matching with their partner on consecutive rounds. The value of the bonus increases on every round of their consecutive streak. At the end of a round, we award each player $g(n)$ bonus points, where n is the number of consecutive matches so far, and $g(n) = 10 * n$. Note that while g is a linear function, its effect is geometric because the total bonus a player receives is $10 * (1 + 2 + 3 + \dots)$. This bonus system not only encourages players to keep playing, but also to keep playing with accuracy in mind. Most importantly, it keeps the game exciting.

4. GAME EVALUATION

To evaluate the game’s correctness in collecting quality tags, we utilize data from the HECTOR project [1].

HECTOR provides us with a large set of contexts (usually complete sentences), each of which contains one demarcated word (we will hereafter refer to such contexts as *challenges*). Each challenge is also assigned a word sense, which corresponds to the particular sense that the challenge word carries in that particular sentence. Different challenges contain the same challenge word and yet can carry word sense.

To measure the game’s ability to distinguish these alternate meanings, we injected a select subset of the HECTOR challenges into our backend and presented the game to a group of 11 players. The challenges were selected such that they involved only ten

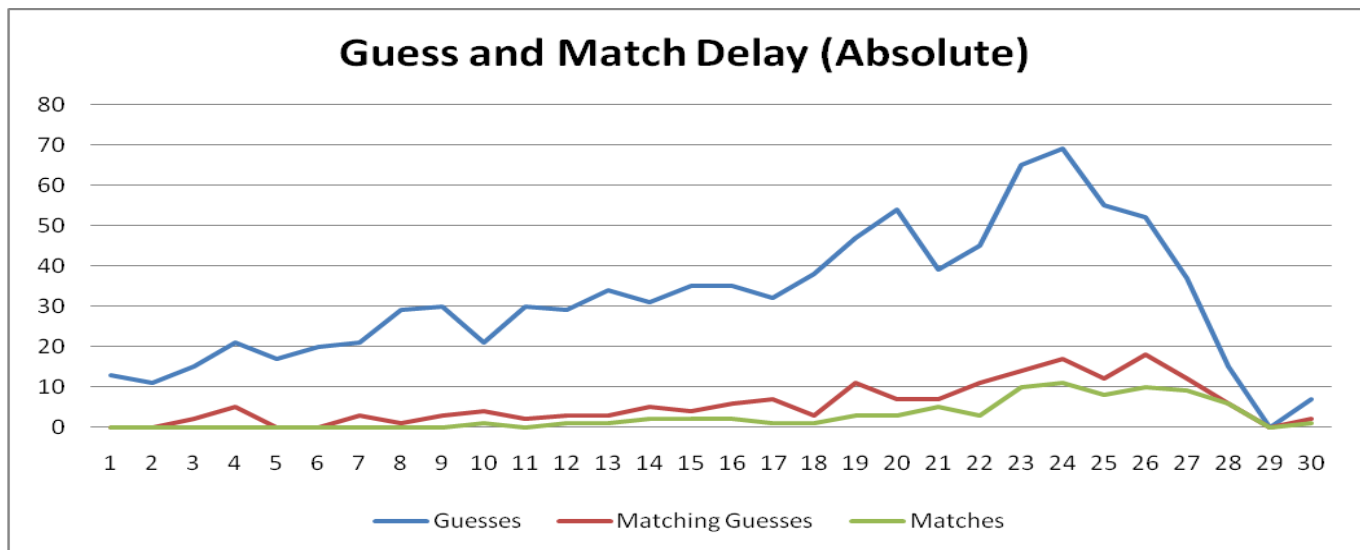


Figure 2: User Performance over Time

distinct challenge words but invoked multiple definitions of each of those words. The trial run lasted one hour.

4.1 Trial Run Observations

Several observations were recorded during the trial run:

1. The players' sentiment indicated that they found the game challenging, and consequently entertaining. While the players were keen to note several kinks in the implementation, most were intrigued by the concept.
2. Players felt that the game was too time constrained. The time limit for each round was 30 seconds. That is, players, once paired, had exactly 30 seconds to agree on a tag; otherwise, they were presented a new challenge. All participants in the study agreed that 30 seconds was too short a time period.
3. Interestingly, we were told by the players that, when presented an especially long challenge, they chose to read only the immediate context (5 to 10 words) surrounding the word. Much of the context provided by HECTOR is unnecessary to identify which meaning of a word is being invoked. Although presenting too little context has a chance of heightening the challenge word's ambiguity, it is clear that presenting less context than HECTOR offers can not only make the game less time constrained, but also make the game appear less formal and more fun.

Figure 2 is a graph of the activity of users over time. The "guesses" line (top) tracks the total number of guesses for a particular round. The "matching guess" line (middle) tracks the number of guesses that match *anyone* during the game. The "matches" line (bottom) tracks the number of guesses that match the paired player during the game. The data indicates a learning effect over time – players became more skilled at generating guesses and at generating guesses that matched. The drop off around round 25 was due to general fatigue and the trial session generally breaking up. This data also had one confounding factor

– challenges were randomly drawn from a limited pool, so some players received the same challenge more than once.

4.2 Tag Analysis

In general the dataset produced by Jinx has interesting linguistic properties since it generates synonyms as perceived by the general public. Some guesses are wrong, but a match generally indicates a synonym of interest. For example, the synonym "bad" was generated for the word "bitter". This reasonable synonym is not listed in WordNet 3.0 [2].

To determine the usefulness of the dataset for WSD, for each challenge with a tag, we looked up the WordNet synonym sets ("synsets") for the word and then attempted to isolate a single synset using the tag. Each synset corresponds to a unique word sense. For each attempt, a tag is classified into one of five distinct categories. A tag may *uniquely* identify a synset word sense. A tag may correspond to *none* of the synsets. The word itself may be *missing* from WordNet (e.g., word *kneed*). The tag may *partially match* a synset (e.g. tag *orchestra* matches synset *dance orchestra*). Or, the tag may match multiple synsets and thus be *ambiguous*. Table 1 summarizes the results of this analysis.

Table 1: Summary of Tag Analysis

Result	Count	Percent	Note
Unique	45	54%	Unique to a word sense
None	24	29%	No word sense
Missing	2	2%	No word definition
Partial	3	4%	Partial match to sense
Ambiguous	9	11%	Two word senses
Total	83	100%	

The data indicates that 54% of the tags are uniquely associated with a word sense. This figure indicates that the game is relatively efficient in producing word sense labels for challenges. The three tags that partially appear in a synset also fall in this category. Some 29% of the tags did not directly correspond to a word sense. In subsequent versions of the game we plan to use *taboo words* [6] as a method of forcing players to attempt to think of other guesses. The same technique can be used to eliminate tags that ambiguously appear in more than one WordNet synset (9%). In the data, 2% of the words were not found in WordNet – for these words, additional word sense definitions must be constructed by hand using the tags – fortunately only a small percentage of words fall into this category.

A unique association does *not* mean that the associated word sense is *correct*. Of the 48 tags that are uniquely or partially associated with a word sense, fourteen had a single tag that covered all senses for a word, providing no discrimination power what-so-ever. Again, these tags are good candidates as taboo words. To evaluate the remaining words, we cross referenced the HECTOR gold label answer to the closest WordNet sense. Twenty-five tags (30%) mapped to the correct sense, nine tags (11%) mapped to the wrong sense.

5. CONCLUSION

In this paper we described a game, named *Jinx*, which is designed to generate word sense disambiguation (WSD) datasets. These datasets can be used to train high quality machine learning algorithms for the WSD problem.

The game accomplishes this task in a low cost, distributed, fashion by employing human beings to consider a word in sentence and generate guesses for synonyms for the word in the context of the sentence. The game uses a point system to provide utility to users and uses a cooperative, paired player, structure to make the game fun and to control the quality of the guesses.

We populated the game with ten words and multiple sentences from a widely recognized word sense evaluation dataset [1]. We then had people play the game for approximately an hour. In post-game interviews, most everyone reported that the game was fun.

Thus we are confident that the game is capable of attracting a large, sustainable, audience.

Preliminary analysis of the guesses of the game indicates that many of the guesses correspond directly to synonym sets for words [2] in that context. Thus the game generates a set of synonyms to a particular word in a sentence *as perceived by the general public*. This dataset in itself contains interesting linguistic data. With respect to WSD, however, many guesses corresponded to more than one word sense or corresponded to incorrect word senses. We are currently exploring more sophisticated data analysis methods to extract a high quality WSD dataset.

6. ACKNOWLEDGMENTS

Thanks to Emily Leathers for work on this problem.

7. REFERENCES

- [1] A. Kilgarriff. 1998. Senseval: An exercise in evaluating word sense disambiguation programs.
- [2] Miller, G. A. 1995. WORDNET: A Lexical Database for English. Communications of ACM
- [3] Ng, T. H. 1997. Getting serious about word sense disambiguation. In Proceedings of the ACL SIGLEX Workshop on Tagging Text with Lexical Semantics: Why, What, and How? (Washington D.C.). 1–7.
- [4] Roberto Navigli. 2009. Word sense disambiguation: a survey. ACM Computing Surveys, 41
- [5] Stork, D. G. The Open Mind Initiative. IEEE Intelligent Systems & Their Applications, 14-3, 1999, pages 19-20.
- [6] Von Ahn, L. and Dabbish, L. 2004. Labeling images with a computer game. In Proc. ACM CHI.
- [7] Kraut, R. E. & Resnick, P. Evidence-based social design: Mining the social sciences to build online communities. Cambridge, MA: MIT Press. In preparation
- [8] Zellweger, P.T., Bouvin, N.O., Jehøj, H., and Mackinlay, J.D. Fluid Annotations in an Open World. Proc. Hypertext 2001, ACM Press (2001), 9-18.

Quality Management on Amazon Mechanical Turk

Panagiotis G. Ipeirotis
panos@stern.nyu.edu

Foster Provost
fprovost@stern.nyu.edu

Jing Wang
jwang5@stern.nyu.edu

Department of Information, Operations, and Management Sciences
Leonard N. Stern School of Business, New York University

ABSTRACT

Crowdsourcing services, such as Amazon Mechanical Turk, allow for easy distribution of small tasks to a large number of workers. Unfortunately, since manually verifying the quality of the submitted results is hard, malicious workers often take advantage of the verification difficulty and submit answers of low quality. Currently, most requesters rely on redundancy to identify the correct answers. However, redundancy is not a panacea. Massive redundancy is expensive, increasing significantly the cost of crowdsourced solutions. Therefore, we need techniques that will accurately estimate the quality of the workers, allowing for the rejection and blocking of the low-performing workers and spammers.

However, existing techniques cannot separate the true (unrecoverable) error rate from the (recoverable) biases that some workers exhibit. This lack of separation leads to incorrect assessments of a worker's quality. We present algorithms that improve the existing state-of-the-art techniques, enabling the separation of bias and error. Our algorithm generates a scalar score representing the inherent quality of each worker. We illustrate how to incorporate cost-sensitive classification errors in the overall framework and how to seamlessly integrate unsupervised and supervised techniques for inferring the quality of the workers. We present experimental results demonstrating the performance of the proposed algorithm under a variety of settings.

1. INTRODUCTION

Crowdsourcing has emerged over the last few years as an important new labor pool for a variety of tasks[3], ranging from micro-tasks on Mechanical Turk to big innovation contents conducted by Netflix and Innocentive. Amazon Mechanical Turk today dominates the market for crowdsourcing small tasks that would be too repetitive and too tedious for an individual to accomplish. Amazon Mechanical Turk established a marketplace where requesters can post tasks and workers complete them for relatively small amounts of money. Image tagging, relevance feedback, document labeling, are all tasks that are now routinely being completed online using the Amazon Mechanical Turk marketplace, delivering higher speed of

completion and lower price than in-house solutions.

EXAMPLE 1. Consider the following document labeling task: A worker has to look at a web site and decide whether there is any adult content on the page. The worker has to classify the page into one of the four categories: G for no adult content, PG for content that requires parental guidance, R for content that is appropriate mainly for adults, and X for hardcore porn. The rate of completion, when done by a trained intern, was 250 web sites per hour, at a cost of \$15/hr. When posted on Amazon Mechanical Turk, the labeling rate went up to 2,500 web sites per hour and the overall cost remained the same. In other words, the use of a highly distributed workforce that works in parallel, allows a tenfold improvement in productivity.

□

Unfortunately, distributing labeling work to crowdsourcing platforms, such as Amazon Mechanical Turk, exposes the requester to quality risks. Verifying the quality of every submitted answer is an expensive operation and negates many of the advantages of crowdsourcing: the cost and time for verifying the correctness of the submitted solutions is typically comparable to the cost and time for performing the task itself. A common solution to this challenge is to rely on redundancy and repeated labeling: the same task is completed by multiple workers. Then using rules, such as majority voting, it is feasible to identify the correct answers. Unfortunately, repeated labeling is rather costly: if we ask 10 workers to complete the same task, then the cost of crowdsourcing solutions tends to be comparable to the cost of in-house solutions.

A solution is to use redundancy not only for identifying the correct answers to each task but also for measuring the labeling quality of the workers. Dawid and Skene [2] proposed a solution, based on an expectation maximization algorithm. The algorithm iterates until convergence, following two steps: (1) estimates the correct answer for each task, using labels assigned by multiple workers, accounting for the quality of each worker; and (2) estimates the quality of the workers by comparing the submitted answers to the inferred correct answers. The final output of the Dawid&Skene algorithm is the set of (estimated) correct answers for each task and a “confusion matrix” for each worker, listing the error probabilities for each worker. From the confusion matrix we can directly measure the overall error rate for each worker as the sum of the non-diagonal elements of the confusion matrix (properly weighted by the priors): this results in a single, scalar value as the quality score for each worker.

Unfortunately, the error rate alone is not sufficient to measure the inherent value of a worker. For example, workers may be careful but biased. In the web site labeling example, parents with young children tend to be more conservative than

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

KDD-HCOMP'10, July 25, 2010, Washington DC, USA.

Copyright 2010 ACM 978-1-4503-0222-7 ...\$10.00.

others, and tend to classify *PG*-rated sites as *R*-rated sites, and *R*-rated sites as *X*-rated. Such workers give *consistently and predictably incorrect* answers. Unlike with spammers, with biased workers it is possible to “reverse” the errors and get a label assignment of much higher quality. So, in the presence of systematic bias, the naive measurement of error rate results in underestimates of the true quality of the worker and in potential incorrect rejections and blocks of legitimate workers.

As the main contribution of this paper, we present an algorithm that separates the unrecoverable error rate from bias. The basic idea is to use the confusion matrix of each worker and transform every assigned “hard” label from the worker into a “soft” label, which reflects the error rate of the worker. Then we can measure the uncertainty and cost associated with each such “soft” label. The final result is a quality score for each worker, potentially adjusted for the different costs of the misclassification errors, which measures the non-recoverable error rate of each worker. Our experiments illustrate that our algorithm is efficient in estimating the true quality of the workers.

2. ESTIMATING WORKER QUALITY

Dawid and Skene [2] presented an expectation maximization algorithm, using maximum likelihood, for inferring the error rates of annotators that assign class labels to objects, when the “gold” truth is unknown. Bayesian versions of the algorithm were recently proposed by Raykar et al. [4] and by Carpenter [1].

The EM algorithm of Dawid and Skene takes as input a set of N objects, $o_1, \dots, o_N$, each being associated with a *latent* true class label $T(o_n)$, picked from one of the L different labels. Each object is annotated by one or more of the K workers, each having a varying degree of quality. To measure the quality of each worker, the algorithm endows each worker (k) with a *latent* “confusion matrix” $\pi_{ij}^{(k)}$, which gives the probability that worker (k) , when presented with an object of true class i , will classify the object into category j .

Algorithm 1 presents a sketch of the process. The algorithm iterates between estimating the correct labels $T(o_n)$ for each of the objects, and estimating the error rates $\pi_{ij}^{(k)}$ for each worker. (The complete details are given in [2] and are rather secondary at this point.)

What is of interest to for our work, are the estimated error rates $\pi_{ij}^{(k)}$ for each worker and how we can use these values for estimating the non-recoverable error rate of each worker. So far, when estimating the quality of the workers, the main idea was to sum the non-diagonal entries of the matrix $\pi^{(k)}$ (i.e., sum all the values $\pi_{ij}^{(k)}$ with $i \neq j$), weighting each error rate using the estimated prior of each class. Unfortunately, this is insufficient. For example, consider the following example:

EXAMPLE 2. Consider two workers that label web sites into two classes: porn and notporn. Worker A is always incorrect: labels all porn web sites as notporn and vice versa. Worker B classifies all web sites, irrespectively of their true class, as porn. Which of the two workers is better? A simple error analysis indicates that the error rate of worker A is 100%, while the error rate of worker B is “only” 50%.¹ However, it is easy to see that the errors of worker A are easily reversible, while the errors of worker B are irreversible. In fact, worker A is a perfect worker, while worker B is a spammer. $\square$

¹Assume, for simplicity, equal priors for the two classes.

Input: Labels $l[k][n]$ from worker (k) to object o_n ,

Output: Confusion matrix $\pi_{ij}^{(k)}$ for each worker (k) , Correct labels $T(o_n)$ for each object o_n , Class priors $Pr\{C\}$ for each class C

- 1 Initialize error rates $\pi_{ij}^{(k)}$ for each worker (k) (e.g., assume each worker is perfect);
- 2 Initialize correct label for each object $T(o_n)$ (e.g., using majority vote);
- 3 **while** not converged **do**
- 4 Estimate the correct label $T(o_n)$ for each object, using the labels $l[\cdot][n]$ assigned to o_n by workers, weighting the votes using the error rates $\pi_{ij}^{(k)}$;
- 5 Estimate the error rates $\pi_{ij}^{(k)}$, for each worker (k) , using the correct labels $T(o_n)$ and the assigned labels $l[k][n]$;
- 6 Estimate the class priors $Pr\{C\}$, for each class C ;
- 7 **end**
- 8 **return** Estimated error rates $\pi_{ij}^{(k)}$, Estimated correct labels $T(o_n)$, Estimated class priors $Pr\{C\}$

Algorithm 1: The EM algorithm for worker quality estimation.

So, naturally a question arises: How can we separate low-quality workers from high-quality, but biased, workers? We examine this question next.

3. SEPARATING ERROR AND BIAS

After running the EM algorithm, we have some reasonably accurate estimates of the error rates $\pi_{ij}^{(k)}$ for each worker. How can we estimate from these values the intrinsic, non-recoverable error rate?

We start with the following observation: Each worker assigns a “hard” label to each object. Using the error rates for this worker, we can transform this assigned label into a “soft” label, which is the best possible estimate that we have for the *true* label assignment. So, if we have L possible classes and the worker assigns class j as a label to an object, we can transform this “hard” assigned label into the “soft” label:

$$\left\langle \pi_{1j}^{(k)} \cdot Pr\{C = 1\}, \dots, \pi_{Lj}^{(k)} \cdot Pr\{C = L\} \right\rangle \quad (1)$$

where $\pi_{ij}^{(k)}$ is the probability that worker (k) classifies into class j , an object that in reality belongs to class i , $Pr\{C = i\}$ is the prior that the object will belong to class i . We should note that the quantities above need to be normalized by dividing them with

$$Pr\{AC = j\} = \sum_i^L \pi_{ij}^{(k)} \cdot Pr\{C = i\} \quad (2)$$

the probability that worker (k) assigns label j to any object.

EXAMPLE 3. Take the case of worker A from the previous example. When this worker assigns a label of Porn (assume that porn is class 1), then the corresponding soft label has all the “probability mass” in the NotPorn category:

$$\underbrace{\begin{pmatrix} 1 \\ 0 \end{pmatrix}}_{\text{Assigned: Porn}} \Rightarrow \underbrace{\begin{pmatrix} 0 \\ 1 \end{pmatrix}}_{\text{Corrected to: NotPorn}}$$

On the contrary, for worker B, who always assigns porn, the corresponding corrected soft label does not give us any information; the soft label simply says that the best guess are simply the class priors:

$$\underbrace{\begin{pmatrix} 1 \\ 0 \end{pmatrix}}_{\text{Assigned: Porn}} \Rightarrow \underbrace{\begin{pmatrix} Pr\{C=1\} \\ Pr\{C=2\} \end{pmatrix}}_{\text{Corrected to: Class priors}}$$

□

So, what can we do with these soft labels? The basic idea is to estimate the *expected cost of a soft label*. To estimate the cost of a soft label, we need to consider the misclassification costs. In the simplest case, we have a cost of 1 when an object is misclassified, and 0 otherwise. In a more general case, we have a cost c_{ij} when an object of class i is classified into category j .

LEMMA 1. *Given the classification costs c_{ij} and a soft label $\mathbf{p} = \langle p_1, p_2, \dots, p_L \rangle$, the expected cost of the soft label $\mathbf{p}$ is:*

$$\text{Cost}(\mathbf{p}) = \sum_{i=1}^L \sum_{j=1}^L p_i \cdot p_j \cdot c_{ij} \quad (3)$$

□

The proof is rather simple. The expected classification cost is equal to the probability of classifying the object in class i (which is p_i), multiplied by the probability of the object belonging to class j in reality (which is p_j), multiplied with the associated cost of classifying an object of class j into class i (which is c_{ji}). Summing across all classes, we have the result above.

The results illustrate that workers with error rate matrices that generate “soft” labels with probability mass concentrated into a single class (i.e., certain “posterior” labels) will tend to have low estimated cost, as the product of Equation 3 will be close to 0. On the contrary, workers that tend to generate “soft” labels that are spread out across classes (i.e., uncertain “posterior” labels) will tend to have high associated costs.

EXAMPLE 4. *Consider the costs for the workers A and B from the previous examples. Assuming equal priors across classes, and $c_{ij} = 1$, if $i \neq j$ and $c_{ij} = 0$, if $i = j$, we have the following: The cost of worker A is 0, as the soft labels generated by A are $(0, 1)$ and $(1, 0)$. For worker B, the cost is 0.5 (the maximum possible) as the soft labels generated by B are all $(0.5, 0.5)$ (i.e., highly uncertain). □*

Given that we know how to compute the estimated cost for each label, we can now easily estimate the expected cost for each worker. We first compute the priors $Pr\{AC = i\}$ (see Equation 2), which is the prior probability of the worker assigning label i to an object. Then we compute the “soft label” that corresponds to the assigned label (see Equation 1). Given the soft label, we use Equation 3 to compute its expected cost. Now, knowing how often the worker assigns a label and the expected cost, we can compute the average misclassification cost of each worker. Algorithm 2 illustrates the process.

As expected, perfect workers will have a cost of zero and random workers or spammers will have high expected costs. Notice, as illustrated in the example above, that it is not necessary for a worker to return the correct answers in order to have low costs! As long as the errors are predictable and reversible, the worker is assigned a low expected cost.

This tends to resolve quite a few issues with online workers that exhibit systematic biases in their answers but also put a lot of effort in coming up with the answers. Prior approaches that relied on agreement generate a significant number of

Input: Error rates $\pi_{ij}^{(k)}$ for each worker, Misclassification costs $c[i][j]$, Class priors $Pr\{C\}$
Output: Expected for each worker

```

1 foreach Worker  $(k)$  do
2   Estimate how often the worker  $(k)$  assigns label  $l$ 
   ( $Pr\{AC = l\}$ ), using Eq. 2;
3    $Cost[k] = 0$ ;
4   foreach Label  $l$ , assigned with probability  $Pr\{AC = l\}$  do
5     Using Eq. 1, compute the soft label  $\mathbf{soft}^{(k)}(l)$  that
     corresponds to label  $l$  assigned by worker  $(k)$ ;
6     Using Eq. 3, compute  $Cost(\mathbf{soft}^{(k)}(l))$  for the soft
     label;
7      $Cost[k] += Cost(\mathbf{soft}^{(k)}(l)) \cdot Pr\{AC = l\}$ ;
8   end
9 end
10 return  $Cost[k]$  for each worker  $(k)$ 

```

Algorithm 2: Estimating the Expected Cost of each Worker

rejections for such workers, which in turn alienates such high-quality workers, and discourages them from working with employers that rely on worker agreement. The proposed algorithm alleviates these concerns.

One question that may arise is how to estimate the baseline cost for a “random” worker. An easy baseline is to assume that a worker assigns as label the class with the maximum prior probability. Alternatively, if we assume a more strategic spammer, we can set as baseline the expected cost of a worker that *always* assigns the *same* label, and this label assignment results in the minimum expected cost. Both approaches result in a good threshold for cutting off bad workers.

4. EXPERIMENTS

We conducted an extensive set of experiments, in order to examine how accurately the proposed algorithm can estimate:

- the true quality of the labelers, and
- the true class of the examples.

Synthetic Experiments: We conducted a set of synthetic experiments, in order to have the flexibility of controlling the composition of the data set and the quality of the workers. First, we created a data set with 1000 objects, classified with equal proportions to four categories. Then, we generated 200 labelers with various degrees of classification accuracy, ranging from 0.35 (slightly above the baseline of 0.25) to 0.8, for each of the four categories. We also applied a bias/disturbance for 30% of the workers, by randomly selecting two classes and swapping the assigned labels.

Figure 4 shows how accurately the algorithm can estimate the “true quality” of a labeler, under a variety of settings. We noticed that, for quality estimation purposes, the number of labels assigned by a worker, is an important factor for the estimating accurately the true quality of the worker. In general, we need 20-30 labels from each worker (and approximately 5 labels per example) in order to estimate with high degree of accuracy the true quality of the worker. We also noticed that the number of labels per worker affects significantly the estimation quality, while the number of labels per object has a much smaller effect.

Figure 4 shows the average classification error for our algorithm, when detecting the true class for each example, for varying the number of labels per worker, and the number of labels per object. We observed that the number of labels per worker tends to affect only mildly the accuracy of the

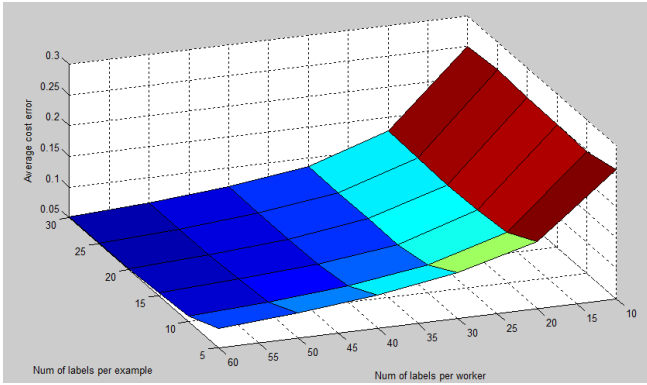


Figure 1: The estimation error when estimating the cost of each worker, under 0/1 classification error costs, with 4 categories of equal priors. 200 labelers with classification accuracy ranging uniformly from 0.35 to 0.8, and with a bias factor for 30% of the workers.

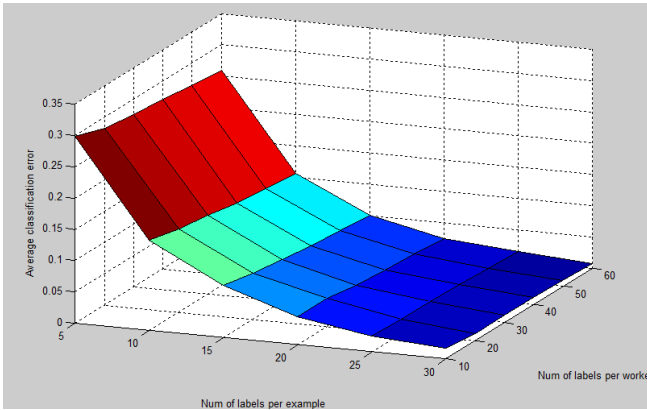


Figure 2: The average classification error under 0/1 classification error costs, with 4 categories of equal priors. 200 labelers with classification accuracy ranging uniformly from 0.35 to 0.8, and with a bias factor for 30% of the workers.

estimation. On the other hand, the effect of the number of labels per example was much more significant.

Experiments with Real Data: We also examined the effect of our algorithm for identifying spammers in a real-life task executed on Mechanical Turk. On this task, Mechanical Turk workers had to look at web pages and classify them into four categories (G, PG, R, X), depending on the presence of adult content on the page. (See Example 1.) We labeled 500 web pages, getting 100 labels for each web page. In total, 339 workers participated in the experiment.

To examine the effect of our algorithm, we executed our algorithm and measured the expected cost of each worker, using both our algorithm and the “naive” cost estimation using directly the confusion matrix of each worker. Then, we eliminated the workers that had an estimated cost of 0.5 and above. (A completely random worker generates a cost of 0.75, given that we have 4 categories.)

Our measurements indicated that our algorithm decreased the cost of annotation by 30%, while at the same time *increas-*

ing the quality of annotation from 0.95 to 0.998. In other words, spammers generated 30% of the submitted answers, and decreased the quality of the results. The same process, using the “vanilla” EM algorithm, decreased the cost by only 1% and the quality remained effectively unchanged. The main reason for this failure is the inability of the EM algorithm to identify the “strategic” spammers; these sophisticated spammers identify the class with the highest class prior (in our case, the “G” class) and label all the pages as such. Our cost algorithm takes into consideration the fact that these “G” labels are very uncertain, while the naive approach does not penalize the spammer for the “correct” G labels, which are unfortunately uninformative.

We expect the results to be much more dramatic in terms of quality when we have significantly fewer labels per example. In such cases, a few strategic spammers can completely destroy the quality of the data, and the presence of accurate cost estimation techniques is paramount.

5. CONCLUSIONS

We presented an algorithm for quality management of the labeling process on crowdsourced environments. The algorithm can be applied when the workers should answer a multiple choice question to complete a task. The main novelty of the proposed approach is the ability to assign a single scalar score to each worker, which corresponds to the quality of the assigned labels. The score separates the intrinsic error rate from the bias of the worker, allowing for more reliable quality estimation. This also leads to more fair treatment of the workers.

Our experimental results indicate that the algorithm becomes accurate with approximately 5 labels per example, and by asking each worker to label at least 20 to 30 objects (preferably 50). Given that multiple choice questions take typically only a few seconds this is a rather easy requirement for a large number of tasks.

We should also note that the algorithm can seamlessly integrate the existence of “gold” data for learning the quality of labelers. It is a trivial change in the algorithm (i.e., do not update the true class of the “gold” examples, in Step 4 of Algorithm 1) and tends to improve significantly the estimation accuracy in the face of sparse data.

The code is open source and available at <http://code.google.com/p/get-another-label/> and a demo is publicly accessible at <http://qmturk.appspot.com/>.

6. ACKNOWLEDGMENTS

Panagiotis G. Ipeirotis and Jing Wang were supported by the National Science Foundation under Grant No. IIS-0643846.

Bibliography

- [1] CARPENTER, B. Multilevel bayesian models of categorical data annotation. Available at <http://lingpipe-blog.com/lingpipe-white-papers/>, 2008.
- [2] DAVID, A. P., AND SKENE, A. M. Maximum likelihood estimation of observer error-rates using the EM algorithm. *Applied Statistics* 28, 1 (Sept. 1979), 20–28.
- [3] MALONE, T. W., LAUBACHER, R., AND DELLAROCAS, C. Harnessing crowds: Mapping the genome of collective intelligence. Available at <http://ssrn.com/abstract=1381502>, 2010.
- [4] RAYKAR, V. C., YU, S., ZHAO, L. H., VALADEZ, G. H., FLORIN, C., BOGONI, L., AND MOY, L. Learning from crowds. *Journal of Machine Learning Research* 11 (Apr. 2010), 1297–1322.

Exploring Iterative and Parallel Human Computation Processes

Greg Little¹, Lydia B. Chilton², Max Goldman¹, Robert C. Miller¹

¹MIT CSAIL
{glittle, maxg, rcm}@mit.edu

²University of Washington
hmslydia@cs.washington.edu

ABSTRACT

Services like Amazon’s Mechanical Turk have opened the door for exploration of processes that outsource computation to humans. These *human computation processes* hold tremendous potential to solve a variety of problems in novel and interesting ways. However, we are only just beginning to understand how to design such processes. This paper explores two basic approaches: one where workers work alone in parallel and one where workers iteratively build on each other’s work. We present a series of experiments exploring tradeoffs between each approach in several problem domains: writing, brainstorming, and transcription. In each of our experiments, iteration increases the average quality of responses. The increase is statistically significant in writing and brainstorming. However, in brainstorming and transcription, it is not clear that iteration is the best overall approach, in part because both of these tasks benefit from a high variability of responses, which is more prevalent in the parallel process. Also, poor guesses in the transcription task can lead subsequent workers astray.

Categories and Subject Descriptors

H5.m. Information interfaces and presentation (e.g., HCI): Miscellaneous.

General Terms

Algorithms, Measurement, Performance, Design, Reliability, Experimentation.

Keywords

Human Computation, Mechanical Turk, crowd sourcing, collaborative writing, brainstorming, OCR

1. INTRODUCTION

Countless examples on the web demonstrate the power of having a system outsource computation tasks to human workers. Human computation arguably wrote the world’s largest encyclopedia. We even used it to decipher the text in Figure 1. However, human



Figure 1: Mechanical Turk workers deciphered almost every word of this heavily blurred passage: “Killer whales are beautiful animals. I remember seeing these huge, beautiful, black and white creatures jumping high into the air at Sea World, as a kid.” (“beautiful” should be “smooth”)

computation processes are still not well understood. In order to make the most of this new technology, to make it more efficient, and apply it to more problems, we need to break it down and understand it better.

One difference we notice between human computation processes on the web is between iterative and parallel. For instance, if we look at how an article is written on Wikipedia, we often see an iterative process. One person starts an article, and then other people iteratively improve it by looking at what people did before them and adding information, correcting grammar, creating a consistent style, etc. On the other hand, designs on Threadless, a t-shirt design site, are generally created in parallel. Many people submit ideas independently, and then people vote to determine the best ideas that will be printed on a t-shirt.

We are interested in understanding the tradeoffs between each approach. The key difference seems to be that the iterative process shows each worker the results from previous workers, whereas the parallel processes asks each worker to solve a problem alone. The parallel process can be parallelized because no workers depend on the results of other workers, whereas the iterative process must solicit contributions serially.

To study the tradeoffs between each approach, we create a simple model wherein we can formally express the iterative and parallel processes. We then run a series of controlled experiments, applying each process to three diverse problem domains: writing image descriptions, brainstorming company names, and transcribing blurry text. These experiments are run using Amazon’s Mechanical Turk as a source of on-demand labor. This allows us to run the experiments multiple times, increasing statistical validity. Each process is coordinated automatically using TurKit [11].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

KDD-HCOMP’10, July 25, 2010, Washington, DC, USA.

Copyright 2010 ACM 978-1-4503-0222-7 ...\$10.00

Our results show that iteration improves average response quality in the writing and brainstorming domains. We observe an increase in the transcription domain as well, but it is not statistically significant, and may be due to other factors. In the case of writing, the increase in quality comes from the ability of workers to collaboratively write longer descriptions. We also observe a few downsides for iteration. In the case of brainstorming, workers riff on good ideas that they see to create other good names, but the very best names seem to be generated by workers working alone. In transcription, showing workers guesses from other workers can lead them astray, especially if the guesses are self-consistent, but wrong.

Overall, prototyping and testing these processes on Mechanical Turk provides insights that can inform the design of new processes. The contributions of this paper include:

- Model and design patterns for human computation processes that coordinate small paid contributions from many humans to achieve larger goals.
- A series of experiments that compare the efficacy of parallel and iterative design patterns in a variety of problem domains.
- A discussion of the tradeoffs observed between the iterative and parallel approaches.

This paper now proceeds with a discussion of related work. Then we present our model of iterative and parallel processes, and apply each process to three problem domains, discussing the results of each in turn. We end with a generalized discussion of our results, as well as proposals for future work.

2. RELATED WORK

Human computation in general has recently received a lot of attention. Quinn and Bederson give a good overview of distributed human computation systems [14]. Individual systems have also been studied and explored in academic literature, including Games with a Purpose [1] [2] [3], Wikipedia [5] [9], and Mechanical Turk [7] [8] [13] [15] [16].

Researchers have also tried to break down and categorize human computation, including Quinn and Bederson [14]. Their *aggregation* dimension is most applicable to this paper. It asks how work is coordinated and combined to achieve a final result. Malone et al. [12] also break down collective intelligence systems along several dimensions. They make a distinction between workers working *dependently* and *independently*, which roughly maps to our iterative and parallel processes. They also distinguish between *creation* and *decision* tasks. For example, Threadless uses a creation task to generate a bunch of designs, and then a decision task to pick a design. We borrow these names, but apply them at a lower level, e.g., a creation task for us refers to a single worker generating a single design. This distinction is also made by Kosorukoff [10] using the names *innovation* and *selection*. Selection in this case comes from thinking of human computation as a genetic algorithm, which is an applicable analogy to both our iterative and parallel processes. However, we prefer a model that allows for the expression of human computation that does not resemble a genetic algorithm. Our model treats human computation as a set of operators in addition to traditional computation.

Iterative and parallel human computation processes have been implemented in a number of places on the internet. Iterative processes are seen in wikis and open source collaborations. Parallel processes are seen in contest sites like Threadless and news aggregation sites like Slashdot, reddit, and Digg, as well as sites that collect content, like YouTube. However, all of these sites are quite different, so it is hard to see a controlled comparison between iterative and parallel processes just by observing the web.

3. MODEL

We are interested in human computation processes which coordinate small contributions from many humans to achieve larger goals. For example, an algorithm might coordinate many workers to write a description for an image. All of the creative and problem solving power in these processes will come from humans, e.g., humans will do the writing.

Typical user generated content, like image descriptions, comes in a form that the computer does not understand. In order for the human computation process to make decisions, it will need to ask humans to evaluate, rate, or compare content such that the result is a number, boolean, or other data representation that a computer can readily process. This suggests a breakdown of the domain into *creation* and *decision* tasks.



3.1 Creation Tasks

When a worker writes a description for an image, this is a creation task. Creation tasks solicit new content: writing, ideas, imagery, solutions. Tasks of this form tend to have few constraints on worker inputs to the system, in part because the computer doesn't understand the input. The goal of a creation task is to produce new high quality content, e.g., a well written and informative description for an image.

Many factors affect the quality of results. We are interested in exploring the potential benefits of iteration, where each worker is shown content generated by previous workers. The hope is that this content will serve as inspiration for workers, and ultimately increase the quality of their results.



3.2 Decision Tasks

If we have two descriptions for the same image, we can use a decision task to let the process know which is best. Decision tasks solicit opinions about existing content. Tasks of this form tend to impose greater constraints on worker contributions, since the computer will need to understand the contributions. The goal of a decision task is to solicit an accurate response. Toward this end, decision tasks may ask for multiple responses, and use the aggregate. In our experiments, we use two decision tasks: comparison and rating. The comparison task shows a worker two items, and asks them to select the item of higher quality. The order of items is always randomized in our comparison tasks.

The rating task shows a worker some content, and asks them to rate the quality of the content, with respect to some goal, e.g., "Please rate the quality of this text as a description of the image." All rating tasks in this paper use a rating scale from 1 to 10.

3.3 Combining Tasks: Iterative and Parallel

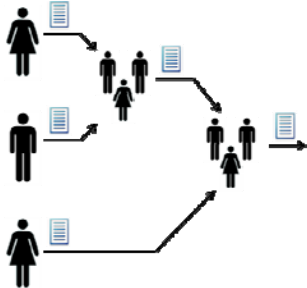
Human computation processes combine basic tasks in certain patterns. At this point, the patterns we explore are quite simple. All of the processes in this paper follow one of two patterns: iterative or parallel.

The iterative pattern consists of a sequence of creation tasks, where the result of each task feeds into the next one. The goal is to explore the benefits of showing workers content generated by previous workers. If it is not easy to merge the results of creation tasks, as is the case for image descriptions, then we place a comparison task between creation tasks:



This comparison lets the computer make a decision about which content to feed into the next task. In our experiments, we want to keep track of the best item created so far, and feed that into each new creation task.

The parallel pattern consists of a set of creation tasks executed in parallel. The parallel pattern acts as a control in our experiments—it is effectively the same as the iterative pattern, except that no workers are shown any work created by others. If it is not easy to automatically merge the results, then after all the results have been generated, we employ a sequence of comparison tasks to find the best result:



Our experiments also feed all subjective contributions into rating tasks in order to compare the effectiveness of each process.

4. EXPERIMENTS

In this section, we describe three experiments run on Mechanical Turk which compare the parallel and iterative processes in three different problem domains: writing, brainstorming, and transcription. Each domain was chosen because it solves a useful problem, and we wanted the combination to span a number of problem solving dimensions. For instance, the problems impose different levels of constraints: the transcription task has a definite answer, whereas the writing and brainstorming tasks are more open ended. The domains also vary in the size of each unit of work, ranging from single words to entire paragraphs.

4.1 Writing Image Descriptions

This experiment compares the parallel and iterative processes in the context of writing image descriptions. The experiment is inspired by Phetch [3], a game where humans write and validate image descriptions in order to make images on the web more accessible to people who are blind. This is a different approach to

• Please describe the image **factually**.

• You may use the provided text as a starting point, or delete it and start over.

• Use no more than 500 characters.

Lightening strike in a blue sky near a tree and a building.

character count: 59/500

Submit

Iterative: This image shows a large white strike of lightning coming down from a blue sky with the silhouettes of tops of the trees and rooftop peeking from the bottom. The sky is a dark blue and the lightening is a contrasting bright white. The lightening has many arms of electricity coming off of it. [rated 8.7](#)

Parallel: White lightning n [sic] a root-like formation shown against a slightly wispy clouded, blue sky, flashing from top to bottom. Bottom fifth of image shows silhouette of trees and a building. [rated 7.2](#)

Figure 2: Turkers are asked to write a factual description of an image. Turkers in the iterative condition are shown the best description so far, while the parallel condition always shows an empty text area. The resulting descriptions from each process are shown for this image.

the same problem that may be applicable to a greater variety of writing domains.

Each process has six creation tasks, each paying 2 cents. Five comparison tasks are used in each process to evaluate the winning description. Each comparison task solicits five votes, each for 1 cent. The instructions for the creation tasks are shown in Figure 2. The task asks a Mechanical Turk worker (*turker*) to describe the image factually in at most 500 characters. A character counter is updated continuously as the user types. The “Submit” button only activates when the content of the text area has changed from the initial text and there are at most 500 characters. Note that the instruction about “using the provided text” appears only in creation tasks that have text from a previous iteration to show. This instruction is omitted in all the parallel tasks.

To compare the processes, we selected 30 images from www.publicdomainpictures.net. Images were selected based on having interesting content, i.e., something to describe. We then ran both the parallel and iterative process on each image. For half

of the images, we ran the parallel process first, and for the other half, we ran the iterative process first.

In order to compare the results from the two processes, we created a rating task. Turkers were shown an image and a description, and asked to rate the quality of the description as a factual description of the image, on a scale of 1 to 10. We obtained 10 ratings for each image description to compute an average rating.

Turkers were not allowed to participate in both processes for a single image. They were also not allowed to rate descriptions for images that they contributed any writing to. However, turkers were allowed to contribute to multiple images, as well as rate multiple descriptions for the same image.

Our hypothesis was that the iterative process would produce better results. We reasoned that workers would be willing to spend a constant amount of time writing a description, and they could do more with that time if they had a description to start from.

4.1.1 Results & Discussion

Figure 2 shows an example image, along with the resulting description for both the iterative and parallel processes. In this case, the iterative description is rated higher than the parallel description. If we average the ratings of resulting descriptions in each process for all 30 images, we get a small but statistically significant difference in favor of iteration (7.9 vs. 7.4, paired t-test $T_{29} = 2.1$, $p = 0.04$). Figure 3 shows what the result would have been if we had run the process for n iterations. Note that the two processes are identical when we use only one iteration.

It is worth noting that there is a correlation between description length and rating: longer descriptions are given higher ratings, accounting for about 30% of the variability in ratings according to the linear regression in Figure 4 ($R^2 = 0.2981$, $N = 360$, $\beta = 0.005$, $p < 0.0001$). This makes sense since we asked for “factual descriptions,” and longer descriptions can hold more facts and details. The two circled outliers indicate cases of text copied from the internet that was only superficially related to the image.

This correlation is relevant because the iterative process produces longer descriptions, about 336 characters on average compared with 241 characters in the parallel process. This difference is enough to explain the difference we see in ratings. If we subtract the ratings predicted by a linear model of description length, then we are left with a residual rating of 0.37 for the iterative process, and 0.39 for the parallel process. However, a t-test reveals no statistically significant difference between these residual ratings ($p = 0.94$), meaning that the difference we saw before was probably due to description length.

However, note that the residual ratings are statistically significantly positive ($p = 0.014$), suggesting that each process produces descriptions that are rated higher than one might predict based on the length alone. This may be attributable to the fact that we have people vote between paragraphs, rather than simply comparing their lengths.

One simple model for what is happening in the iterative process is that it starts with a description of a certain length, and then subsequent turkers add more content. On average, turkers add about 25 characters in each iteration after the initial description. However, the standard deviation is very large (160), suggesting that turkers often remove characters as well. If we look more

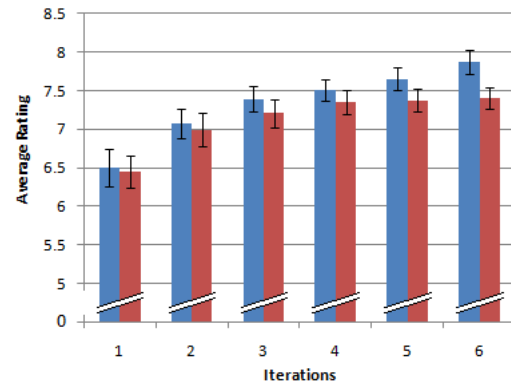


Figure 3: The average image description rating after n iterations (iterative process blue, and parallel process red). Error bars show standard error. As we run each process for additional iterations, the gap between the two seems to enlarge in favor of iteration, where the gap is statistically significant after six iterations (7.9 vs. 7.4, paired t-test $T_{29} = 2.1$, $p = 0.04$).

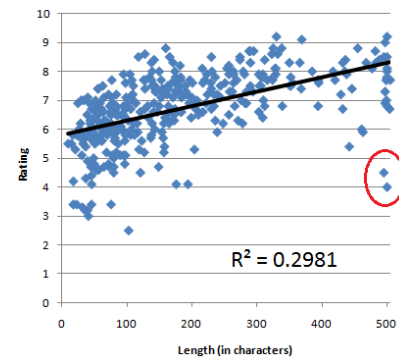


Figure 4: Descriptions are plotted according to their length and rating. A linear regression shows a positive correlation ($R^2 = 0.2981$, $N = 360$, $\beta = 0.005$, $p < 0.0001$). The two circled outliers represent instances of text copied from the internet.

closely at each of these instances, we can roughly classify their modifications as follows:

- 31% mainly append content at the end, and make only minor modifications (if any) to existing content;
- 27% modify/expand existing content, but it is evident that they use the provided description as a basis;
- 17% seem to ignore the provided description entirely and start over;
- 13% mostly trim or remove content;
- 11% make very small changes (adding a word, fixing a misspelling, etc);
- and 1% copy-paste superficially related content found on the internet.

Note that most modifications (83%) keep some of the existing content and structure, suggesting that these turkers may be doing less work. However, the average time spent by turkers writing or improving descriptions in each process is about the same, 211 seconds.

4.2 Brainstorming

This experiment compares the iterative and parallel processes in a different domain, namely brainstorming company names. Brainstorming is a popular process whereby many people generate ideas, either individually, or in a group. This process is well studied, and Taylor, et. al. [17] suggest that combining the results of individual brainstorms is more effective than having people brainstorm in a group. While group brainstorms typically generate fewer unique names, we try to mitigate this effect by programmatically enforcing that each worker contribute the same number of unique names in each process.

Each process has six creation tasks, each paying 2 cents. The instructions for these tasks are shown in Figure 5. The instructions ask a worker to generate five new company name ideas based on the provided company description. The “Submit” button only becomes active when there is text in each of the five input fields. The section that lists “Names suggested so far” only exists in the iterative condition. This list contains all names suggested in all previous iterations for a given company.

We fabricated descriptions for six companies. We then ran both the iterative and parallel process on each company description. As with the previous experiment, we ran the parallel variation first for half of the companies, and the iterative first for the other half. No turkers were allowed to contribute to both the iterative and parallel process of a single company description.

In order to compare the results of these processes, we used the rating technique discussed in the previous experiment to rate each generated company name. Again, we solicited 10 ratings for each company name, and averaged the ratings. Our hypothesis was that the iterative process would produce higher quality company names, since turkers could see the names suggested by other people, and build on their ideas.

4.3 Results & Discussion

By coincidence, no turkers in the parallel condition suggested duplicate names, resulting in 180 unique names for this condition. We removed duplicate names in the iterative condition from our data. They could have been prevented in JavaScript, and 13 of the 14 duplicate names came from the last three iterations for a single company. This may have been due to a group of turkers working together that collectively misunderstood the directions, or tried to cheat (unfortunately we did not record IP addresses, so we do not know if these turkers were collocated).

Figure 5 shows a fake company description, along with a sorted sample of the names suggested for this company. The best rated name generated in the iterative process is rated 7.3, compared with 8.3 for the parallel process. In fact, the parallel process generated the best rated name for 4 out of the 6 fake companies.

However, the *average* name generated in the iterative process is rated higher (6.4 vs. 6.2). The significance of iteration becomes clear in Figure 6, where we show the average rating of names generated in each iteration of the iterative process. The red line indicates the average rating of names in the parallel process. The iterative process is close to this line in the first iteration, where turkers are not shown any example names. The average rating seems to steadily increase as turkers are shown more and more examples (except for iteration four, discussed next). The last iteration is statistically significantly higher than the parallel process (6.7 vs. 6.2, two-sample $T_{203} = 2.3$, $p = 0.02$).

• Company details: Our company sells headphones. There are many types and styles of headphones available, useful in different circumstances, and our site helps users assess their needs, and get the pair of headphones that are right for them. • Please supply 5 new company name ideas for this company. Submit Names suggested so far: • Easy hearer • Least noisy hearer • Hearer of silence • Silence's hearer • Sharp hearer	
Iterative:	• Easy on the Ears <i>rated 7.3</i> • Easy Listening <i>7.1</i> • Music Explorer <i>7.1</i> • Right Choice Headphone <i>7.1</i> ... • Least noisy hearer <i>5.1</i> • Headphony <i>4.9</i> • Shop Headphone <i>4.8</i>
Parallel:	• music brain <i>rated 8.3</i> • Headphone House <i>7.4</i> • Headshop <i>7</i> • Talkie <i>6.8</i> ... • company sell <i>4.3</i> • head phones r us <i>4.2</i> • different circumstances <i>3.7</i>

Figure 5: Turkers are asked to generate five new company names given the company description. Turkers in the iterative condition are shown names suggested so far. The highest and lowest rated names from both the iterative and parallel processes are shown for this company.

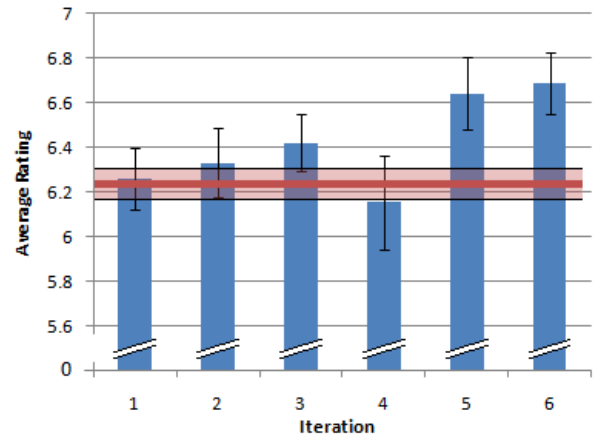


Figure 6: Blue bars show average ratings given to names generated in each of the six iterations of the iterative brainstorming processes. Error bars show standard error. The red stripe indicates the average rating and standard error of names generated in the parallel brainstorming processes. (See the text for a discussion of iteration 4, which appears below the red line.)

Iteration four breaks the pattern, but this appears to be a coincidence. Three of the contributions in this iteration were considerably below average. Two of these contributions were made by the same turker (for different companies). A number of their suggestions appear to have been marked down for being grammatically awkward: “How to Work Computer”, and “Shop Headphone”. The other turker suggested names that could be considered offensive: “the galloping coed” and “stick a fork in me”.

Like the previous task, we did not observe a statistically significant difference in the time that turkers spent generating names in each process: 202 seconds for iterative, and 178 seconds for parallel (two-sample $T_{70} = 1.1$, $p = 0.28$).

4.3.1 Getting the Best Names

Although iteration seems to increase the average rating of new names, it is not clear that iteration is the right choice for generating the *best* rated names (recall that the parallel process generated the best rated name for 4 of the 6 fake companies). This may be because the iterative process has a lower variance: 0.68 compared with 0.9 for the parallel process (F-test, $F_{180,165} = 1.32$, $p = 0.036$). The higher variance of the parallel distribution means that the tail of the distribution does not shrink as quickly, and must eventually surpass the iterative process. Figure 7 illustrates this phenomenon, and shows the crossover point at a rating of 8.04. This suggests that the parallel process is more likely than the iterative process to generate names rated 8.04 and above, assuming that the names are normally distributed according to this model.

Note that this model may not tell the whole story. The maximum possible rating is 10, which suggests that Gaussian curves are not the best model (since they do not have any bounds). A Beta distribution may be more appropriate. This does not mean that the effect we are seeing isn’t real, but it is worth further investigation.

One high level interpretation of this is that showing turkers suggestions may cause them to riff on the best ideas they see, but makes them unlikely to think too far afield from those ideas. We did see some anecdotal evidence of people’s ideas being heavily influenced by suggested names. For an online chat company, the first turker suggested five names that all incorporated the word “chat”. The next two turkers also supplied names that all incorporated the word “chat”. The corresponding iterations in the parallel process only used the word chat three out of fifteen times. For another company, a turker used the word “tech” in all their names, and subsequent turkers used the word “tech” seven times, compared with only once in the corresponding parallel iterations.

This suggests some interesting questions to explore in future work: does showing people name suggestions inhibit their ability to generate the very best new name ideas? If so, is there anything we can do in an iterative process that *will* help generate the best names? Alternatively, is there any way we can increase the variance in the parallel process, to have an even better chance of generating the best names?

4.4 Blurry Text Recognition

This experiment compares the iterative and parallel processes in the transcription domain. The task is essentially human OCR, inspired by reCAPTCHA [4]. We considered other puzzle possibilities, but were concerned that they might be too fun, which could have the side effects discussed in [13]. Unlike the

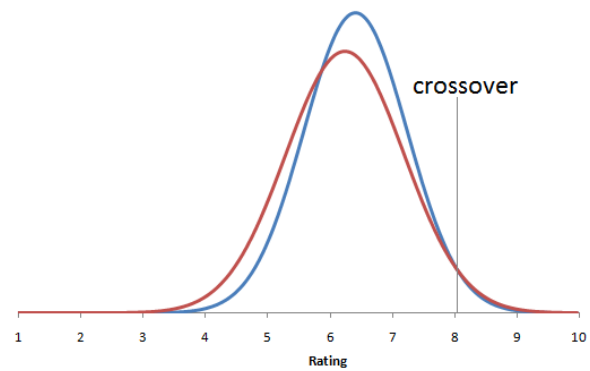


Figure 7: Gaussian distributions modeling the probability of generating names with various ratings in the iterative (blue) and parallel (red) processes. The mean and variance of each curve is estimated from the data. The iterative process has a higher average, but the parallel process has more variance (i.e. the curve is shorter and wider). Note that in this model, the parallel distribution has a higher probability of generating names rated over 8.04.

- Please transcribe as many words as you can.
- Put a * in front of words you are unsure about.

TV is supposed to be bad for you, but I am watching some TV shows. I think some TV shows are really entertaining, and I think it is good to be watched.

If a festival two me but is

If two If

*festival festival

Submit

Iterative: TV is supposed to be bad for you, but I ~~am~~ watching some TV shows. I think some TV shows are really entertaining, and I think it is good to be ~~watched~~. (94% correct)

Parallel: TV is supposed to be bad for you, but I like watching some TV shows. I think some TV shows are really ~~advertising~~, and I think it is good to be entertained. (97% correct)

Figure 8: Turkers are shown a passage of blurry text with a textbox beneath each word. Turkers in the iterative condition are shown guesses made for each word from previous turkers. The resulting transcription from each process is shown, with incorrect words struck out in red.

previous experiments, we use sixteen creation tasks in both the iterative and parallel processes, each task paying 5 cents.

Figure 8 shows an example blurry text recognition task. The instructions are simply to transcribe as many words as possible, and we place a textbox beneath each word for this purpose. In the iterative condition, these textboxes contain the most recent guess for each word. We also ask workers to put a “*” in front of words that they are unsure about. This is meant as a cue to future

workers that a word requires more attention. Note that this instruction appears in the parallel tasks as well, even though no workers see any other workers' stars.

We composed twelve original passages. It was important to use original text, rather than text obtained from the web, since turkers could conceivably find those passages by searching with some of the keywords they had deciphered.

We then ran each passage through an image filter. The filter works on a pixel level. Each pixel in the new image is created by randomly choosing a pixel near that location in the old image, according to a 2-dimensional Gaussian. This is identical to the lossy "blur" tool from some popular image editing programs. The result is blurry text that is very difficult to decipher. Some words appear to be entirely illegible on their own. The hope is that by seeing the entire passage in context, turkers will be able to work out the words. Note that we don't need to pay any workers to rate the results, since we can assess the accuracy of the results automatically using the ground truth text from which the blurry images were created.

We applied both the iterative and parallel process to each passage. As before, each process was run first half the time, and no turkers were allowed to participate in both processes for a single passage. The final transcription of the blurry text is extracted from each process on a word by word basis. We look at all the guesses for a single word in all sixteen iterations. If a particular word is guessed a plurality of times, then we choose it. Otherwise, we pick randomly from all the words that tied for the plurality. Note that other algorithms are possible, and we will have more to say about this below.

Our hypothesis was that the iterative process would have a higher probability of deciphering each passage, since turkers would be able to use other people's guesses as context for their own guesses. The analogy would be solving a cross-word puzzle that other people have already started working on.

4.4.1 Results & Discussion

Figure 8 shows a passage, along with the transcription extracted from both the iterative and parallel processes. In this case, the parallel process does a slightly better job (97% of words transcribed correct vs. 94%). When we average over all 12 passages, we fail to see a statistically significant difference between each process (iterative 65% vs. parallel 62%, paired t-test $T_{11} = 0.4$, $p = 0.73$).

If we look at the accuracy of each process after n iterations in Figure 9, we see that both processes gain accuracy over the first eight iterations, and then seem to level off. Note that the iterative process appears to be above the parallel process pretty consistently after the fourth iteration. The difference is greatest after eight iterations, but is never statistically significant.

The results suggest that iteration may be helpful for this task. However, it is also worth noting that iteration sometimes appears to get stuck due to poor guesses early in the process. For instance, one iterative process ended up with 30% accuracy after sixteen iterations. The final result was very similar to the eighth iteration, where most of the words had guesses, and they made a kind of sense:

8th iteration: "Please do ask * anything you need *me. Everything is going fine, there * *, show me then * * anything you desire."

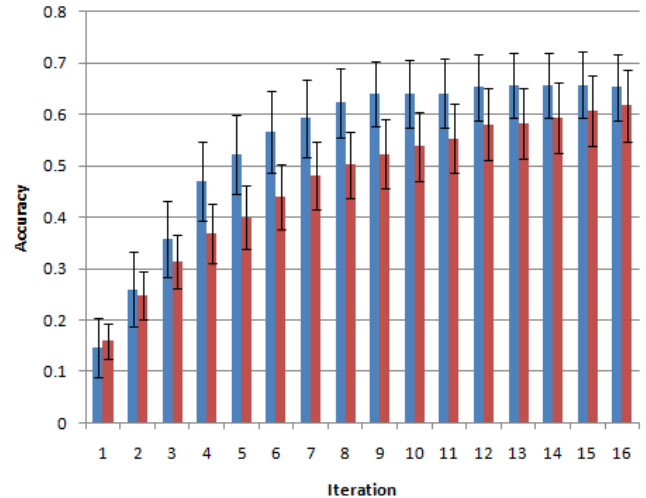


Figure 9: Blue bars show the accuracy after n iterations of the iterative text recognition process. Red bars show accuracy for the parallel process with n submissions. Error bars show standard error. The overlap suggests that the processes may not be different, or that we do not have enough power in our experiment to see a statistically significant difference.

16th iteration: "Please do ask ~~*about~~ anything ~~you need *me~~. Everything is ~~going fine, there *were *~~, show me then ~~*bring *~~ anything you ~~desire~~."

Incorrect guesses have been crossed out in red. Here is the actual passage: "Please do not touch anything in this house. Everything is very old, and very expensive, and you will probably break anything you touch." Note that multiple turkers deciphered this entire passage almost perfectly in the parallel process, suggesting that progress was hampered by poor guesses rather than by unreadable text. In fact, one turker left a comment alluding to this possibility (but for a different passage): "It's distracting with the words filled in--it's hard to see anything else but the words already there, so I don't feel I can really give a good "guess" ".

Note that in this experiment, we did see a statistically significant difference in the time spent deciphering text. Turkers in the iterative task spent less time, an average of 130 seconds, compared with 159 seconds in the parallel task (two-sample $T_{382} = -2.03$, $p = 0.043$).

4.4.2 Extraction Algorithm

Although iteration appears to be marginally better, some of the benefit may have to do with the algorithm we use to extract a final answer from all the guesses. If we had a perfect algorithm, one that could look at all the guesses for a particular word and choose the correct guess if it existed, even if it was not the most common guess, then the parallel process would do better: 76% versus 71% for the iterative process. This difference is not significant ($p = 0.64$), and only serves to cast greater doubt on the benefit of iteration for this task.

This suggests that any benefit we see from iteration may have to do with the fact that it has an implicit mechanism for multiple turkers to vote on a single turker's guess. If a turker leaves a guess alone, then they are implicitly voting for it. In the parallel

process, a word can only get multiple votes if multiple turkers arrive at the same guess independently.

One way the parallel process could be improved would be using other information to help pick out the best guesses. For instance, a word may be more likely to be correct if it came from a turker who made guesses for surrounding words as well, since the word may be satisfying more constraints.

It may be possible to improve the iterative process as well by randomly hiding guesses for certain words, in order to solicit more options for those words. Also, we could show people multiple options for a word, rather than just the most recent guess.

Although it is interesting to think of ways to improve the algorithm for this process, our real hope was that this task would be so difficult that a single turker could not accomplish it on their own. In future work, it would be nice to explore a task that had a higher level of difficulty, or impose a time limit to simulate higher difficulty.

5. DISCUSSION

All of these experiments demonstrate success in performing several relatively high-level creative and problem solving tasks, using processes that orchestrate the efforts of multiple workers. We also see that the breakdown into creation and decision tasks is applicable to a diverse set of problem domains.

5.1 Tradeoff between Average and Best

In the brainstorming task, we saw a tradeoff between increasing average response quality, and increasing the probability of the best responses. Showing prior work increased the average quality of responses, but reduced the variance enough that the highest quality responses were still more likely to come from turkers not shown any prior suggestions.

There is a sense in which this tradeoff exists in the other two tasks as well. The writing task generates descriptions with a higher average rating, but lower variance, when turkers are shown prior work. Also, the transcription task increases the average frequency of correct guesses for each word when turkers are shown a prior guess, but it decreases the variety of guesses for each word.

However, an alternate explanation for the reduced variance is simply that there is a maximum quality, so pushing the average toward this barrier must eventually reduce the variance (e.g., if the average rating reaches 10, then the variance must be 0). The real question is whether the reduction in variance is enough to exhibit the tradeoff we seem to observe in the brainstorming case. We will need to employ more robust mathematical models to make more progress on this front, since Gaussian distributions are limited in how well they can model a bounded random variable.

Investigating this tradeoff further may be worthwhile, because if it exists, then it implies two different alternatives for achieving quality responses, depending on our target quality. If our target quality is reasonably low, then we can increase the average, whereas if it is very high, then we may do better to increase the variance and find some method of detecting the high quality responses when they arrive.

Note that we may be able to proactively increase the variance in the brainstorming task, perhaps by showing people completely random words (which may have nothing to do with the topic), in order to get them thinking outside the proverbial box.

5.2 Not Leading Turkers Astray

In our experiments, showing prior work can have a negative effect on quality by leading future workers down the wrong path. This effect is most pronounced in the blurry text recognition task, and may be an issue in other tasks of this form where puzzle elements build on each other (like words in a crossword puzzle). Turkers take suggestions from previous turkers, and try to make the best guesses they can, but backtracking seems more rare.

This suggests that a hybrid approach may be better, where multiple iterative processes are executed in parallel to start with, and then further iteration is performed on the best branch.

6. FUTURE WORK

There are many directions for future work exploring human computation processes on Mechanical Turk. First, it seems fruitful to further explore the two basic building blocks: creation tasks and decision tasks. For instance, there are many factors that may influence creation tasks, including price, how much work is expected, whether examples are shown, and whether prior work is shown. We have only scratched the surface of exploring these possibilities, but as these different elements are better understood, it will be easier to design creation tasks with the desired balance between the average quality and variance of responses.

Decision tasks also leave room for investigation. The basic goal in our tasks has usually been to determine the best items in a set, but there are a number of ways to achieve this, including absolute ratings, pair-wise comparisons, and sorting multiple items in a single task. There are even combinations of these elements that may be useful, like having people sort items, but also provide ratings. Again, this is a large space, with the promise of providing useful knowledge to help optimize evaluation of subjective content.

Another direction for future work is exploring new building blocks. Even inside the paradigm of creation and decision tasks, there is room for building blocks which sit somewhere in-between. For instance, a creation writing task could ask a turker to select which of two previous versions they want to start from (where they are effectively voting for which of those is best). Note that this could have the side effect of turkers selecting the worse version from which to start, since it may be easier to improve. Hence, these ideas need to be tested and validated.

Finally, the real creative potential in this space is exploring new high-level processes for coordinating workers to perform better, or achieve loftier objectives. For instance, in paragraph writing, one can imagine breaking down the task into two steps. The first step might have people brainstorm phrases or concepts that should be included in a paragraph, and then these could be shown to the people writing the actual paragraph.

One might also imagine writing something larger than a paragraph through several steps: writing an outline, having separate processes to write each paragraph from the outline, and having another process to combine these results with transition sentences into a complete essay.

The ultimate goal is to learn how to design processes on Mechanical Turk that reliably and efficiently achieve their objectives, and to push the boundaries of those objectives.

7. CONCLUSION

This paper compares iterative and parallel human computation processes. In the iterative algorithm, each worker sees the results from the previous worker. In the parallel process, workers work alone. We apply each algorithm to a variety of problem domains, including writing, brainstorming, and transcription. We use Mechanical Turk and TurKit to run several instances of each process in each domain. We discover that iteration increases the average quality of responses in the writing and brainstorming domains, but that the best results in the brainstorming and transcription domains may come from the parallel process, because it yields a greater variety of responses. We also see that providing guesses for words in the transcription domain can lead workers down the wrong path. These results provide insights that can inform the design of new human computation processes.

8. ACKNOWLEDGMENTS

We would like to thank everyone who contributed to this work, including Mark Ackerman, Michael Bernstein, Thomas W. Malone, Robert Laubacher, and members of the UID group. This work was supported in part by Xerox, by the National Science Foundation under award number IIS-0447800, by Quanta Computer as part of the TParty project, and by the MIT Center for Collective Intelligence. Any opinions, findings, conclusions or recommendations expressed in this publication are those of the authors and do not necessarily reflect the views of the sponsors.

9. REFERENCES

- [1] von Ahn, L. Games With A Purpose. IEEE Computer Magazine, June 2006. Pages 96-98.
- [2] von Ahn, L., and Dabbish, L. Labeling Images with a Computer Game. ACM Conference on Human Factors in Computing Systems, CHI 2004. Pages 319-326.
- [3] von Ahn, L., Ginosar, S., Kedia, M., and Blum, M. Improving Accessibility of the Web with a Computer Game. ACM Conference on Human Factors in Computing Systems, CHI Notes 2006. pp 79-82.
- [4] von Ahn, L., Maurer, B., McMillen, C., Abraham, D. and Blum, M. reCAPTCHA: Human-Based Character Recognition via Web Security Measures. Science, September 12, 2008. pp 1465-1468.
- [5] Bryant, S. L., Forte, A. and Bruckman, A.. Becoming Wikipedian: transformation of participation in a collaborative online encyclopedia. GROUP 2005.
- [6] Dai, P., Mausam, Weld, D. S. Decision-Theoretic Control of Crowd-Sourced Workflows. AAAI 2010.
- [7] Heer, J., Bostock, M. Crowdsourcing Graphical Perception: Using Mechanical Turk to Assess Visualization Design. CHI 2010.
- [8] Kittur, A., Chi, E. H., and Suh, B. 2008. Crowdsourcing user studies with MTurk. CHI 2008.
- [9] Kittur, A. and Kraut, R. E. 2008. Harnessing the wisdom of crowds in wikipedia: quality through coordination. CSCW '08. ACM, New York, NY, 37-46
- [10] Kosorukoff A. Human based genetic algorithm. IlliGAL report no. 2001004. 2001, University of Illinois, Urbana-Champaign.
- [11] Little, G., Chilton, L. B., Goldman, M., and Miller, R. C. TurKit: Tools for Iterative Tasks on Mechanical Turk. HCOMP 2009.
- [12] Malone, T. W., Laubacher, R. and Dellarocas, C. Harnessing Crowds: Mapping the Genome of Collective Intelligence. MIT, Cambridge, 2009.
- [13] Mason, W., Watts, D. J. Financial Incentives and the "Performance of Crowds". HCOMP 2009.
- [14] Quinn, A. J., Bederson, B. B. A Taxonomy of Distributed Human Computation. Technical Report HCIL-2009-23 (University of Maryland, College Park, 2009).
- [15] Snow, R., O'Connor, B., Jurafsky, D., and Ng, A. Y. Cheap and fast---but is it good?: evaluating non-expert annotations for natural language tasks. EMNLP 2008.
- [16] Sorokin, A. and D. Forsyth. Utility data annotation with Amazon MTurk. Computer Vision and Pattern Recognition Workshops, Jan 2008.
- [17] Taylor, D. W., Berry, P. C. and Block, C. H. Does Group Participation When Using Brainstorming Facilitate or Inhibit Creative Thinking? Administrative Science Quarterly, Vol. 3, No. 1 (Jun., 1958), pp. 23-47.

Toward Automatic Task Design: A Progress Report

Eric Huang
School of Engineering and
Applied Sciences
Harvard University
Cambridge, MA 02138 USA
ehhuang@post.harvard.edu

Haoqi Zhang
School of Engineering and
Applied Sciences
Harvard University
Cambridge, MA 02138 USA
hq@eecs.harvard.edu

David C. Parkes
School of Engineering and
Applied Sciences
Harvard University
Cambridge, MA 02138 USA
parkes@eecs.harvard.edu

Krzysztof Z. Gajos
School of Engineering and
Applied Sciences
Harvard University
Cambridge, MA 02138 USA
kgajos@eecs.harvard.edu

Yiling Chen
School of Engineering and
Applied Sciences
Harvard University
Cambridge, MA 02138 USA
yiling@eecs.harvard.edu

ABSTRACT

A central challenge in human computation is in understanding how to design task environments that effectively attract participants and coordinate the problem solving process. In this paper, we consider a common problem that requesters face on Amazon Mechanical Turk: how should a task be designed so as to induce good output from workers? In posting a task, a requester decides how to break down the task into unit tasks, how much to pay for each unit task, and how many workers to assign to a unit task. These design decisions affect the rate at which workers complete unit tasks, as well as the quality of the work that results. Using image labeling as an example task, we consider the problem of designing the task to maximize the number of quality tags received within given time and budget constraints. We consider two different measures of work quality, and construct models for predicting the rate and quality of work based on observations of output to various designs. Preliminary results show that simple models can accurately predict the quality of output per unit task, but are less accurate in predicting the rate at which unit tasks complete. At a fixed rate of pay, our models generate different designs depending on the quality metric, and optimized designs obtain significantly more quality tags than baseline comparisons.

Categories and Subject Descriptors

J.4 [Computer Applications]: Social and Behavioral Sciences; J.m [Computer Applications]: Miscellaneous

General Terms

Design, Economics, Experimentation

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

KDD-HCOMP'10, July 25, 2010, Washington, DC, USA.
Copyright 2010 ACM 978-1-4503-0222-7 ...\$10.00.

Keywords

Human computation, Peer Production, Mechanical Turk

1. INTRODUCTION

In recent years there has been growing interest in the use of human computation systems for coordinating large-scale productive activity on the Internet. For example, a peer-production system like Wikipedia attracts tens of thousands of active editors that make millions of edits each month [13]. A game with a purpose like the ESP game attracts hundreds of thousands of players to label tens of millions of images through gameplay [12]. A crowdsourcing marketplace like Amazon Mechanical Turk allows requesters to post thousands of arbitrary jobs for hire each day, and attracts over a hundred thousand workers to complete these tasks [7].

A central challenge in designing human computation systems is in understanding how to design task environments that can effectively attract participants and coordinate the problem solving process. At a high level, the design of a human computation system consists of two components. One component is the design of incentives—social rewards, game points, and money—that help to attract a crowd and to encourage high quality work. The other component is the organization of individuals—the selection of participants, the assignment of subtasks, and the design of hierarchies—that help to usefully harness individual efforts to advance a system's purpose. In designing the environment for a particular task, the goal of the designer is to maximize the rate and quality of output, while minimizing costs.

In this paper, we consider a common problem that requesters face on Amazon Mechanical Turk: how should a task be designed so as to induce good output from workers? The problem exemplifies both the incentive and organizational aspects of the design challenge: in posting a task, a requester decides how to break down the task into unit tasks (called HITs, for human intelligence tasks), how much to pay for each HIT, and how many workers to assign to a HIT. These design decisions may affect the rate at which workers view and complete unit tasks, as well as the quality of the resulting work.

There are a number of challenges in effectively designing a task for posting on Mechanical Turk. The most notice-

able problem is that the effect of design on the rate and quality of work is often imprecisely known *a priori*, and likely dependent on the particular task and quality metric specified. While a designer may have some prior knowledge and be able to experiment with different designs, the design space is exponential in the number of design parameters and the number of experiments that can be performed is small. Furthermore, Mechanical Turk is inherently noisy, and any measurements obtained are affected in part by system conditions. Moreover, some statistics of interest, such as the number of currently active workers looking for tasks to perform, are unobservable by the requester.

In this work, we introduce a general approach for automatically designing tasks on Mechanical Turk. We construct models for predicting the rate and quality of work. These models are trained on worker outputs over a set of designs, and are then used to optimize a task’s design. We demonstrate our approach on an image labeling task, for which we aim to maximize the number of quality labels received within a given amount of time, subject to budget constraints. We consider two measures of quality: one based on the number of distinct labels received, and another based on the number of distinct labels received that match an external gold standard.

Experimental results show that simple models can accurately predict the output per unit task for both quality metrics, and that the models generate different designs depending on the quality metric we care about. For predicting the rate of work, we observe that a task’s completion time is correlated with the amount of work requested per dollar paid, and depends on the time during the day when a task is posted. But despite these effects, we find that the task completion time is nevertheless difficult to predict accurately and can vary significantly even for the same design. Focusing on using the quality prediction models for design, we find that for the same budget and rate of pay, optimized designs generated by our models obtain significantly more quality tags on average than baseline comparisons for both quality metrics.

1.1 Related work

Recent works have explored the effect of monetary incentives on worker performance on Mechanical Turk. Through a set of experiments, Mason and Watts [6] show that increasing monetary incentives induces workers to perform more units of a task, but does not affect the quality of work. In our image labeling task, we find that the quality of work can be accurately predicted without factoring in compensation. While related, our results are based on varying the task design and not the incentives for a particular design, and thus do not confirm or reject their claim. Focusing on labor supply, Horton and Chilton [2] study the effect of incentives on attracting workers to perform (multiple) HITs, and provide a method for estimating a worker’s reservation wage.

Other works have considered designing Turk tasks by organizing workers and aggregating output. Snow et al. [10] consider a number of different natural language annotation tasks, and show that annotations based on the majority output among a group of Turkers is comparable in quality to expert annotations, but is cheaper and faster to obtain. Su et al. [11] consider the effect of qualification tests on worker output, and show that workers with higher test scores achieve higher accuracy on the actual task. Along

an orthogonal direction, our work focuses on effectively distributing work across parallel subtasks.

An interesting example of organizing workers is TurKit [5], a toolkit for creating iterative tasks in which workers vote and improve upon work done by other workers. Little et al. [5] show that the use of voting and iteration allows for complicated tasks to be completed by a group of workers, even when the task is not easily divisible.

Building on TurKit, Dai et al. [1] propose TurKontrol, a system for controlling the request of additional voting or improvement tasks based on costs and the inferred work quality. Their work applies decision-theoretic planning techniques to optimizing sequential multi-HIT workflows. In contrast, our work focuses on a complementary challenge of learning about workers and on designing individual HITs. While the TurKontrol authors have yet to test their method on Mechanical Turk, we believe such decision-theoretic approaches can be effective when combined with learning about workers and within-HIT design.

Our work is inspired in part by theoretical work by Zhang et al. [14, 15, 16] on *environment design*, which studies the problem of perturbing agent decision environments to induce desired agent behaviors. The authors introduce models and methods for incentive design in sequential decision-making domains, and advance a general approach to design that learns from observations of agent behavior to (iteratively) optimize designs.

1.2 Outline

In Section 2 we introduce the Mechanical Turk marketplace and describe the image labeling task. Before exploring different designs for this task, we detail in Section 3 an experiment to capture the amount of variability on Mechanical Turk, where we post the same task design multiple times under varying system conditions. In Section 4 we discuss our initial experiments and report on the performance of models for predicting the rate and quality of work. We consider optimizing the task based on trained models in Section 5, where we compare the performance of optimized designs to baseline designs that pay at the same rate. We discuss the implications of our experiments for automatic task design, and outline the possibilities and challenges moving forward, in Section 6.

2. MECHANICAL TURK AND THE IMAGE LABELING TASK

Amazon Mechanical Turk (www.mturk.com) is a crowdsourcing marketplace for work that requires human intelligence. Since its launch in 2005, a wide variety of tasks have been posted and completed on Mechanical Turk. Example tasks include audio transcription, article summarization, and product categorization. Increasingly, Mechanical Turk is also attracting social scientists who are interested in performing laboratory-style experiments [3].

On Mechanical Turk, a *requester* posts jobs for hire that registered workers can complete for pay. A *job* is posted in the form of a group of *HITs* where each HIT represents an individual unit of work that a worker can accept. A requester can seek multiple *assignments* of the same HIT, where each assignment corresponds to a request for a unique worker to perform the HIT. The requester also sets the lifetime during which the HITs will be available and the amount of time

a worker has to complete a single HIT. Optionally, the requester can also impose a qualification requirement for a worker to be eligible to perform the task.

When choosing a task to work on, a worker is presented with a sorted list of available jobs, where for each job the title, reward, expiration time, and number of HITs available are displayed. The list can be sorted by the number of HITs available (the default), the reward, creation time, or expiration time. Workers can see a brief task description by clicking the title, or choose to ‘view a HIT in this group’ to see a preview of a HIT. At this point the worker can choose to accept or skip the HIT. If accepted, the HIT is assigned to that worker until the HIT is submitted, abandoned, or expired. Workers are not provided with additional information on the difficulty of tasks by the system, although there is evidence of workers sharing information on requester reputation via browser extensions and on Turk-related forums.¹

Upon receiving completed assignments, the requester determines whether to approve or reject the work. If an assignment is rejected, the requester is not obligated to pay the worker. While tasks vary greatly in pay and amount of work, the *reward* per HIT is often between \$0.01 to \$0.10, and most individual HITs require no more than ten minutes to complete. There are thousands of job requests posted at any given time, which corresponds to tens and sometimes hundreds of thousands of available HITs. For each HIT completed, Amazon charges the requester 10% of the reward amount or half a cent, whichever is more.

2.1 Our approach to task design

An exciting aspect of Mechanical Turk as a human computation platform is that it allows for any requester to post arbitrary tasks to be completed by a large population of workers. A requester has the freedom to design his or her task as desired, with the aim of inducing workers to exert effort toward generating useful work for the requester. The task design allows a requester to optimize tradeoffs among the rate of work, the quality of work, and cost. While some of the qualitative aspects of the tradeoffs are well understood—e.g., paying more will increase the rate of work, both because more workers will want to accept HITs and that each worker will want to complete more HITs [2]—optimizing the design to achieve particular tradeoffs requires a quantitative understanding of the effect. For some non-monetary aspects of task design, e.g., the division of a task into HITs and assignments, the effect on the quality and quantity of work is less well understood, even qualitatively. Such effects are also likely to be specific to the task at hand, and depend on particular requester goals and constraints.

We advance a particular design approach. For a given task, as a first step we consider a requester who experiments with a number of different designs, and uses the workers’ output and measurements of system conditions to learn a *task-specific model* of the effect of design on the rate and quality of work. As a second step, we then consider the problem of using learned models to generate good designs based on their predictions. We would like to understand whether a learned model can be effective in providing a useful way to guide the search for better designs. For the rest of the paper, we detail an application of this approach for the design of an image labeling task.

¹See <http://turkopticon.differenceengines.com/> and <http://www.turkernation.com/>, respectively.

Provide tags for images
Requester: EnvDes
Qualifications Required: HIT approval rate (%) is greater than 95

Reward: \$0.01 per HIT
HITs Available: 64
Duration: 30 minutes

Tag 1 image.

Guidelines:

- For each image, you must provide 3 distinct and relevant tags.
- You should strive to provide relevant and non-obvious tags.
- Tags must describe the image, contents of the image, or some relevant context.
- Your submitted tags will be checked for appropriateness.

Payments:

We approve HITs and pay in batches. Your submission will be approved/rejected within 7 days.

Image 1



Tags:

Figure 1: A HIT of the image labeling task

2.2 The image labeling task

We consider an image labeling task in which workers are asked to provide relevant labels (or equivalently, tags) to a set of images. Each HIT contains a number of images, and for each image, requests a particular number of labels for that image. Workers are informed of the number of images and number of labels required per image within the guidelines provided in the HIT, and are asked to provide ‘relevant and non-obvious tags.’ Workers can provide tags containing multiple words if they like, but this is not required nor specified in the instructions. See Figure 1 for a sample HIT that requests 3 labels for 1 image, for which possible labels may include ‘NASCAR’, ‘race cars’, ‘red’, ‘eight’, and ‘tires.’

We obtain a large dataset of images for our task from the ESP game,² which contains 100,000 images and labels collected through gameplay. From this dataset we use images that contain at least 10 labels, of which there are 57,745. Of these, we have used 11,461 images in our experiments. Any particular image we use appears in only one HIT.

We consider two metrics for judging the quality of labels received from workers. One metric counts the number of unique labels received, and is thus concerned with the number of relevant labels collected. The other metric counts the number of such labels that also appear as labels in our gold standard (GS) from the ESP dataset. Since such labels are those most agreed upon in the ESP game, they are labels that are likely to capture the most noticeable features of an image. In computing these metrics, we first pre-process labels to split any multi-word labels into multiple single-word labels, and convert upper case letters to lower case. We then apply the standard Porter Stemming Algorithm [8] to normalize worker and gold standard labels. This ensures that labels such as ‘dog’ and ‘dogs’ are considered the same label, which is useful for our measure of uniqueness and for comparing received labels to the gold standard. Finally, we remove *stop words* like ‘a’ and ‘the’, which accounts for 0.9% of gold standard labels and 4.6% of distinct labels collected.³

²<http://www.cs.cmu.edu/~biglou/resources/>

³We used a fairly short, conservative list of stop words from <http://www.textfixer.com/resources/>.

In designing the image labeling task, a designer can decide on the reward per HIT, the number of images and tags requested per image per HIT, the total number of HITs, the number of assignments per HIT, the time allotted per HIT, and the qualification requirements. The goal of the requester is to maximize the number of useful labels received as judged by the quality metric of interest, subject to any time and budget constraints. For example, a requester may have \$5 to spend, and aim to collect as many unique tags as possible within the next six hours. One can compare two different designs based on the amount of useful work completed within a certain time frame, or by examining the tradeoff between the work completed per dollar spent and the rate of work.

While all the design variables may have an effect on output, we focus our efforts on designing the reward per HIT, the number of images per HIT, the number of labels requested per image, and the total number of HITs. For all of our experiments, we fix the time allotted per HIT at 30 minutes (the default), but do not expect workers to spend more than a few minutes per HIT. We fix the number of assignments per HIT at 5; this gives us multiple sets of labels per image and will enable a study of the marginal effects of recruiting an additional worker to a HIT on the quality of output in future research. We require all workers to have an approval rate of at least 95%, such that only workers with 95% or more of their previously completed HITs approved are allowed to work on our task. This helps to keep chronic spammers out, but is not overly restrictive on the workers we can attract.

When posting tasks, we collect measurements of worker views and accepts over time, the amount of time a worker spends on a HIT, and the value of output as judged by our quality metrics. We also collect system conditions such as the time of day, the number of HITs available on Turk, the page position of our posting in different list orderings, and the number of completed HITs overall in Mechanical Turk. The last statistic is not available directly, and is estimated by tracking the change in the number of HITs available for tasks in the system at two minute intervals.

3. MEASURING OUTPUT VARIABILITY

Before considering the effect of design on output, we first report on the amount of variability in the output from Mechanical Turk when using a *fixed task design*. This lets us know how much variance to expect from the system, and allows us to study the effect of system conditions on output.

In particular, we consider a design for which each HIT has a reward of \$0.01, contains 1 image, and requests 3 labels. We posted a group of 20 HITs at a time, and posted 24 groups of the same task design from 4/12/10 to 4/20/10. Each group of HITs was allowed to run for approximately 8 hours, and groups of HITs were posted sequentially around the clock. All groups had at least 75% of the assignments completed, with 18 of the 24 groups finishing before the time expired.

Table 1 summarizes the mean and standard deviation of the rate and quality of output along a number of measurements.⁴ The task took 5 hours and 30 minutes to complete

⁴We measure the completion time of an unfinished task as the time until the job expires (~8 hours), and only measure the number of tags and unique workers for completed tasks.

Statistic	Mean	Std. Dev.
Time to 50% completion (min)	129.54	95.13 / 73%
Time to 100% completion (min)	330.44	124.93 / 38%
Total # of unique tags	264.56	18.06 / 7%
Total # of unique tags in GS	98.56	9.50 / 10%
# of unique workers	13.33	2.99 / 22%
Time to complete a HIT (s)	74.79	25.12 / 34%

Table 1: Statistics on an image labeling task with 20 HITs, where each HIT pays \$0.01 and requests 3 labels for 1 image.

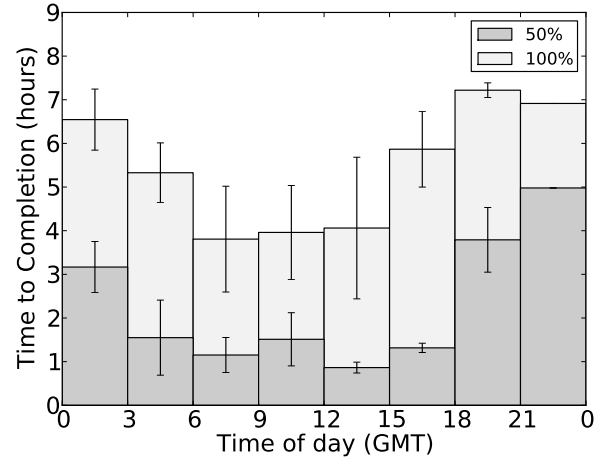


Figure 2: The effect of time of posting on time until 50% and 100% completion. Bins depict the average completion time of runs posted within a 3 hour period. Error bars represent the standard error.

on average, with the quickest run completing in just under 52 minutes and the longest run taking 8 hours and 37 minutes. Unlike task completion time, the number of unique labels received and the number of such labels that are in the gold standard vary much less, suggesting that the quality of output from workers remains relatively constant under different system conditions.

One possible explanation for the significant variation in completion time is that the activity level of workers on Mechanical Turk varies over time. While we don't know how many workers are active on Mechanical Turk at any given time, it is reasonable to think that activity level is correlated with the time of day, where the system is likely to be more active during particular 'work hours' than at other times. In Figure 2 we plot the relationship between the time of posting and the time by which 50% or 100% of the task completes. We observe that jobs posted between 6AM GMT and 3PM GMT complete most quickly; this corresponds to posting between 2AM to 11AM EST in the United States and 11:30AM to 8:30PM IST in India, the two countries that provide 80% of workers on Mechanical Turk [4]. Given that these times correspond to waking hours in India, we expect most of the workers interested in this task to be from India. We geolocate workers based on their IP addresses by using the Linux shell command `whois`. Of the IP addresses for which we can determine the country of origin (247 out of 307), 62% were from India and 23% were from the US, which is consistent with our intuition.

	Estimated Model		R^2	RMSE	RRSE	MAE
	$N_{pic}N_{tag}$	$N_{pic} \log(N_{tag})$				
Diminishing returns in tags		1.9157 (0.0743)	0.7681	1.6617	0.4816	1.1341
Linear in tags	0.8426 (0.0140)		0.9576	0.7105	0.2059	0.5491
Diminishing returns in tags		0.7241 (0.0115)	0.9576	0.2574	0.2057	0.1743
Linear in tags	0.3050 (0.0115)		0.7652	0.6064	0.4853	0.4406

Table 2: Summary of model coefficients with standard errors and goodness of fit for predicting the average number of quality labels received per assignment. The top two models predict the number of unique tags collected, and the bottom two models predict the number of unique tags collected that are in the gold standard.

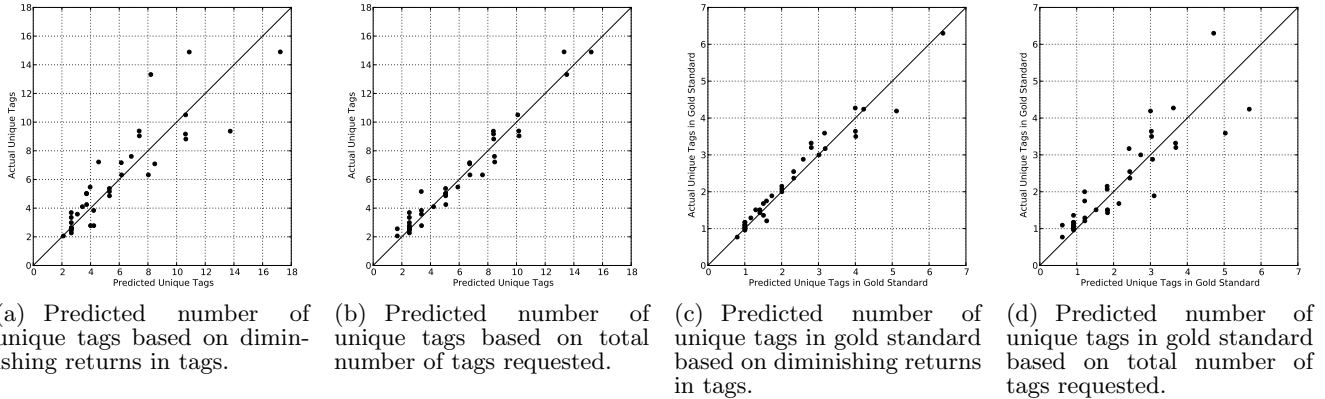


Figure 3: Predicted vs. actual number of quality tags received per assignment

4. INITIAL EXPERIMENTS AND BEHAVIORAL MODELS

From the variability measurements we learned that the completion time of a task may be highly variable, and may be difficult to predict accurately even for a fixed design. While some of the time variability can be explained by the time of day in which the task is posted, there is still a substantial amount of residual noise. In contrast, we find that the quality of work does not vary much with system conditions. Based on these observations, we expect the task design to only partially influence the rate of work, but account almost entirely for the quality of work.

In order to understand the effect of design on worker output, we consider learning models for predicting the quality of labels received per HIT and the completion time. We perform a series of 38 initial experiments—which serves as our training data—in which we vary the task’s design (or configuration) by changing the reward (R), the number of images (N_{pic}) and number of labels per image per HIT (N_{tag}), and the number of HITs (N_{hits}). We consider rewards in the range of \$0.01 and \$0.10, and vary the number of images and tags requested between 1 and 10, respectively. In choosing the configurations, we aim to cover a large range of values along each dimension, and to vary the total number of tags requested per dollar pay, i.e., $N_{pic}N_{tag}/R$. For the most part we consider jobs that consist of groups of 20 HITs (in 31 configurations), but also include a few jobs containing 30, 150, 500, and 1000 HITs, respectively. Configurations were randomly ordered and allowed to run until completion, and were automatically posted in series over a three week

period from 2/2/10 to 2/24/10 with no time gaps between postings. As stated previously, we fix the number of assignments (N_{asst}) requested per HIT at 5, and require all workers to have an approval rate of at least 95%.

In considering models for predicting the rate and quality of work, we measure the goodness of fit by reporting the coefficient of determination (R^2), the root mean square error (RMSE), the root relative square error (RRSE), and the mean absolute error (MAE), between predicted and actual output. All statistics are computed for the hold-out data via leave-one-out cross-validation.

4.1 Predicting label quality

We consider simple models for predicting the average number of quality labels received from workers. A summary of model coefficients and fitness is presented in Table 2.

4.1.1 Predicting unique tags

For predicting the average number of unique tags received per assignment (N_{unique}),⁵ we hypothesize that we would experience diminishing marginal returns as we request more tags per image, suggesting the following model:⁶

$$N_{unique} = \beta N_{pic} \log(N_{tag}) + \epsilon$$

⁵We compute the per assignment contribution by dividing the number of quality tags collected per HIT by the number of assignments, which is fixed at 5.

⁶When taking a log, we smooth the input data by adding 1 to the number of tags (N_{tag}) to ensure the feature has weight instead of evaluating to zero.

	Estimated Model				R^2	RMSE	RRSE	MAE
	$\frac{N_{pic}N_{tag}}{R}$	$\cos(t)$	$\sin(t)$	Intercept				
Pay	25.58 (6.81)			-373.45 (3452.30)	0.45	12418.24	0.88	7898.78
Pay+Time	21.47 (5.26)	12173.01 (2542.22)	1019.55 (2265.39)	5177.23 (2864.85)	0.70	9819.85	0.70	7549.67
Pay	49.79 (8.06)			4955.10 (4081.98)	0.68	14914.62	0.72	11583.57
Pay+Time	44.71 (6.35)	13775.48 (3072.81)	-3103.65 (2738.20)	10801.1 (3462.78)	0.79	12630.06	0.61	9655.45

Table 3: Summary of model coefficients with standard errors and goodness of fit for predicting completion time (in seconds). The top two models predict the 50% completion time, and the bottom two models predict the completion time.

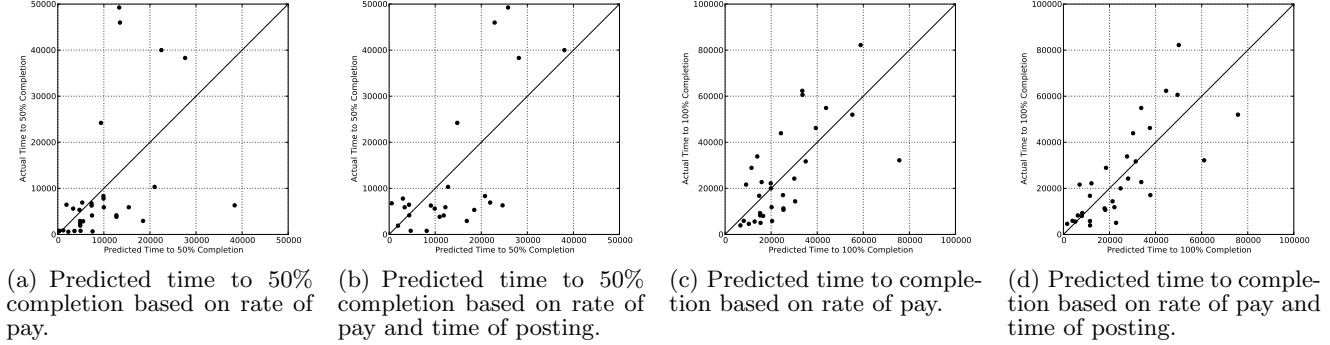


Figure 4: Predicted vs. actual time until 50% and 100% completion (in seconds).

We find that the model’s predictions are somewhat accurate, with $R^2 = 0.77$ and $RRSE = 0.48$. We also consider a model without diminishing marginal returns in the number of labels requested:

$$N_{unique} = \beta N_{pic} N_{tag} + \epsilon$$

Surprisingly, we observe a significantly better fit, with $R^2 = 0.96$ and $RRSE = 0.21$; see Figures 3(a) and 3(b) for a comparison between the two models’ predictions. This model suggests that the proportion of overlap in tags entered across the 5 assignments is invariant to the number of tags requested, and that at least within the range of values in our training data we don’t observe workers running out of tags to describe an image.

4.1.2 Predicting unique tags in gold standard

For predicting the average number of unique tags received per assignment that are in the gold standard (N_{gs}), we again hypothesize that there would be an effect of diminishing marginal returns as we request more tags per image. Since there is a limited number of tags per image within the gold standard with which the collected tags can match, we would expect the effect of diminishing returns to be much stronger than for our other quality metric. We consider the following model:

$$N_{gs} = \beta N_{pic} \log(N_{tag}) + \epsilon$$

The prediction is highly accurate, with $R^2 = 0.96$ and $RRSE = 0.21$. The model’s fit is significantly better than the fit of a model without diminishing returns ($R^2 = 0.77$, $RRSE = 0.49$); see Figures 3(c) and 3(d) for a comparison.

4.2 Predicting completion time

We turn to consider models for predicting completion time based on a task’s design. A summary of model coefficients and fitness is presented in Table 3.

Intuitively, a task is more attractive if the pay is high but the amount of work is low. Given similar amounts of work, we would expect the number of tags requested per dollar pay (rate of pay) to be correlated with a task’s completion time. We consider all 31 configurations with 20 HITs from our training data, and predict the 50% completion time ($T_{1/2}$) and 100% completion time (T) using the following model:

$$T = \beta_0 + \beta_1 \frac{N_{pic} N_{tag}}{R} + \epsilon$$

We see that the rate of pay is correlated with the completion time, with $R^2 = 0.68$ and $RRSE = 0.72$ for predicting 100% completion. The correlation is weaker for predicting 50% completion time, with $R^2 = 0.45$, $RRSE = 0.88$.

From the results of our variability study, we also expect the time of posting to affect the completion time. As we saw in Figure 2, the effect of time of day on completion time is sinusoidal. To incorporate this effect into our model, we convert the time of day to an angle t between 0 and 2π , corresponding to 0:00 GMT and 24:00 GMT respectively, and then encode it as two units, $\cos(t)$ and $\sin(t)$. This encoding scheme ensures that each time of day has a distinct representation and that the values for times around midnight are adjacent. Adding these time variables, we fit the following model:

$$T = \beta_0 + \beta_1 \frac{N_{pic} N_{tag}}{R} + \beta_2 \cos(t) + \beta_3 \sin(t) + \epsilon$$

	pay per tag	R	N_{pic}	N_{tag}	N_{hits}	N_{asst}	Posting Fees	Total Cost
‘low pay’ baseline	1/3¢	\$0.01	1	3	66	5	\$1.65	\$4.95
optimized for N_{unique} (low pay)	1/3¢	\$0.06	9	2	15	5	\$0.45	\$4.95
optimized for N_{GS} (low pay)	1/3¢	\$0.03	9	1	28	5	\$0.70	\$4.90
‘high pay’ baseline	1¢	\$0.04	1	4	22	5	\$0.55	\$4.95
optimized for N_{unique} , N_{GS} (high pay)	1¢	\$0.10	10	1	9	5	\$0.45	\$4.95

Table 4: Baseline and optimized designs for an image labeling tasks with a \$5 budget.

We observe an improvement in the fit, with $R^2 = 0.79$, $RRSE = 0.61$ for 100% completion time, and $R^2 = 0.70$, $RRSE = 0.70$ for 50% completion time; see Figure 4 for a comparison between the models’ predictions. This improvement is more significant for predicting 50% completion time (R^2 from 0.45 to 0.70) than for 100% completion time (R^2 from 0.68 to 0.79). One possible explanation is that the effect of the time of posting diminishes as the HIT posting begins to span a longer time frame and overlap into other times of the day.

The fit of these models suggests that the rate of pay and the time of posting are correlated with the completion time, but that there is still a substantial amount of unexplained variance. To use these models for prediction and design, it would be useful to consider not only the expected completion time, but also to be mindful of the variance in the prediction. Furthermore, the current models are only trained on configurations with 20 HITs, and do not incorporate the effect of varying the number of HITs. We leave exploring these directions for future work, and for now focus on using the quality prediction models for design.

5. DESIGN EXPERIMENT

The initial experiments provide us with an understanding of how worker output responds to different design, and provide the building blocks for effective task design. Even at the same level of desirability to workers—e.g., as measured by the pay per tag, or more generally, the estimated pay per hour—we expect some designs to induce more quality output than other designs. The learned models can thus help us make informed design decisions, for particular quality metrics we care about.

We demonstrate the potential benefit of optimizing designs based on the learned models by considering a simple design experiment in which we compare different designs at a fixed pay per tag. We focus our comparison on the number of quality labels received (per dollar spent), and do not concern ourselves with the rate at which work completes.⁷ Fixing the rate of pay allows us to compare designs based on the kind of work they request, and removes the effect of assigning more work at a lower rate of pay to get more quality labels from confounding the comparison.

We consider experiments at two pay rates: a low rate that pays $\frac{1}{3}$ ¢ per tag, and a high rate that pays 1¢ per tag. For each pay rate, we compare the output of *baseline designs* to designs optimized for each of our two quality metrics. Each design is given a budget of \$5, which must account for fees paid to Amazon as well as payments to workers. As in

⁷In practice, we can set the rate of pay based on how quickly we want work to get done. But since time is not considered in this experiment, fixing the rate of pay allows for a fair comparison between designs.

our initial experiments, the number of assignments per HIT (N_{asst}) is fixed at 5.

To optimize the task design, we choose values for the reward per HIT (R), number of images per HIT (N_{pic}), number of tags requested per image (N_{tag}), and the total number of HITs (N_{hits}) to maximize the total number of quality tags received as predicted by the model with the best fit, subject to budget and rate of pay constraints. We consider rewards in the range of \$0.01 and \$0.10 per HIT, and the number of images and tags requested per image in the range of 1 and 10, respectively. For example, the following formulation captures the optimization problem for finding a design that maximizes the total number of unique tags received as predicted by our model, subject to a \$5 budget and a pay rate of \$0.01 per tag:

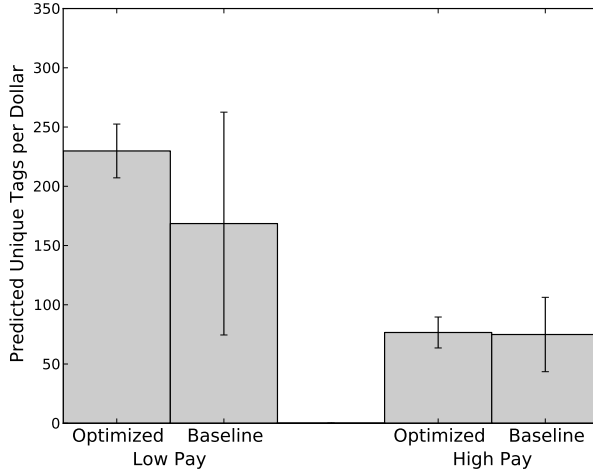
$$\max_{R, N_{pic}, N_{tag}, N_{hits}} 0.8426 N_{pic} N_{tag} N_{hits} N_{asst} N_{HIT} N_{asst} (R + \max(0.1R, 0.005)) \leq 5 \quad (1)$$

$$R/N_{pic} N_{tag} = 0.01 \quad (2)$$

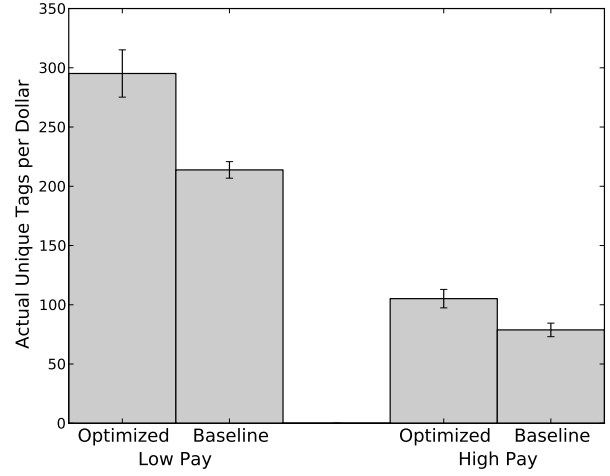
Constraint 1 ensures that the cost of the design stays within budget, and constraint 2 ensures that the pay per tag is \$0.01. The max term in the budget constraint corresponds to Mechanical Turk’s per assignment fees, which is 10% of the reward or half a cent, whichever is more.

Table 4 summarizes the baseline and optimized designs for both pay rates and quality metrics. For the low pay rate, we use as the baseline the same design that we had considered previously for measuring variability (but with more HITs). For maximizing the number of unique tags collected, we see that the optimized design attempts to save on posting fees by putting more work into a HIT and paying more per HIT, which allows for more tags to be requested. For maximizing the number of unique tags that are in the gold standard, the optimized design avoids diminishing returns by requesting 1 tag per image, and also saves on posting fees by putting more work in a HIT. For the high pay rate, we consider a baseline design that requests 4 tags for one image. Here the optimized designs for the two quality metrics are the same; more work is put into each HIT to save on posting fees (hitting the upper bound on reward per HIT), and only 1 tag is requested per image to avoid diminishing returns.

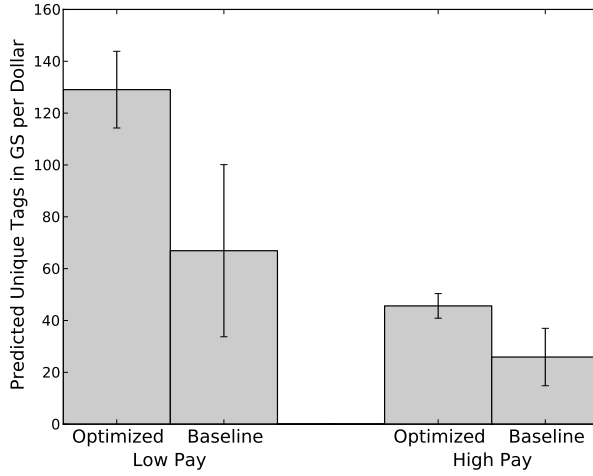
Figures 5(a) and 5(c) show the models’ predictions with bars representing the 95% prediction interval for these designs. We see that the difference in the predicted numbers of unique tags per dollar spent between baseline and optimized designs is small, since the benefits of the optimized design is only from savings in posting fees. By avoiding diminishing returns in tags, designs optimized for the numbers of unique tags that are in the gold standard are expected to perform significantly better.



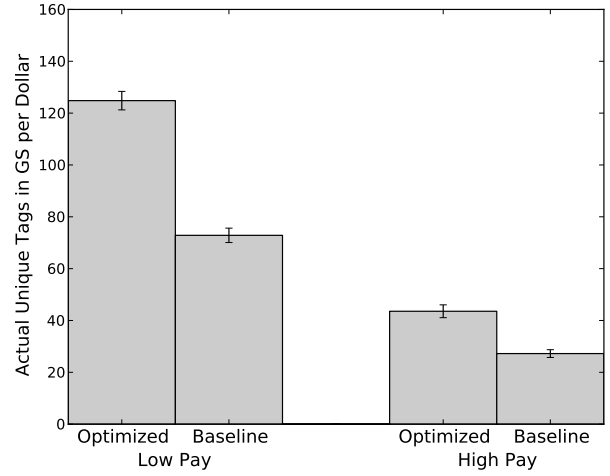
(a) Predicted number of unique tags per dollar spent.



(b) Actual number of unique tags per dollar spent.



(c) Predicted number of unique tags in gold standard per dollar spent.



(d) Actual number of unique tags in gold standard per dollar spent.

Figure 5: Predicted and actual number of quality tags received per dollar spent for baseline and optimized designs. Error bars in predictions indicate the 95% prediction intervals, and error bars in results represent the standard error over 5 runs of each design.

5.1 Results

We post 5 groups of each design in round-robin order. Each group was initially ran for 6 hours; if it didn't finish within 6 hours, it was resumed at a later time.⁸ Figures 5(b) and 5(d) show the average number of unique tags and unique tags in the gold standard received per dollar spent, with bars capturing the standard error of the mean.

In all comparisons, we find that the optimized models received more quality tags than baseline comparisons. The optimized designs for unique tags received 38% more tags in the low pay condition, and 33% more in the high pay condition. For collecting unique tags that are in the gold

⁸We initially posted the baseline designs between 3/25/10 and 3/29/10, and the optimized designs between 4/22/10 and 4/26/10. While almost all trials of the high pay configurations completed within this time frame, many of the low pay configurations did not; these configurations were ran to completion between 4/29/10 and 5/7/10.

standard, the optimized designs received significantly more quality tags than the baseline comparisons, with 71% more in the low pay condition and 60% more in the high pay condition. For all baseline and optimized designs, the actual number of gold standard tags received is very close to our model's predictions (within 11%), and well within the prediction intervals.

Interestingly, our optimized designs received significantly more unique tags than our models predicted, by 28% in the low pay condition and 38% in the high pay condition. One possible explanation is that our model underpredicts the number of unique tags when the number of tags requested per image is low as is the case in our designs. After checking the model's predictions on the training data, we notice that our model underpredicts for 10 out of the 11 configurations that request one or two tags per image (by 15% on average). Our model also underpredicted the number of unique tags obtained by the baseline in the low pay condition by 27%,

suggesting that the model may need to be refined to obtain more accurate predictions.

6. DISCUSSION

By learning about how workers respond to designs in our initial experiments, we are able to construct models that can accurately predict worker output in response to different designs. These models are then used to optimize the task's design, subject to designer constraints such as budget and rate of pay, to induce quality output from workers. Our results show that optimized designs generated by our models obtain significantly higher quality labels than baseline comparisons.

We are interested in extending the current work along a number of directions. We would like to understand the effect of distributing work across multiple assignments on the quality of output, and to include the number of assignments as a design variable. We are also interested in revisiting models for predicting the rate of work, and figuring out how to incorporate them to design with respect to time-related tradeoffs. One possible direction is to learn the relative rates at which work completes for different designs, which may be sufficient for accurately predicting the relative output between designs. Furthermore, while this work focuses on the design of a task with identical, parallel subtasks, we are also interested in the design of 'organizational hierarchies' [9] among workers, where workers can leverage each others' work and perform different roles in the task.

We believe the same approach of learning from observations of behavior to optimize designs can be effectively used to design other tasks, with respect to different performance metrics, and in richer design spaces. While linear regressions were used for this work, other modeling approaches and learning techniques can be similarly incorporated into the design process. Models of behavior need to be specific to the particular task and performance metric at hand; constructing accurate models will likely require drawing from social science theories, gaining an understanding of the task domain and the population of workers, and learning from experimentation.

In addition to accurate models, we need methods that help to discover effective designs after few experiments. While we trained our models on a set of manually picked designs and then used these models to design once, we can imagine a process that automatically picks subsequent experiments in a way that drives the search for better designs. One such method is suggested by Zhang et al. [14], which iteratively optimizes a design by selecting subsequent designs to experiment with based on the model's current predictions of the best design. Methods can also take into account the cost of exploration, and make tradeoffs between exploiting known designs and conducting more experiments. The development of such methods for task design—methods that automate the process of experimentation, learning, and optimization—presents an exciting area for future investigation.

Acknowledgments

The second author acknowledges support from the Department of Defense (DoD) through the National Defense Science & Engineering Graduate Fellowship (NDSEG) Program. In addition, research was sponsored by the Army Research Laboratory and was accomplished under Cooper-

ative Agreement Number W911NF-09-2-0053. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the Army Research Laboratory or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation here on.

7. REFERENCES

- [1] P. Dai, Mausam, and D. S. Weld. Decision-theoretic control of crowd-sourced workflows. In *the 24th AAAI Conference on Artificial Intelligence (AAAI'10)*, Atlanta, Georgia, 2010.
- [2] J. Horton and L. Chilton. The labor economics of paid crowdsourcing. *CoRR*, abs/1001.0627, 2010.
- [3] J. Horton, D. G. Rand, and R. J. Zeckhauser. The Online Laboratory: Conducting Experiments in a Real Labor Market. *SSRN eLibrary*, 2010.
- [4] P. G. Ipeirotis. Demographics of mechanical turk. *CeDER Working Papers*, 2010.
- [5] G. Little, L. B. Chilton, M. Goldman, and R. C. Miller. TurkIt: tools for iterative tasks on mechanical turk. In *KDD-HCOMP '09*, Paris, France, June 2009.
- [6] W. Mason and D. J. Watts. Financial incentives and the "Performance of Crowds". In *KDD-HCOMP '09*, Paris, France, June 2009.
- [7] J. Pontin. Artificial intelligence, with help from the humans. *The New York Times*, March 2007.
- [8] M. F. Porter. *An algorithm for suffix stripping*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1997.
- [9] H. A. Simon. The sciences of the artificial. *Cambridge, Ma: MIT Press*, 1969.
- [10] R. Snow, B. O'Connor, D. Jurafsky, and A. Y. Ng. Cheap and fast - but is it good? evaluating non-expert annotations for natural language tasks. In *the 2008 Conference on Empirical Methods in Natural Language Processing*, pages 254–263, Honolulu, October 2008.
- [11] Q. Su, D. Pavlov, J.-H. Chou, and W. C. Baker. Internet-scale collection of human-reviewed data. In *the 2007 International World Wide Web Conference*, Banff, Alberta, Canada, May 2007.
- [12] L. von Ahn and L. Dabbish. Designing games with a purpose. *Commun. ACM*, 51(8):58–67, 2008.
- [13] Wikipedia statistics, January 2010. <http://stats.wikimedia.org/EN/TablesWikipediaEN.htm>.
- [14] H. Zhang, Y. Chen, and D. C. Parkes. A general approach to environment design with one agent. In *the 21st International Joint Conference on Artificial Intelligence (IJCAI-09)*, Pasadena, CA, 2009.
- [15] H. Zhang and D. C. Parkes. Value-based policy teaching with active indirect elicitation. In *the 23rd AAAI Conference on Artificial Intelligence (AAAI'08)*, Chicago, IL, 2008.
- [16] H. Zhang, D. C. Parkes, and Y. Chen. Policy teaching through reward function learning. In *the 10th ACM Electronic Commerce Conference (EC'09)*, Stanford, CA, 2009.

Author Index

- | | |
|------------------------------|-----------------------------|
| Aras, Hidir ... 49 | Morton, Brandon ... 45 |
| Azenkot, Shiri ... 1 | Paritosh, Praveen ... 10 |
| Bederson, Benjamin ... 54 | Parkes, David ... 77 |
| Bell, Marek ... 34 | Popovic, Zoran ... 41 |
| Chalmers, Matthew ... 34 | Provost, Foster ... 64 |
| Chen, Yiling ... 77 | Resnik, Philip ... 54 |
| Chilton, Lydia ... 1, 68 | Rohwer, Paul ... 38 |
| Chu, Jonathan ... 60 | Ross, Joel ... 18 |
| Ferguson, John ... 34 | Sakrajda, Andrzej ... 30 |
| Gajos, Krzysztof ... 77 | Schmidt, Erik ... 45 |
| Goldman, Max ... 68 | Seemakurty, Nitin ... 60 |
| Haller, Andreas ... 49 | Silberman, M. Six 18 |
| Horton, John ... 1 | Smith, Adam ... 41 |
| Hsiao, Dun-Yu ... 41 | Snavely, Noah ... 41 |
| Hu, Chang ... 54 | Speck, Jacquelin ... 45 |
| Huang, Eric ... 77 | Stewart, Osamuyimen ... 30 |
| Huerta, Juan ... 30 | Takhtamysheva, Aneta ... 22 |
| Ipeirotis, Panagiotis ... 64 | Tomasic, Anthony ... 60 |
| Irani, Lilly ... 18 | Tomlinson, Bill ... 18 |
| Kim, Youngmoo ... 45 | Toomim, Michael ... 53 |
| Kochhar, Shailesh ... 10 | Tuite, Kathleen ... 41 |
| Krause, Markus ... 22, 49 | von Ahn, Luis ... 60 |
| Landay, James ... 53 | Wang, Jing ... 64 |
| Little, Greg ... 68 | Wang, Jun ... 56 |
| Lubensky, David ... 30 | Weng, Li ... 26 |
| Malaka, Rainer ... 22, 49 | Wittstock, Marion ... 22 |
| Marcotte, Julie ... 30 | Wu, Cheng ... 30 |
| Mazzocchi, Stefano ... 10 | Yu, Bei ... 56 |
| Menczer, Filippo ... 26 | Zhang, Haoqi ... 77 |
| Miller, Robert ... 1, 68 | |